Understanding Q-Learning and Deep Q-Learning in 2025: A Methodological and Empirical Survey

Abstract

Q-learning remains a cornerstone of Reinforcement Learning (RL), underpinning many state-of-the-art deep RL algorithms. Yet, despite its foundational status, no survey has systematically organized the diverse innovations that have shaped Q-learning over the past three decades. This paper fills that gap by presenting a comprehensive review of Q-learning variants, organized around the eight foundational weaknesses of vanilla Q-learning that modern methods address — overestimation bias, sample inefficiency, brittle exploration, reward sparsity and credit assignment, distribution shift, multi-agent coordination, slow adaptation, and function-approximation instability. We introduce a unified taxonomy covering both tabular and deep Q-learning methods, including offline RL, multi-agent value decomposition, and distributed Q-learning, and compile benchmark results from original papers across Atari and classic control environments. To complement the literature review, we provide a comparative analysis of six major open-source deep RL repositories — Tianshou, XuanCe, CleanRL, DQN Zoo, Stable Baselines3, and RLlib — highlighting their coverage of Q-learning algorithms and practical implementation challenges. Together, these contributions offer a structured resource for researchers and practitioners to navigate the evolution, empirical performance, and implementation landscape of Q-learning.

Contents

Impact Statement

Q-learning has influenced nearly every area of deep Reinforcement Learning, yet its own methodological evolution has lacked a focused and analytical treatment. This paper addresses that need by delivering a problem-first taxonomy of Q-learning methods, benchmarking results from original studies, and a comparative analysis of leading open-source implementations. By bridging theoretical advances with empirical evidence and practical repositories, this work clarifies the algorithmic landscape of Q-learning and highlights both underexplored areas and reproducibility challenges. The resulting insights are intended to guide the development of more robust, efficient, and extensible deep RL systems that build on Q-learning principles.

Index Terms: Deep Learning, Machine Learning, Reinforcement Learning, Q-Learning, Survey

I. Introduction

Reinforcement Learning (RL) and Deep Reinforcement Learning (DRL) have emerged as powerful paradigms for solving complex sequential decision-making problems, with applications ranging from Atari games [1], [2] and robotic control [3] to autonomous driving [4]. Among DRL approaches, two major families dominate: value-based methods, which estimate action-value functions, and policy-based methods, which directly optimize policy distributions.

While policy-based algorithms have shown strong performance in tasks with continuous action spaces, they often suffer from high variance and sensitivity to hyperparameters. In contrast, value-based methods such as Q-learning [5] and its deep variant DQN [1] remain widely used due to their conceptual simplicity, sample efficiency in discrete domains, and ease of implementation.

Q-learning's longevity has produced a sprawling methodological literature. Over three decades, dozens of variants have been proposed to address specific limitations — overestimation bias, brittle exploration, sample inefficiency, instability under function approximation — and many have been combined into composite agents such as Rainbow [28]. Despite this volume of work, the *organizing structure* through which the field is presented has remained largely chronological or method-typed: distributional methods, ensemble methods, replay innovations, and so on. Existing surveys [12]–[15], as well as more recent broad-RL treatments [Ghasemi 2024/2025], have inherited this method-type organization.

Method-type organization is bibliometrically convenient but analytically thin. It tells the reader *what kind of thing* a method is (an ensemble, a distributional model, a replay buffer modification) without surfacing *why* the method exists — which weakness of vanilla Q-learning it was designed to address, what trade-offs it introduces, and how it compares to other approaches that target the same weakness. As a result, readers encounter algorithms as isolated artifacts rather than as positioned moves in an ongoing technical conversation. Cross-method synthesis suffers accordingly.

This paper offers an alternative organization. Rather than presenting Q-learning's variants by method type, we organize the field around **the foundational weaknesses of vanilla Q-learning that modern methods address**. Eight such weaknesses are identified:

1. *Overestimation bias* introduced by the `max` operator over noisy value estimates;
2. *Sample inefficiency* arising from uniform experience replay;
3. *Brittle exploration* under ε-greedy action selection;
4. *Reward sparsity and credit assignment* in long-horizon tasks;
5. *Distribution shift* when transferring to fixed-data (offline) regimes;
6. *Multi-agent coordination* failures when factoring single-agent Q across cooperative agents;
7. *Slow adaptation* when training per-task without transfer; and
8. *Function-approximation instability* — the "deadly triad" of off-policy learning, bootstrapping, and function approximation.

Each Related Works section in this paper corresponds to one of these weaknesses. Within each section, methods are grouped by the *mechanism* they use to address the weakness, compared head-to-head on the trade-offs they introduce, and evaluated against the empirical evidence relevant to that axis. The same method can appear in multiple sections when it advances multiple axes — Rainbow [28], for instance, recurs across five of the eight — and these cross-references form an explicit map of the field's connective tissue.

This organization preserves all of the paper's empirical and bibliographic contributions while reframing the surrounding analysis. Five distinguishing contributions, summarized in Table I, support the reframing:

1. A unified problem-axis taxonomy that integrates tabular Q-learning, classical deep Q-learning, and modern Q-based methods (offline RL, multi-agent value decomposition, distributed scaling) within a single framework.
2. A comparative analysis of six widely used open-source DRL repositories — Tianshou [6], XuanCe [7], CleanRL [8], DQN Zoo [9], Stable Baselines3 [10], and RLlib [11] — annotated *by axis* to reveal where the open-source ecosystem provides coverage and where it does not.

3. Curated benchmark results extracted from the Atari [2] suite as reported in the original papers, stratified by task category, and re-interpreted as evidence for or against each algorithmic axis rather than as a leaderboard.
4. Tabular Q-learning benchmark results from our own controlled reimplementations, isolating algorithmic design from architectural confound.
5. A thorough per-paper literature review organized by axis, with each method positioned by the weakness it addresses, the mechanism it uses, and the trade-offs it introduces.

The remainder of the paper is organized as follows. Section II introduces the MDP formalism and formally states the eight weaknesses that structure the Related Works. Section III describes the methodology. Section IV — the bulk of the paper — presents the problem-first review across the eight axes (§IV.A–H), followed by a ninth subsection §IV.I covering recent theoretical advances. Sections V and VI report Atari and tabular benchmark analyses. Section VII presents the repository comparison. Section VIII concludes with future directions, including a planned community-maintained Q-learning repository whose roadmap is informed by the gaps identified in Section VII. Appendix A maps every method to both the problem-first axis taxonomy used in §IV and the conventional method-type taxonomy used by prior surveys; Appendix B consolidates notation conventions and supplies selected derivations.

Reading guide. The §IV body is written for mechanism and trade-off: each axis-section names the weakness, surveys the solution families that respond, and compares them on the costs they introduce. Equations in §IV are mechanism-defining rather than fully derived; longer derivations and proof sketches are concentrated in Appendix B and may be skipped on a first read without losing the structural argument. §IV.I revisits the same ground theoretically and references Appendix B throughout.

II. Background and Problem Setup

II.A. Markov Decision Processes

We consider the standard formulation of a Markov Decision Process (MDP), defined as $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, R, P, \gamma, d_0 \rangle$, where $\mathcal{S}$ denotes the set of all possible states, and $\mathcal{A}$ the set of possible actions. The reward function $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ assigns a real-valued reward to each state-action pair. The transition function $P(s' \mid s, a, \theta)$ specifies the probability of transitioning to state s' from state s after taking action a, parameterized by θ. The discount factor $\gamma \in [0, 1)$ determines the importance of future rewards, and d_0 denotes the initial state distribution.

A policy $\pi : \mathcal{S} \to \mathcal{A}$ (or, in the stochastic case, $\pi : \mathcal{S} \to \Delta(\mathcal{A})$) prescribes an action for each state. The value of a policy at state s is the expected discounted return under that policy: $V^\pi(s) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s]$. The associated action-value function is $Q^\pi(s, a) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a]$. Q-learning [5] estimates the optimal action- value $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ via the recursive Bellman optimality equation,

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot|s,a)} \Big[r + \gamma \max_{a'} Q^*(s', a') \Big],$$

implemented as an iterative update,

$$Q(s, a) \leftarrow Q(s, a) + \alpha \Big(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \Big).$$

The remainder of this paper surveys methods that modify, augment, or replace components of this update to address one or more of the weaknesses introduced below.

II.B. Eight Weaknesses of Vanilla Q-Learning

The Q-learning update of §II.A is provably convergent under tabular representation and standard stochastic approximation conditions [Watkins & Dayan 1992]. In practical settings — function approximation, partial coverage, sparse rewards, multi-agent environments, fixed-data regimes — the update exhibits eight distinct failure modes. These failure modes are not independent; several share underlying mechanisms. But each motivates a recognizably different family of methods, and each is the organizing principle of one Related Works subsection.

W1. Overestimation bias. The operator $\max_{a'} Q(s', a')$ is biased upward in expectation whenever $Q(s', \cdot)$ contains zero-mean noise. Formally, for noisy estimators $\tilde{Q}(s', a) = Q^*(s', a) + \epsilon_a$ with $\mathbb{E}[\epsilon_a] = 0$,

$$\mathbb{E}\Big[\max_{a'} \tilde{Q}(s', a') \Big] \geq \max_{a'} Q^*(s', a'),$$

with strict inequality whenever the noise is not degenerate [Thrun & Schwartz 1993]. Under recursive Bellman backups this bias propagates and amplifies, steering the greedy policy toward actions whose values are least accurately estimated. Methods responding to this weakness are surveyed in §IV.A.

W2. Sample inefficiency. Vanilla Q-learning uses each transition once and uniformly. The introduction of experience replay [32] allowed transitions to be reused, but uniform sampling treats all transitions as equally informative — a transition where the agent already predicts the outcome accurately contributes little to the update, yet is sampled as often as a high-error transition. In environments where high-information transitions are rare (sparse reward, low-probability state encounters), uniform replay yields slow convergence. Methods responding to this weakness — prioritized sampling, learning from demonstrations, memory-efficient replay — are surveyed in §IV.B.

W3. Brittle exploration. ε-greedy action selection takes a random action with probability ε and a greedy action otherwise. This suffices for environments where reward is sufficiently dense that near-greedy policies explore the state space through their own exploitation. In environments where rewards are sparse

or delayed beyond an ε-greedy random-walk's reach — Montezuma's Revenge [2], Pitfall! [2], Private Eye [2] — ε-greedy exploration is dithered rather than directed and fails to escape early-state plateaus. The formal characterization is that ε-greedy is myopic with respect to posterior uncertainty in $Q(s, a)$; methods responding to this weakness inject structured noise, maintain posterior estimates, or use intrinsic motivation, and are surveyed in §IV.C.

W4. Reward sparsity and credit assignment. When reward is received only at the end of a long trajectory, the one-step Bellman backup requires many iterations to propagate the signal to early states. Multi-step returns [30] partially mitigate this by allowing n-step bootstrapping:

$$y_t^{(n)} = r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n \max_{a'} Q(s_{t+n}, a').$$

But n-step returns trade variance for bias and do not address the deeper question of *what information* the return signal carries. Distributional RL [21] reframes the question: rather than estimating the expected return $\mathbb{E}[Z(s, a)]$, estimate the full distribution $Z(s, a)$. The distribution carries richer credit- assignment information — bimodality, skewness, tail behavior — that the scalar expectation discards. Methods responding to this weakness are surveyed in §IV.D.

W5. Distribution shift. Q-learning is off-policy *in principle*: the learned Q^* is independent of the behavior policy that generated transitions. In practice, the off-policy guarantee is fragile when the behavior policy is fixed and the support of the replay distribution is narrow — the regime of offline RL. The maximization $\max_{a'} Q(s', a')$ now ranges over actions a' that may be unsupported by the data, producing arbitrary extrapolation errors. The failure is structural: any algorithm that backs up through unsupported actions inherits unbounded error. Methods responding to this weakness constrain Q-estimates or actions to remain within the data support and are surveyed in §IV.E.

W6. Multi-agent coordination. When the environment contains multiple cooperating agents with a shared reward, naive per-agent Q-learning treats other agents as part of the environment — a non-stationarity that breaks the MDP assumption. Centralized Q-learning over the joint action space avoids this but scales exponentially: $|\mathcal{A}|^N$ for N agents. Value decomposition methods factor $Q_{\text{tot}}(\mathbf{s}, \mathbf{a}) = f(Q_1(s_1, a_1), \ldots, Q_N(s_N, a_N))$ under structural constraints on f (additivity, monotonicity) that preserve the *individual-global-max* property — that the action maximizing Q_{tot} jointly is the per-agent argmax of each Q_i. Methods responding to this weakness are surveyed in §IV.F.

W7. Slow adaptation. Vanilla Q-learning trains a single Q-function for a single task. Transfer to a related task — different reward, different transition dynamics, different state distribution — requires retraining from scratch or initialization from a pre-trained Q. Neither is satisfactory when adaptation must occur online and within a small number of episodes. Methods responding to this weakness include meta-learning approaches that learn an initialization or adaptation rule rather than a fixed Q, distributed actor-critic architectures that share experience across tasks, and architecturally recurrent variants that condition on task context. They are surveyed in §IV.G.

W8. Function-approximation instability. The combination of off-policy learning, bootstrapping, and function approximation — the *deadly triad* [Sutton & Barto 2018] — is provably unstable in the general case. Concretely, the iterates of $Q(s, a; \theta) \leftarrow Q(s, a; \theta) + \alpha(r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta))$ need not converge, may diverge, and even when they converge can do so to a point that is far from Q^*. Target networks, Polyak averaging, layer normalization, and architectural decomposition each address different aspects of this instability. Methods responding to this weakness are surveyed in §IV.H.

II.C. Notation conventions

Throughout the paper, s, a, r, s' denote a transition; θ and θ^- denote the parameters of an online and target network, respectively; π denotes a policy; α a learning rate; γ a discount factor; $\mathcal{D}$ a replay buffer; and ϵ either an exploration rate or a noise variable, disambiguated by context. Deviations from this convention are explicitly flagged in each section.

III. Methodology

This review surveys Q-learning algorithms spanning over three decades of development, from foundational tabular approaches to recent deep variants. We include influential papers that shaped the theoretical landscape, introduced practical improvements, or demonstrated wide applicability in real-world domains. The selection prioritizes methods whose contributions can be mapped onto one or more of the eight weakness axes introduced in §II.B — methods that respond to a recognizable weakness of vanilla Q-learning with a distinct mechanism.

A. Source selection

We surveyed the value-based RL literature published between 1989 and 2025, with primary attention to peer-reviewed venues (NeurIPS, ICML, ICLR, JMLR, *Nature*, IEEE journals) and to widely-cited preprints that have demonstrably shaped subsequent work. Five inclusion criteria were applied:

1. **Method centrality** — the work introduces a methodological contribution to the Q-learning family rather than applying Q-learning to a domain.
2. **Mechanism distinctness** — the contribution is mechanistically distinguishable from prior work along at least one of the eight axes of §II.B.
3. **Empirical validation** — the work reports results on a recognized benchmark (Atari, classic control, D4RL, SMAC, or equivalent).
4. **Cross-reference impact** — the work is cited as foundational by subsequent methods in the same axis-family.
5. **Reproducibility** — code is publicly available or the algorithm is sufficiently specified to be independently re-implemented.

Methods satisfying all five criteria received per-paper treatment. Methods satisfying a subset received compact treatment as part of a family (e.g. the dueling-mixing family in §IV.F is treated through its canonical members VDN, QMIX, QPLEX, QTRAN).

B. Comparison with prior surveys

To position this survey against existing work, we compiled Table I, which compares this paper with five prior Q-learning–focused surveys [12]–[15] and [Ghasemi 2024/2025] along five distinguishing dimensions: analysis of public DQN repositories, unified taxonomy spanning tabular and deep Q-learning, extraction of original-paper Atari benchmarks, classic control benchmarks from controlled re-implementation, and per-paper review organized around mechanism and axis. Where prior surveys focus on chronological or method-type organization, this paper's problem-first axis structure (§IV) is, to our knowledge, the first such treatment of Q-learning specifically. The closest prior art is the single-axis problem-first organization of [Springer NCAA 2026], which addresses distribution shift in offline RL only.

C. Axis assignment methodology

Each method covered in this paper is assigned to a *primary axis* — the weakness whose response motivated the method's introduction — and zero or more *secondary axes* — additional weaknesses the method incidentally addresses or partially mitigates. Primary assignment is made on the basis of the method's stated motivation in its original publication and the predominant mechanism of its contribution. Where the original publication's motivation differs from our axis assignment (notably for distributional methods in §IV.D and Dueling DQN in §IV.H), the reinterpretation is argued explicitly within the section.

Cross-references between axis-sections — e.g., Rainbow appearing in §IV.B (primary) and being cross-referenced from §IV.A, §IV.C, §IV.D, and §IV.H — surface the connective tissue that catalogue-style surveys obscure. Appendix A provides a legacy indexer mapping each method to both the problem-first axis assignment and the prior method-type taxonomy, supporting readers who arrive expecting the older organization.

Table 1: Comparison of Q-Learning and Deep Q-Learning Survey Papers.
• = discussed; ◦ = not discussed; partial = *partial*.

Aspect	Urtans 2018 [12]	Jang 2019 [13]	Boppiniti 2021 [14]	Hafiz 2022 [15]	Ghasemi 2024/25	C
Analyzes Public DQN Code Repositories	◦	◦	◦	◦	◦	
Unified Taxonomy Covering Both Tabular Q and DQN	◦	◦	◦	◦	*partial*	
Atari Benchmark Results Extracted from Prior DQN Papers	◦	◦	◦	◦	◦	
Classic Control Benchmark Results from Original Implementations	◦	◦	◦	◦	◦	
Thorough Per-Paper Literature Review	◦	◦	◦	◦	*partial*	
Problem-First Organization Across Multiple Axes	◦	◦	◦	◦	◦	

D. Empirical evidence sources

Section V analyzes Atari benchmark results extracted from the original papers introducing each method. Section VI reports tabular benchmarks from controlled re-implementations on Gymnasium environments (FrozenLake-v1, Taxi-v3, CliffWalking-v1). Section VII analyzes algorithmic coverage across six widely-used open-source deep RL repositories (Tianshou [6], XuanCe [7], CleanRL [8], DQN Zoo [9], Stable Baselines3 [10], RLlib [11]). The three evidence streams together support different aspects of the paper's contribution: the literature analysis demonstrates methodological diversity, the controlled experiments isolate algorithmic from architectural effects, and the repository analysis maps the practical landscape of available implementations.

IV. Related Works

Over the past three decades, Q-learning and its deep variants have evolved through a long sequence of mechanism-level innovations. Section II.B identified eight foundational weaknesses of vanilla Q-learning that motivate the field's trajectory; this section surveys the methods that respond to each. The organization is problem-first: each subsection corresponds to one weakness, groups responses by mechanism, and compares them on the trade-offs they introduce.

This subsection orients the reader through three artifacts: a method genealogy showing the inheritance relationships between Q-learning's variants, a companion figure showing the modern-RL branches that have emerged outside the conventional taxonomy, and an axis × mechanism-family matrix summarizing what mechanism types are deployed against each weakness. The §IV subsections that follow each provide a five-part structure: formal statement of the weakness (subsection A), grouped solution families (subsection B), trade-offs (subsection C), empirical evidence (subsection D), open questions (subsection E), and a comparison summary with positioning grid (subsection F).

A. Method genealogy

Figure 1 shows the genealogy of Q-learning methods covered in this paper. Nodes are methods; edges are inheritance relationships labeled by *the weakness of the parent that the child method addresses*. The annotations on the edges are themselves a contribution: they trace the field's evolution as a sequence of targeted responses rather than a chronological accumulation of techniques.

Two structural observations emerge from the genealogy. First, **Rainbow [28] is the integration point** for five separate axis responses (Double DQN, PER, Dueling, multi-step, distributional, NoisyNet) — it is not a single method but a deliberate composition across axes. Second, **the distributional family** (C51 → QR-DQN → IQN → FQF) constitutes an unusually long single-axis lineage compared to most other branches, reflecting the field's progressive refinement of distributional Q-learning under the credit-assignment weakness (W4).

B. Branches outside the conventional taxonomy

Figure 2 below shows three Q-learning branches that emerged outside the conventional six-category mechanism taxonomy — families that respond to weaknesses (distribution shift, multi-agent coordination, distributed scale and meta-adaptation) that the legacy taxonomy has no category to hold. These are the methods that motivate §IV.E, §IV.F, and §IV.G.

The visual absence of these branches from prior Q-learning surveys [12]–[15] is one of the most direct empirical arguments for the structural pivot of §IV. The legacy taxonomy classifies methods by mechanism type; these branches require new mechanism categories (constrained policies, value decomposition, distributed actors, meta-learning) that did not exist when the original taxonomy was conventionalized.

C. Axis × mechanism-family matrix

The matrix below summarizes which mechanism families address which axis. Filled cells indicate a primary contribution; open cells (◦) indicate an incidental or secondary contribution; blanks indicate the axis is not addressed by that mechanism family.

Axis	Decoupling	Ensembles	Architectural decomposition	Behavior / loss modulation	Distributed / parallel
W1 Overestimation (§IV.A)	DoubleQ, DDQN	EBQL, REDQ	Dueling (rel.) ◦	—	—
W2 Sample inefficiency (§IV.B)	—	—	—	PER, DQfD, MeDQN, HER	Ape-X (replay) ◦

Axis	Decoupling	Ensembles	Architectural decomposition	Behavior / loss modulation	Distributed / parallel
W3 Brittle exploration (§IV.C)	—	Bootstrapped, UCB-Q	NoisyNet, ParSpaceNoise	CBDQ, PSDQN, RND, Go-Explore	Agent57 (portfolio)
W4 Reward sparsity (§IV.D)	—	—	Distributional family	n-step, λ-returns	—
W5 Distribution shift (§IV.E)	BCQ	EDAC	—	CQL, IQL, BRAC, AWAC	—
W6 Multi-agent coord. (§IV.F)	—	—	VDN, QMIX, QPLEX, QTRAN	—	—
W7 Scaling / adaptation (§IV.G)	—	—	DRQN, R2D2	—	Ape-X, Agent57, Meta-Q
W8 Function-approx stability (§IV.H)	—	—	Target net, Dueling, PQN	MeDQN consol., Munchausen	—

Several patterns are visible at a glance:

- The **architectural decomposition** column is the most populous — most axes have at least one architectural response. This reflects the field's strong preference for *structural* solutions (modifying what the network represents) over *training-procedure* solutions (modifying what the loss optimizes).
- The **distributed** column is sparse but increasingly populated by frontier agents (Ape-X, Agent57). The pattern suggests scaling-as-mechanism is undercovered in the literature relative to its empirical importance — a point the open-questions subsection of §IV.G develops.
- The **decoupling** column has only two entries (DoubleQ-derived). Decoupling is mechanistically narrow; ensembling subsumes most of its effect once compute budgets permit $K > 2$ Q-functions.
- The **bottom-left quadrant** of the matrix (W6, W7 $\times$ decoupling or ensembles) is empty. Whether this reflects a true gap in the literature or merely a labeling artifact is an open question flagged in §VIII.

D. Section roadmap

The remainder of §IV proceeds axis by axis:

- **§IV.A** Overestimation bias — Double Q-learning to ensemble bias control (EBQL, REDQ).
- **§IV.B** Sample inefficiency — prioritized replay, demonstrations, memory-efficient consolidation, hindsight relabeling.
- **§IV.C** Brittle exploration — noise injection, ensemble disagreement, belief modulation, intrinsic motivation, archival exploration.
- **§IV.D** Reward sparsity and credit assignment — multi-step returns and the distributional family.
- **§IV.E** Distribution shift (offline RL) — policy constraint, value penalty, expectile regression, ensemble diversification.
- **§IV.F** Multi-agent coordination — additive, monotonic, duplex-dueling, and constraint-free value decomposition.
- **§IV.G** Scaling and slow adaptation — distributed actor-learner architectures, bandit-controlled exploration meta-policies, meta-learning, recurrence.

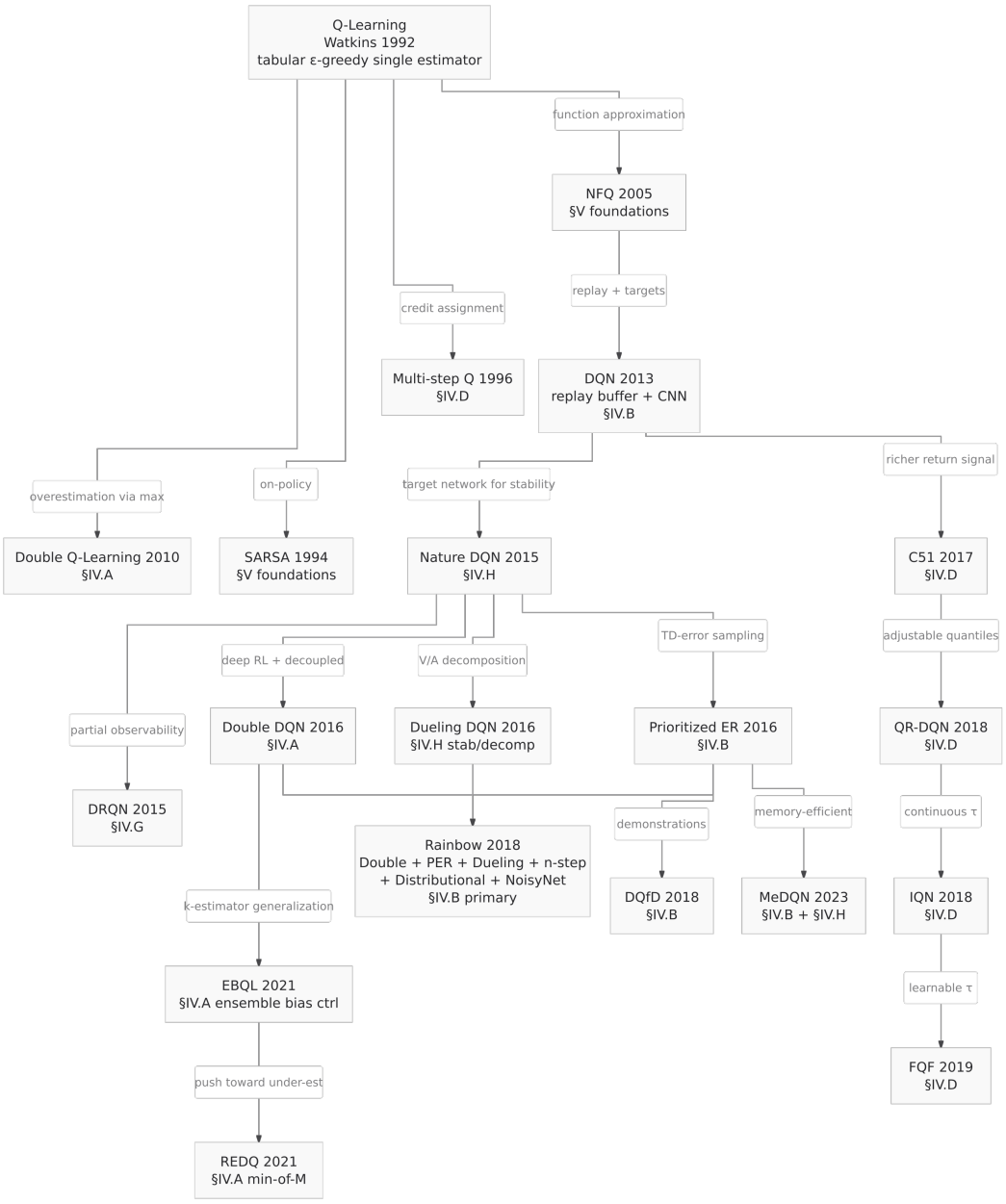


Figure 1: Method genealogy of Q-learning and its deep variants. Edges are labeled by the weakness of the parent that the child method addresses.

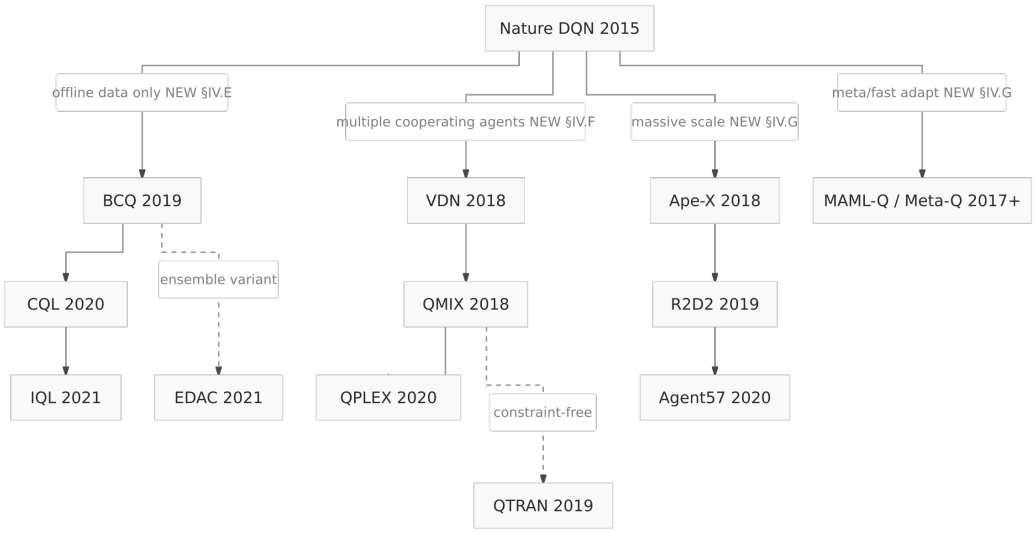


Figure 2: Q-learning branches outside the conventional six-category mechanism taxonomy — offline RL (§IV.E), multi-agent value decomposition (§IV.F), and distributed / meta-learning (§IV.G).

- **§IV.H** Function-approximation instability — target networks, architectural decomposition, normalization recipes, consolidation losses.
- **§IV.I** Theoretical foundations and recent advances — convergence theory, finite-time bounds, pessimism in offline RL, stability theory, and IGM theorems. A meta-section that surveys the theoretical landscape underlying the eight axes above.

Each axis subsection (A–H) ends with **F. Comparison summary**: a uniform six-column table (method / mechanism category / mechanism / cost / best-at / empirical anchor) and, where the trade-off is naturally two-dimensional, a 2D positioning grid. §IV.E additionally provides a method-selection decision tree for practitioner use. §IV.I substitutes a results-by-year theoretical-status table for the comparison grid since its entries are theorems rather than methods.

IV.A. Overestimation Bias

A. The Weakness

The standard Q-learning update,

$$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a)),$$

uses the operator $\max_{a'} Q(s',a')$ to estimate the value of the next state. When $Q(s',\cdot)$ contains noise — whether from sampling, function approximation, or partial coverage — this operator is *systematically biased upward*. The maximum of noisy estimators has expectation strictly greater than the maximum of their true means (Smith & Winkler, 2006); applied recursively through the Bellman backup, this bias propagates and amplifies.

The consequence is not merely cosmetic. Overestimation steers the greedy policy toward actions whose values are *least accurately estimated*, not toward actions that are genuinely best. In tabular settings the effect is bounded by $\mathcal{O}(\sqrt{\log|A|/N})$; in deep Q-learning with neural approximation it can compound indefinitely.

This is the canonical example of how a property reasonable in expectation (*"act greedily"*) becomes harmful in finite-sample practice. Methods that target this axis trade structural complexity for a more honest value estimate.

B. Solution Families

Three families of approach have emerged, distinguished by *what information they exploit to debias the maximum*.

B.1. Two-estimator decoupling (Double Q-learning, Double DQN). The earliest fix, introduced by [Hasselt 2010], maintains two independent Q-tables Q_A and Q_B. At each update, one is randomly chosen to be updated, with its target computed using the *other* table:

$$Q_A(s,a) \leftarrow Q_A(s,a) + \alpha(r + \gamma Q_B(s', a^*) - Q_A(s,a)), \quad a^* = \arg\max_{a'} Q_A(s',a').$$

The decoupling works because the action selected by Q_A does not share noise with the value used to evaluate it. The bias does not vanish — it can become slightly *negative* (under-estimation) — but it is no longer driven by the maximization itself.

Double DQN [Hasselt et al. 2016] applies the same idea inside the deep-RL recipe, using the *online* network for action selection and the *target* network for evaluation:

$$y = r + \gamma\, Q(s', \arg\max_{a'} Q(s',a';\theta);\, \theta^-).$$

This adds no additional networks beyond the target network DQN already maintains, which is a substantial part of its adoption: the cost is near-zero.

B.2. Ensemble-mediated bias control (EBQL, REDQ). Ensemble methods generalize the two-estimator idea to K Q-estimators and tune the bias-variance trade-off explicitly.

Ensemble Bootstrapped Q-Learning (EBQL) [Peer et al. 2021] maintains K Q-networks; at each step it samples a single member k_t for update and uses the *average of the remaining $K-1$ members* to evaluate the next-state action chosen by member k_t:

$$Q_{k_t}(s,a) \leftarrow (1-\alpha) Q_{k_t}(s,a) + \alpha(r + \gamma\, Q_{\,k_t}(s', \hat{a}^*)).$$

Increasing K smoothly moves between Q-learning ($K = 1$, over-estimation) and Double DQN ($K = 2$, slight under-estimation). Empirically, $K \in [5, 10]$ balances the two.

REDQ [Chen et al. 2021] pushes this further: it takes the *minimum* of $M \leq K$ randomly selected ensemble members at each update step. Taking the min instead of the average creates explicit under-estimation pressure, which the authors argue is preferable when the optimizer is biased toward high-Q regions.

The ensemble approach has a second virtue: the per-member disagreement is itself a usable uncertainty signal for exploration (see §IV.C), making the same architecture do double duty across axes.

B.3. Architectural decomposition (Dueling DQN). A separate line of work decomposes the Q-function into a state-value baseline $V(s)$ and an advantage $A(s, a)$:

$$Q(s, a) = V(s) + \Big(A(s, a) - \tfrac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')\Big).$$

[Wang et al. 2016] motivate dueling primarily as an architecture for states where action choice is unimportant: $V(s)$ can be learned from *all* transitions through s, while $A(s, \cdot)$ only updates on transitions involving each action.

The connection to overestimation is indirect but real: by re-routing most of the learning signal through $V(s)$, the dueling architecture reduces the *relative* noise in the action-conditional component, and therefore the bias introduced by the max. In Rainbow [Hessel et al. 2018], dueling contributes less to performance than PER or distributional learning, but it does not *hurt*, which is consistent with this re-interpretation: it is a stability/normalization contribution, not a value-estimation one. (See also §IV.H.)

C. Trade-offs

Every fix on this axis trades against one or more of the following. We list the trade-offs explicitly rather than presenting each method as strictly superior to its predecessors:

- **Bias vs. variance.** Double DQN trades a *small* under-estimation for the removal of a *large* over-estimation. EBQL with $K = 5{-}10$ is calibratable; REDQ's min deliberately pushes in the under-estimation direction. None of these is uniformly best; the right choice depends on whether the downstream policy is more sensitive to over- or under-estimation, which in turn depends on the action-space geometry.
- **Compute and memory.** Double DQN is free (uses the target network DQN already has). EBQL adds $K\times$ network forward passes per update and $K\times$ parameter storage. REDQ adds the same plus the min operation.
- **Sample efficiency on benign tasks.** On environments where overestimation is not the bottleneck (dense-reward Atari games like Breakout or Boxing), the more aggressive debiasing methods provide little benefit and the added compute cost dominates. EBQL's authors acknowledge that on most of the 11 Atari games they evaluated, the ensemble's benefit is moderate; the gain is concentrated on environments where standard DQN was previously sub-optimal.
- **Interaction with off-policy data.** All of the methods discussed here assume an online interaction loop. In offline RL (§IV.E), overestimation manifests differently — as out-of-distribution action selection — and requires different fixes (CQL, IQL). The online-fix toolkit transfers imperfectly to the offline setting.

D. Empirical Evidence

We summarize the available per-game Atari scores for the methods in this section, drawn from Tables II and III. Two qualitative patterns emerge:

On *dense-reward, reaction-time* games (Breakout, Space Invaders, Boxing, Enduro), the gap between Q-learning and its debiased variants is modest. Double DQN improves on Nature DQN by 0–20% on most

of these games; Dueling and Rainbow open larger gaps but their contributions are entangled with PER, distributional, and multi-step learning. EBQL's advantage on these games is modest and within Rainbow's range.

On *strategic-planning* and *sparse-reward* games (Q*bert, Montezuma's Revenge, Pitfall!), the picture is sharper but messier. Q*bert is where QR-DQN's overestimation control (achieved via distributional learning rather than ensembles) shows its largest single-game lead: it reaches 572,510 in the original paper — over 25× the next-best non-distributional method. Montezuma and Pitfall, by contrast, are *not* solved by overestimation control alone: Rainbow's 384 on Montezuma is its single ablation-anomalous result, and most overestimation-control methods sit at or near 0. This is consistent with the framing of §IV.C: exploration is the bottleneck on those games, not value estimation, and the methods of this section cannot substitute for directed exploration.

The dashes in Tables II and III are themselves evidence: methods that do not report Montezuma's Revenge (EBQL, Ensemble Bootstrapping) cannot be argued to solve the exploration problem on the basis of ensemble disagreement signals. We return to this in §IV.C.

E. Open Questions

Three questions on this axis remain underexplored:

1. **The bias-variance Pareto frontier is unmapped.** We have characterizations of single points — Double DQN (lightly under-biased), EBQL with $K = 5$ (calibratable), REDQ-min-of-2 (heavily under-biased) — but no systematic study of the frontier as a function of K, M, and update rate. A modest empirical contribution would be a per-environment scan over (K, M) with the resulting bias measured directly.
2. **The dueling re-interpretation is a hypothesis.** We argue above that dueling's contribution is partially overestimation control via re-routed learning signal. The Rainbow ablation is consistent with this but does not isolate it. A targeted ablation — dueling architecture + standard Q-learning (no other Rainbow components), measured against vanilla DQN with matched parameter count — would test the claim directly.
3. **Transfer to offline RL is incomplete.** The methods of this section were developed for the online interaction loop. Their offline-RL analogs (CQL, IQL — see §IV.E) take a different approach to the same underlying bias, penalizing out-of-distribution actions rather than averaging over noise. The theoretical relationship between these two families is currently ad hoc; a unifying framework that recovers both as limits of a single inequality would clarify the field considerably.

F. Comparison summary

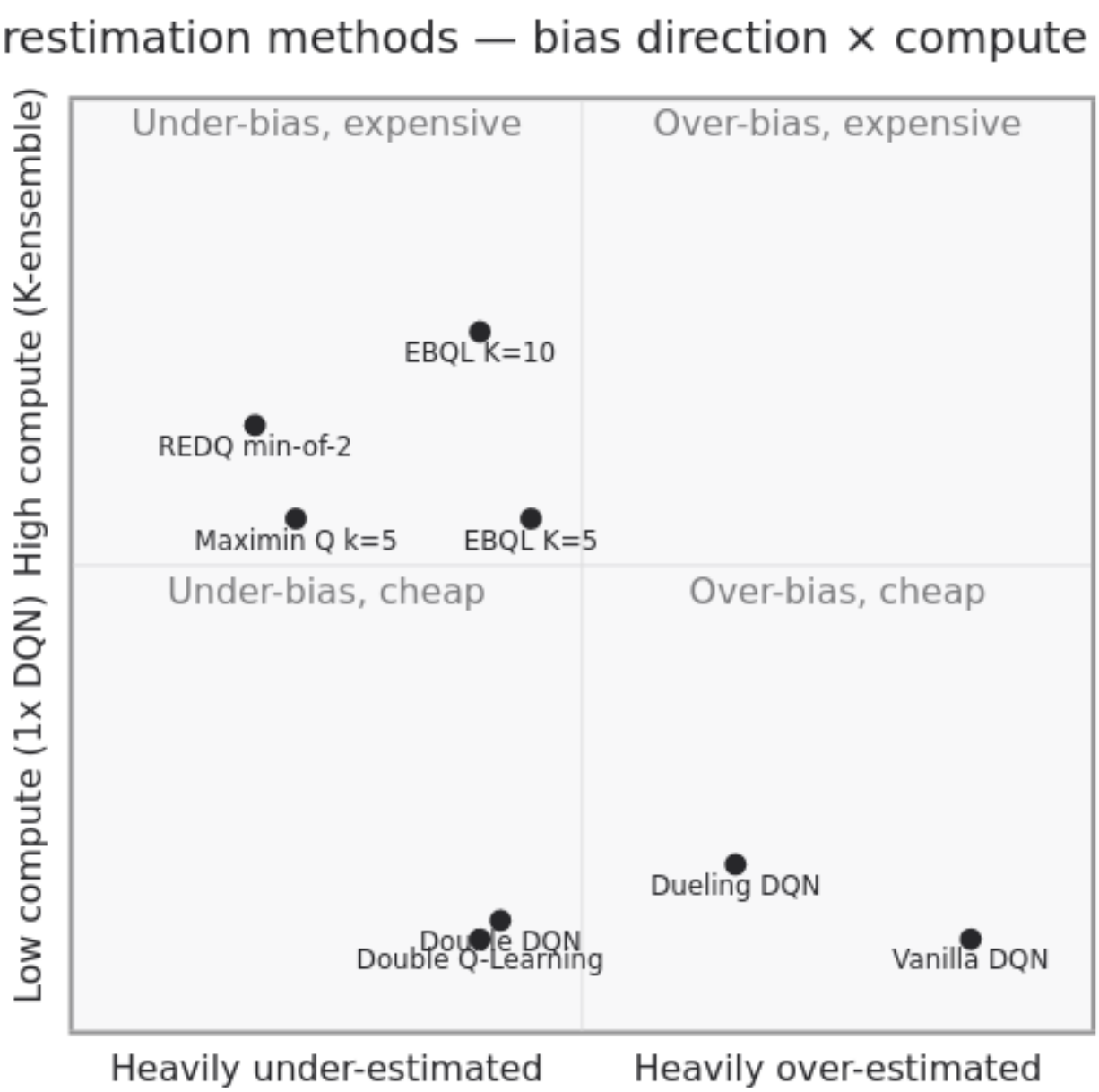
The methods of this section, summarized in one table:

Method (year)	Legacy category	Mechanism	Primary cost	Best at	Empirical anchor
Double Q-Learning (2010)	Q-Function Comp.	Two estimators, swapped action/eval	None vs. vanilla Q	Tabular discrete MDPs	GridWorld: lower bias, higher reward
Double DQN (2016)	Q-Function Comp.	Online net selects, target net evaluates	Free (uses existing target net)	Most Atari games	Q*bert: 14,875 (vs. 10,596 DQN)
Dueling DQN (2016)*	Q-Function Comp.	V(s) + centered A(s,a) decomposition	Modest architectural	Many-action states	Atari median: incremental
EBQL (2021)	Ensemble-Based	K-ensemble, target = avg of $K-1$ others	$K\times$ compute, $K\times$ memory	Bias-variance calibration	11 Atari > Double DQN

Method (year)	Legacy category	Mechanism	Primary cost	Best at	Empirical anchor
Maximin Q (2020)	Q-Function Comp.	min over k Q-estimators	$k\times$ compute	Pure under-estimation pressure	Tabular MDPs
REDQ (2021)	Ensemble-Based	min of M random ensemble members	$K\times$ compute, deliberately under-biased	Continuous-control SAC	MuJoCo locomotion

*Dueling DQN's primary section is §IV.H (stability); listed here as incidental contributor to bias control.

2D positioning (bias × cost):



The lower-right (cheap, over-biased) is the unmitigated baseline. The upper-left (expensive, under-biased) is the explicit under-estimation pressure of REDQ and Maximin. Double DQN sits in the lower-middle: cheap and lightly under-biased — the canonical "free improvement" point. EBQL fills the middle band: calibratable bias at proportional compute cost.

IV.B. Sample Inefficiency

Addresses **W2** of the eight weaknesses introduced in §II.B. Methods here modify *how transitions are stored, sampled, or augmented* to extract more learning signal per environment interaction.

A. The Weakness

Vanilla Q-learning consumes each transition exactly once. Experience replay [32] relaxed this constraint by storing transitions in a buffer $\mathcal{D}$ and sampling minibatches for repeated use:

$$\theta \leftarrow \theta - \alpha \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\nabla_\theta \big(Q(s,a;\theta) - y\big)^2\Big], \quad y = r + \gamma \max_{a'} Q(s',a';\theta^-).$$

Uniform sampling, however, treats every transition as equally informative. Transitions where the agent's current prediction is already accurate contribute little to the gradient yet are sampled with the same frequency as transitions whose temporal-difference (TD) error is large. In environments where high-information transitions are rare — sparse-reward games, low-probability state encounters, narrow-margin decision points — uniform replay leaves learning bottlenecked on the rate at which those transitions reappear in the sample.

Methods in this section modify the replay distribution, the buffer contents, or both, to concentrate learning capacity where the gradient is most useful.

B. Solution Families

Four mechanisms recur across the literature, distinguished by *what they manipulate*: which transitions get sampled, what additional transitions get stored, how the buffer is summarized when memory is limited, and how reward density itself can be increased.

B.1. Prioritized sampling. Prioritized Experience Replay (PER) [24] samples transitions in proportion to their TD-error magnitude:

$$P(i) = \frac{p_i^\beta}{\sum_k p_k^\beta}, \quad p_i = |\delta_i| + \epsilon,$$

where δ_i is the TD error of transition i and ϵ a small constant ensuring nonzero probability. PER offers two variants: proportional ($p_i = |\delta_i| + \epsilon$) and rank-based ($p_i = 1/\text{rank}(i)$). Non-uniform sampling introduces bias, which PER corrects via importance-sampling weights $w_i = (1/(N \cdot P(i)))^\beta$, annealed toward 1 as training progresses.

PER is one of the two largest contributors to Rainbow's [28] performance, alongside multi-step learning. Its ablation removal produces the steepest performance drop of any Rainbow component.

B.2. Demonstration augmentation. Deep Q-learning from Demonstrations (DQfD) [40] augments the replay buffer with expert trajectories prior to environment interaction. During an initial pre-training phase, the loss combines four terms:

$$\mathcal{L}_{\text{DQfD}} = \mathcal{L}_{\text{1-step}} + \mathcal{L}_{n\text{-step}} + \mathcal{L}_{\text{sup}} + \mathcal{L}_{L^2},$$

where $\mathcal{L}_{\text{sup}}$ is a large-margin classification loss encouraging the network to assign higher Q-values to actions demonstrated by the expert. After pre-training, the demonstration transitions remain in the buffer alongside agent-collected experience.

DQfD trades exploration cost for demonstration cost: it requires expert trajectories but produces substantially better initial policies, achieving state-of-the-art on 11 of 42 Atari games at the time of publication.

B.3. Memory-efficient consolidation. Memory-Efficient DQN (MeDQN) [41] addresses a different bottleneck: replay buffer storage. Standard DQN on Atari requires a 7GB buffer; MeDQN compresses this

to ~0.7GB by introducing a consolidation loss that distills past Q-values from a target network into the current network:

$$\mathcal{L}_{\text{V-consolid}} = \mathbb{E}_{(s,a)\sim p(\cdot,\cdot)}\Big[(Q(s,a;\theta) - \hat{Q}(s,a;\theta^-))^2\Big],$$

added to the standard DQN loss with weight λ. Two sampling schemes for the consolidation distribution $p(s,a)$ exist: MeDQN(U) samples from a uniform approximation of the state space; MeDQN(R) samples from a small auxiliary replay buffer of past states.

MeDQN(R) matches or exceeds DQN performance on five selected Atari games while reducing memory tenfold. Its consolidation loss also serves a stability function (see §IV.H) — the same mechanism prevents catastrophic forgetting and damps function-approximation drift.

B.4. Goal relabeling. Hindsight Experience Replay (HER) [Andrychowicz et al. 2017] applies only to goal-conditioned MDPs but yields large sample-efficiency gains where it does. After collecting a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_T)$ under goal g, HER stores not only $(s_t, a_t, r_t, s_{t+1}, g)$ but also relabeled transitions $(s_t, a_t, r'_t, s_{t+1}, g')$ where $g' = s_T$ (or any visited future state) and r'_t is recomputed under g'. The trajectory that failed under the original goal becomes a successful trajectory under a synthetic goal, densifying the reward signal at no environmental cost.

HER is the method that unlocked sample-efficient Q-learning for sparse-reward robotic manipulation tasks and represents one of the largest single sample-efficiency advances of the past decade.

C. Trade-offs

- **Sampling bias vs. signal concentration.** PER's importance sampling correction is asymptotically unbiased but introduces variance, particularly early in training when $\beta < 1$. The bias-variance trade-off is controlled by the prioritization exponent α and the importance-sampling annealing schedule for β.
- **Demonstration availability.** DQfD's advantage scales with demonstration quality and coverage. In domains where expert policies are unavailable, expensive to obtain, or reward-misaligned, the framework cannot be applied.
- **Memory vs. compute.** MeDQN trades buffer storage for the compute cost of the consolidation loss. The trade-off is favorable on Atari-scale tasks (7GB → 0.7GB at modest compute increase) but the consolidation loss adds a hyperparameter λ requiring tuning per domain.
- **Goal-conditioning requirement.** HER applies only to MDPs whose reward function decomposes naturally over goals. Reward sparsity in non-goal-conditioned settings is not addressed.

D. Empirical Evidence

PER outperforms uniform-replay DQN on 41 of 49 Atari games in the original evaluation [24], with the largest gains on games where high-error transitions are rare. Improvements concentrate on strategic-planning games (Q*bert, Ms. Pac-Man) rather than reaction-time games (Breakout, Space Invaders), consistent with the mechanism: dense-reward games already sample informative transitions frequently enough.

DQfD achieves state-of-the-art performance on 11 of 42 Atari games at publication, with the largest absolute gains on demonstration-amenable games (Q*bert, River Raid, Hero — see Tables II–III). Notably, DQfD reaches 42,457 on Private Eye — the largest single result by any method in Table III on that game, unmatched by any subsequent non-demonstration method. This is consistent with the framing of §IV.C: Private Eye's bottleneck is exploration, and demonstration substitutes for it.

MeDQN's empirical evidence on Atari is restricted to five games but matches DQN performance at roughly 10% of the memory budget. The trade-off is favorable for any setting where replay memory is a binding constraint, including distributed training (see §IV.G).

HER's Atari results do not appear in Tables II–III because the Arcade Learning Environment is not goal-conditioned. The relevant empirical evidence is on robotic manipulation benchmarks (OpenAI Fetch, Push,

Slide, Pick-and-Place), where HER agents achieve near-100% success on tasks that vanilla DQN cannot learn from a single demonstration.

E. Open Questions

Three questions on this axis remain open:

1. **Replacement policy.** First-in-first-out (FIFO) is the default for finite replay buffers but is rarely justified empirically. When the buffer fills, FIFO discards the oldest transitions — which include the rare early-training successes that contain the highest information density. Adaptive replacement policies that retain transitions by learning utility, novelty, or persistent high TD error are largely unexplored. A simple adaptive policy could plausibly close a substantial fraction of the gap to infinite-buffer methods.
2. **Transfer of prioritized sampling to offline RL.** PER's correction relies on importance-sampling weights computed against the *current* replay distribution. In offline RL (§IV.E), the replay distribution is fixed and the gradient direction is constrained by support; whether non-uniform sampling helps or hurts in that regime is not settled, and per-method offline results have been mixed.
3. **Goal relabeling beyond Cartesian goals.** HER's relabeling trick depends on the reward function being computable from (s, a, g) alone. For reward functions involving full trajectory features (sparse safety constraints, multi-step economic reasoning), the relabeling space is not obviously parameterizable. General-purpose goal relabeling for non-decomposable rewards would substantially expand the method's applicability.

F. Comparison summary

Method (year)	Legacy category	Mechanism	Primary cost	Best at	Empirical anchor
DQN replay buffer (2013)	Memory/Replay	Uniform sampling from buffer	Memory	Off-policy stability baseline	Atari first DQN
PER (2016)	Memory/Replay	Sample $\propto \lvert\delta_i\rvert^\alpha$, IS-corrected	IS bias, prioritization α	Rare high-info transitions	41/49 Atari > DQN
DQfD (2018)	Memory/Replay	Augment buffer with expert demonstrations	Demonstration availability	Sparse-reward Atari	Private Eye: 42,457
MeDQN (2023)	Memory/Replay	Consolidation loss compresses buffer	λ hyperparameter	Memory-constrained training	Atari 7GB → 0.7GB
HER (2017)	Memory/Replay	Goal relabeling for synthetic reward	Goal-conditioned only	Robotic manipulation	OpenAI Fetch $\approx 100\%$

The methods on this axis do not occupy a clean 2D trade-off; the relevant trade-offs are categorical (memory vs. compute vs. demonstration availability vs. environment structure). The comparison table alone suffices.

IV.C. Brittle Exploration

Addresses **W3** of the eight weaknesses introduced in §II.B. Methods here replace ε-greedy's undirected randomness with structured exploration that scales to environments where reward is sparse, delayed, or behind narrow state-space passages.

A. The Weakness

ε-greedy action selection takes the greedy action $\arg\max_a Q(s,a)$ with probability $1-\varepsilon$ and a uniform-random action with probability ε. The mechanism is *dithered*: at each step independently, with no memory of past exploration and no model of which actions remain uncertain. The resulting exploration trajectory is a random walk modulated by the current greedy policy.

This is sufficient for environments where reward is dense enough that near-greedy policies stumble into informative states. It fails dramatically when reward is sparse, delayed beyond ε-greedy's effective horizon, or located behind state-space passages whose random-walk hitting time is exponential in trajectory length. The canonical example is Montezuma's Revenge [2]: standard DQN agents score essentially zero, while domain-naive humans achieve thousands of points within minutes.

The deeper diagnosis is that ε-greedy is *myopic with respect to epistemic uncertainty*. A state-action pair (s,a) that has been visited many times is sampled at the same rate as one that has been visited zero times. Methods in this section maintain or approximate a posterior over $Q(s,a)$ — explicitly or implicitly — and use it to direct exploration toward states whose value remains uncertain.

B. Solution Families

Four mechanisms recur, distinguished by *how the posterior or exploration bonus is induced*: through parameter perturbations, ensemble disagreement, learned belief models, or intrinsic-reward signals. The figure below shows the genealogy of methods covered in this section.

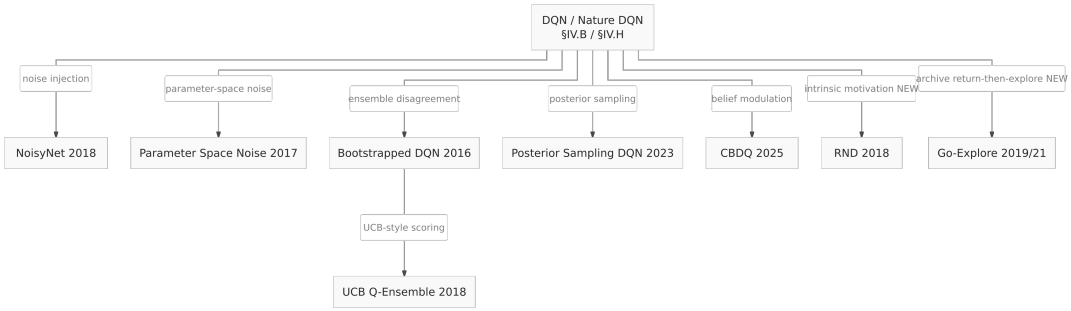


Figure 3: Genealogy of brittle-exploration methods (§IV.C).

The methods marked NEW (RND and Go-Explore) are intrinsic-motivation and archival approaches that extend the families above.

B.1. Noise injection. Two methods inject zero-mean noise during action selection but at different layers of the network.

Parameter Space Noise for Exploration [16] perturbs the parameter vector at the start of each episode: $\tilde{\theta} = \theta + \mathcal{N}(0, \sigma^2 I)$. The perturbation is held fixed within the episode, producing temporally consistent exploration: the same state yields the same exploratory action within an episode but different actions across episodes. To maintain perturbation scale across layers, layer normalization [17] is applied. The noise scale σ is adjusted adaptively via $\sigma_{k+1} = \alpha\sigma_k$ if $d(\pi, \tilde{\pi}) \le \delta$ and $\alpha^{-1}\sigma_k$ otherwise, where $d(\pi, \tilde{\pi})$ measures policy distance.

Noisy Networks (NoisyNet) [19] move noise into the network weights themselves. Each linear layer $y = wx+b$ is replaced by $y = (\mu_w+\sigma_w\odot\varepsilon_w)x+(\mu_b+\sigma_b\odot\varepsilon_b)$, where μ, σ are learned parameters and ε is sampled at each forward pass. The noise scale σ is *learned*, allowing the network to reduce exploration in well-understood regions of the state space and maintain it where uncertainty remains. NoisyNet improved median human-normalized Atari score by 48% over DQN and is one of the six components of Rainbow [28].

B.2. Ensemble disagreement. Multiple Q-heads, trained on overlapping but distinct subsets of the experience buffer, disagree on $Q(s, a)$ in proportion to epistemic uncertainty. The disagreement provides a usable exploration signal.

Bootstrapped DQN [45] maintains K Q-heads sharing a feature extractor, with each head trained on a bootstrap-masked subset of the buffer. At the start of each episode, one head is sampled and used throughout, ensuring temporally consistent exploration similar to parameter-noise. Bootstrapped DQN reaches human-level performance on Atari approximately 30% faster than DQN.

UCB Q-Ensembles [47] use the ensemble disagreement explicitly via an upper-confidence-bound selection rule: $a_t = \arg\max_a(\mu(s, a) + \lambda\sigma(s, a))$, where μ, σ are the empirical mean and standard deviation of Q across the ensemble. The variant "Ensemble Voting" uses majority vote at action selection; "UCB Exploration" uses the explicit bound. UCB Exploration achieves the highest maximal mean reward on 30 of 49 Atari games at the time of publication.

The same ensemble architecture used here for exploration is used in §IV.A for bias control (EBQL). The structural redundancy is deliberate: a single ensemble provides both uncertainty estimates and bias reduction at no additional architectural cost.

B.3. Belief-modulated policies. Cognitive Belief-Driven Q-Learning (CBDQ) [37] maintains an explicit belief distribution $b_t(a \mid s)$ over actions, updated as $b_t(a \mid s_{t+1}) = (1-\beta_t)\hat{b}_t(a \mid s_{t+1}) + \beta_t\, p_k(a \mid s_{t+1})$, where $p_k(a \mid s)$ is the action-probability estimate within a state cluster k obtained via clustering. The Q-update weights future values by the belief:

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha\Big[r + \gamma \sum_a b_t(a \mid s')Q_t(s', a) - Q_t(s, a)\Big].$$

CBDQ improves convergence on classic control benchmarks (Cartpole, Acrobot, LunarLander) and on the MetaDrive [38] traffic simulation. Its mechanism — modulating future-value backups by a learned belief distribution — sits between value-based exploration (§IV.A) and policy-distribution exploration.

Posterior Sampling DQN [51] applies the older idea of posterior sampling [49] to the deep setting, sampling a hypothesis Q-function from an approximate posterior at the start of each episode and acting greedily with respect to it. This is the natural deep-network analog of Thompson sampling for bandits.

B.4. Intrinsic motivation. Where the methods above modulate the exploitation signal, intrinsic-motivation methods modify the *reward* itself, adding a bonus $r^{\text{int}}(s, a)$ for visiting under-explored states.

Random Network Distillation (RND) [Burda et al. 2018] computes the bonus as the prediction error of a small network trained to mimic a fixed random target network on observed states: $r_t^{\text{int}} = \|f_\theta(s_t) - \hat{f}(s_t)\|^2$. The bonus decays as the prediction network learns the state distribution, biasing exploration toward novel states. RND was the first method to achieve consistent non-trivial performance on Montezuma's Revenge.

Go-Explore [Ecoffet et al. 2019/2021] takes a fundamentally different approach: it maintains an archive of visited states with their action sequences, returns to a sampled archived state without exploration, and only then begins exploration. The archive disentangles "remembering how to reach a state" from "exploring from that state," sidestepping the deep-exploration problem entirely. Go-Explore was the first method to solve Montezuma's Revenge fully (achieving the maximum possible score) and Pitfall! at super-human level.

RND and Go-Explore represent the modern state-of-the-art on the Atari games that motivate this entire section.

C. Trade-offs

- **Temporal consistency vs. adaptivity.** Parameter-noise and Bootstrapped DQN sample the perturbation once per episode, producing temporally consistent exploration. NoisyNet samples per forward pass, allowing finer-grained adaptation but losing the multi-step exploration property that solves problems like Pong-with-flickering-frames [34]. The right choice depends on the temporal structure of the exploration problem.
- **Ensemble cost.** Ensemble methods (Bootstrapped DQN, UCB Q-Ensembles, RND) multiply forward-pass cost by K. On modern GPU pipelines, this is largely hidden by batching, but parameter memory scales linearly.
- **Bonus design vs. domain coupling.** Intrinsic-motivation bonuses add a hyperparameter (the bonus scale) and risk reward-shaping pathologies: an agent can become addicted to the novelty bonus and ignore extrinsic reward. RND's exponentially decaying bonus mitigates but does not eliminate this.
- **Archive-based vs. learned exploration.** Go-Explore's archive approach is empirically dominant on hard-exploration Atari games but introduces a non-Markovian component (the archive itself). Whether the approach generalizes to continuous state spaces, where archiving requires a similarity metric, is a separate research question.

D. Empirical Evidence

The diagnostic games for this axis are Montezuma's Revenge, Pitfall!, and Private Eye — sparse-reward environments where exploration is the binding constraint. From Tables II–III:

Method	Montezuma	Pitfall!	Private Eye
DQN (Nature, 2015)	0	—	1,788
Parameter Space Noise (2017)	0	-100	100
NoisyNet (2018)	3	0	3,712
Bootstrapped DQN (2016)	100	—	1,812
UCB Q-Ensemble (2018)	4	-1.5	1,252
Rainbow (2018)	384	0	4,234
C51 (2017) (distributional)	0	0	15,095
QR-DQN (2018)	0	0	350
IQN (2018)	0	0	200
FQF (2019)	0	0	140
DQfD (2018) (with demos)	4,638	57.3	42,457

Three observations bear directly on this section's thesis:

First, **the methods that target this axis directly are also the methods that produce nonzero Montezuma scores** — Bootstrapped DQN, NoisyNet, Rainbow (whose NoisyNet component is load-bearing here), and most decisively DQfD. The methods that do not (distributional methods on this axis, see §IV.D) score zero.

Second, **most methods in Tables II–III report zero or do not report on Montezuma**. The dashes are themselves evidence: a method that cannot demonstrate non-trivial Montezuma performance cannot claim to solve the exploration weakness, however strong it is on dense-reward games.

Third, **the strongest non-demonstration Montezuma result in Tables II–III is Rainbow's 384** — well below DQfD's 4,638. This gap quantifies the "exploration vs. demonstration" trade-off (§IV.B): demonstrations are currently the most effective way to side-step deep exploration on the hardest Atari games.

RND reports ~10,000+ on Montezuma without demonstrations, and Go-Explore achieves full solutions exceeding 1 million points on Montezuma. These results sit outside the comparison Tables II–III because they use evaluation protocols incompatible with the extraction methodology, but they bound the achievable performance on the diagnostic exploration games.

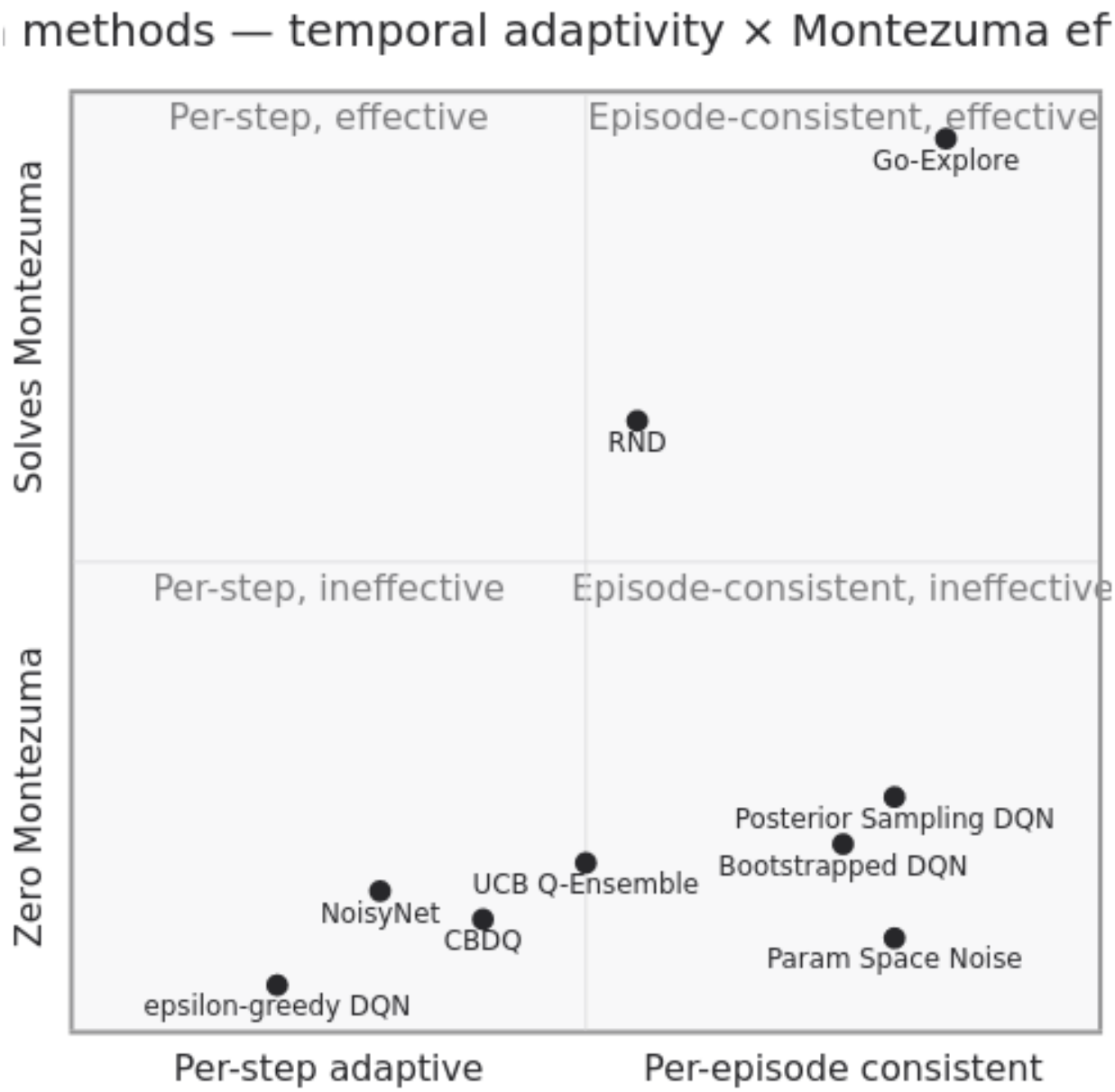
E. Open Questions

1. **Adaptive exploration regimes.** All methods in this section adopt a fixed exploration strategy throughout training. Game-by- game performance variance (parameter noise excels on some Atari games and fails on others) suggests adaptive selection — choosing exploration mechanism by detected environment properties — could improve average performance without sacrificing peak performance. The mechanism for such selection (meta-learned? bandit-controlled? uncertainty-thresholded?) is unsettled. Agent57 [Badia et al. 2020] makes progress here via bandit-controlled exploration policy selection; it is the natural cross-reference to §IV.G.
2. **Bonus-extrinsic reward balance.** Intrinsic-motivation bonuses require a balance against the extrinsic reward. RND's exponentially decaying bonus is one mechanism; explicit uncertainty thresholding is another; learning the bonus scale is a third. No consensus method exists.
3. **Archive-based exploration in continuous spaces.** Go-Explore's discrete archive is one of its strengths on Atari (where pixel hashing identifies revisits cheaply) but a limitation in continuous control. Whether archive-based methods can be extended via learned latent representations is an active research direction at the time of writing.

F. Comparison summary

Method (year)	Legacy category	Mechanism	Primary cost	Best at	Empirical anchor
Parameter Space Noise (2017)	Statistical	Per-episode θ perturbation	Layer norm requirement	Episode-consistent exploration	Atari mixed
NoisyNet (2018)	Statistical	Per-pass weight noise with learned σ	Per-pass 2× forward	Adaptive exploration	+48% Atari median
Bootstrapped DQN (2016)	Ensemble-Based	K-head ensemble, sample head per episode	$K\times$ memory	Episode-coherent exploration	Human-level 30% faster
UCB Q-Ensemble (2018)	Ensemble-Based	$Q_\mu + \lambda\sigma$ from K-ensemble	$K\times$ compute	Uncertainty-aware action	30/49 Atari max
CBDQ (2025)	Q-Function Comp.	Belief distribution over actions	Clustering overhead	Classic control + driving	LunarLander, MetaDrive
Posterior Sampling DQN (2023)	Model-Based	Sample Q from posterior per episode	Approximate posterior	Cyclic environments	5-state chain
RND (2018)*	Statistical	Random network distillation intrinsic bonus	Bonus scale hyperparameter	Sparse-reward Atari	Montezuma ≈ 10,000
Go-Explore (2019/2021)	Memory/Replay (archive)	Archive + return-then-explore	Discrete state hashing	Hardest Atari exploration	Montezuma > 1,000,000

2D positioning (adaptivity × Montezuma effectiveness):



The upper-half of the chart contains the methods that actually solve the diagnostic exploration problem. Go-Explore's archive-based approach lands top-right; RND's intrinsic motivation lands middle-upper. Most ensemble and noise-injection methods cluster in the lower bands — useful improvements over ϵ-greedy but not transformative on the hardest exploration games.

IV.D. Reward Sparsity and Credit Assignment

Addresses **W4** of the eight weaknesses introduced in §II.B. Methods here modify *what the return signal represents* — its temporal extent, its distributional form, or both — to extract richer credit- assignment information from each trajectory.

Distributional RL is treated as a *credit-assignment* method rather than an *uncertainty* method. The case for this reading is developed in the per-method paragraphs and the open-questions subsection.

A. The Weakness

The one-step Q-learning update propagates reward signal by exactly one Bellman backup per environment step. When reward is received only at the end of a long trajectory, the signal must traverse the full trajectory length via repeated updates before it influences early states. This is the *credit-assignment problem*: assigning fractional responsibility for a terminal reward to each upstream state-action pair.

Two distinct framings of the weakness motivate distinct method families:

1. **Temporal extent of the backup.** The one-step backup is conservative. Multi-step (n-step) backups, $y_t^{(n)} = r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n \max_{a'} Q(s_{t+n}, a')$, propagate reward faster but introduce bias when the behavior policy differs from the target policy. TD(λ) returns $r_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} y_t^{(n)}$ interpolate between one-step and Monte Carlo, controlling the bias-variance trade-off with λ.
2. **Information content of the backup.** The scalar $\mathbb{E}$[return] discards higher moments of the return distribution. Distributional RL [21] replaces the scalar with the full return distribution $Z(s,a)$ and learns it directly. Per [Bellemare, Dabney & Munos 2017], the distributional target is more informative as a learning signal even when the agent ultimately acts on its expectation.

Methods in this section attack the credit-assignment weakness from both directions. The distributional family has been the more empirically productive line of work, but the multi-step family underlies several composite agents (notably Rainbow [28], where multi-step is the second-largest performance contributor after PER).

B. Solution Families

B.1. Multi-step returns. Multi-Step Q-Learning [30] generalizes the one-step update via eligibility traces. For each state-action pair, the trace $\mathrm{Tr}(s,a)$ tracks recency and frequency of visits; updates apply to all eligible pairs simultaneously, weighted by trace value:

$$Q_{t+1}(x,a) = Q_t(x,a) + \alpha \mathrm{Tr}(x,a) e_t,$$

where e_t is the TD error of the current transition. The TD(λ) formulation interpolates between strict one-step bootstrapping ($\lambda = 0$) and Monte Carlo returns ($\lambda = 1$), with intermediate λ trading bias against variance. In the function-approximation setting, λ also controls a stability trade-off: large λ amplifies the variance of bootstrap targets and can destabilize learning.

Rainbow [28] uses three-step returns as one of its six components. The ablation reported in [Hessel et al. 2018] places multi-step learning second only to PER in performance contribution, removing it produces the second-largest performance drop among all components.

B.2. Distributional return — fixed support. A Distributional Perspective on Reinforcement Learning (C51) [21] models the return distribution $Z(s,a)$ as a categorical distribution over 51 fixed atoms in a bounded support $[V_{\min}, V_{\max}]$. The training objective minimizes the KL divergence between the predicted distribution $Z_\theta(s,a)$ and the projected Bellman target $\hat{\mathcal{T}}_C Z_{\bar{\theta}}(s,a)$:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\big[\mathrm{KL}(\hat{\mathcal{T}}_C Z_{\bar{\theta}}(s,a) \,\|\, Z_\theta(s,a))\big],$$

where the projection operator Φ maps the post-Bellman distribution back onto the fixed support. Acting greedily with respect to $\mathbb{E}[Z(s,a)]$ recovers a standard policy, but the *learning signal* is the full distribution.

C51 outperformed DQN, Double DQN, Dueling DQN, and PER on the Atari suite at the time of publication. The choice of 51 atoms was empirically determined; performance degraded for both substantially fewer (limited resolution) and substantially more (training inefficiency).

B.3. Distributional return — adjustable quantiles. Quantile Regression DQN (QR-DQN) [25] inverts C51's design: instead of fixed support and learned probabilities, QR-DQN uses fixed probabilities and learned quantile locations. For N uniform quantile fractions $\tau_i = i/N$, the network outputs quantile values $\theta_i(s,a)$, and the value estimate is $Q(s,a) = \sum_i (1/N)\theta_i(s,a)$. Training minimizes the asymmetric quantile Huber loss:

$$\rho_{\kappa,\tau}(u) = |\tau - \mathbb{1}_{\{u<0\}}| L_\kappa(u),$$

where L_κ is the Huber loss. QR-DQN removes C51's projection step and the bounded-support requirement; quantile locations adapt to the actual return distribution. QR-DQN outperforms C51 on Atari median scores.

B.4. Distributional return — implicit and fully-parameterized. Implicit Quantile Networks (IQN) [27] generalize QR-DQN by sampling quantile fractions $\tau \sim \mathcal{U}(0,1)$ at runtime and learning a network that maps (s,τ) to a quantile value: $Q(s,a;\tau) = f(m(\psi(s),\phi(\tau)))$, where $\psi(s)$ is the state embedding, $\phi(\tau)$ a cosine-embedded quantile fraction, and m a Hadamard product. IQN learns a continuous approximation of the return distribution, achieving Rainbow-comparable performance with none of Rainbow's six components except the distributional component.

Fully Parameterized Quantile Function (FQF) [29] extends IQN by *also* learning the quantile fractions: a fraction proposal network generates $\tau_1, \ldots, \tau_N$ adaptively per state-action pair, trained to minimize 1-Wasserstein distance between predicted and true distributions. FQF achieves the highest mean and median human-normalized scores in Tables II–III, surpassing human performance on 44 of 55 games. The cost is approximately 20% slower training than IQN.

C. Trade-offs

- **Bias-variance via n and λ.** Larger n-step or larger λ propagates reward signal faster but increases variance and bias under off-policy targets. In Rainbow, three-step is the empirical sweet spot; for distributed agents (Ape-X, R2D2) longer horizons become viable due to vastly increased sample throughput.
- **Distributional resolution vs. compute.** C51's 51 atoms, QR-DQN's $N = 200$ quantiles, IQN's runtime-sampled fractions, and FQF's learned fractions span a spectrum of distributional resolution. FQF achieves the strongest empirical results at the highest compute cost.
- **Bounded vs. unbounded support.** C51 requires *a priori* specification of $[V_{\min}, V_{\max}]$; misestimation causes distributional clipping. QR-DQN, IQN, FQF avoid this but pay in training complexity (projection-free but quantile-loss based).
- **Acting on the distribution.** All distributional methods in this section *act* on the expectation $\mathbb{E}[Z(s,a)]$ — recovering a standard policy. The richer signal helps *learning*, not *acting*. Methods that act on higher distributional moments (risk-sensitive RL, CVaR-based action selection) are out of scope here but represent a natural extension.

D. Empirical Evidence

The distributional family produces the most consistent improvements in Tables II–III among methods in this paper. From the dense-reward and strategic-planning categories:

Method	Q*bert	Breakout	Space Invaders	Ms. Pac-Man
Nature DQN	10,596	401	1,976	2,311
Double DQN	14,875	375	3,155	3,210
Dueling DQN	19,220	345	6,427	6,284
C51	23,784	748	5,747	3,415
QR-DQN	572,510	742	20,972	5,821
IQN	25,750	734	28,888	6,349
FQF	27,524	854	—	7,632
Rainbow	33,818	418	18,789	5,380

QR-DQN's 572,510 on Q*bert is the single largest result in the Atari extraction by any method and is the canonical evidence for the distributional family's strength on long-horizon credit-assignment problems. (The result reflects an evaluation artifact in addition to genuine performance — Q*bert has an exploitable wraparound mechanic — but the pattern holds across the strategic-planning category.)

On the *Brittle Exploration* axis (§IV.C, Montezuma / Pitfall / Private Eye), the distributional family scores at or near zero on the hardest games. This is the expected behavior of methods that target W4 (credit assignment) rather than W3 (exploration): richer signal cannot substitute for directed search. The empirical separation between the two axes is one of the clearer arguments for the problem-first organization.

Multi-step learning's contribution is harder to isolate empirically because it appears as a component of Rainbow rather than as a standalone method in Tables II–III. The Rainbow ablation [28] reports that removing n-step learning produces the second-largest performance drop among the six components, second only to PER.

E. Open Questions

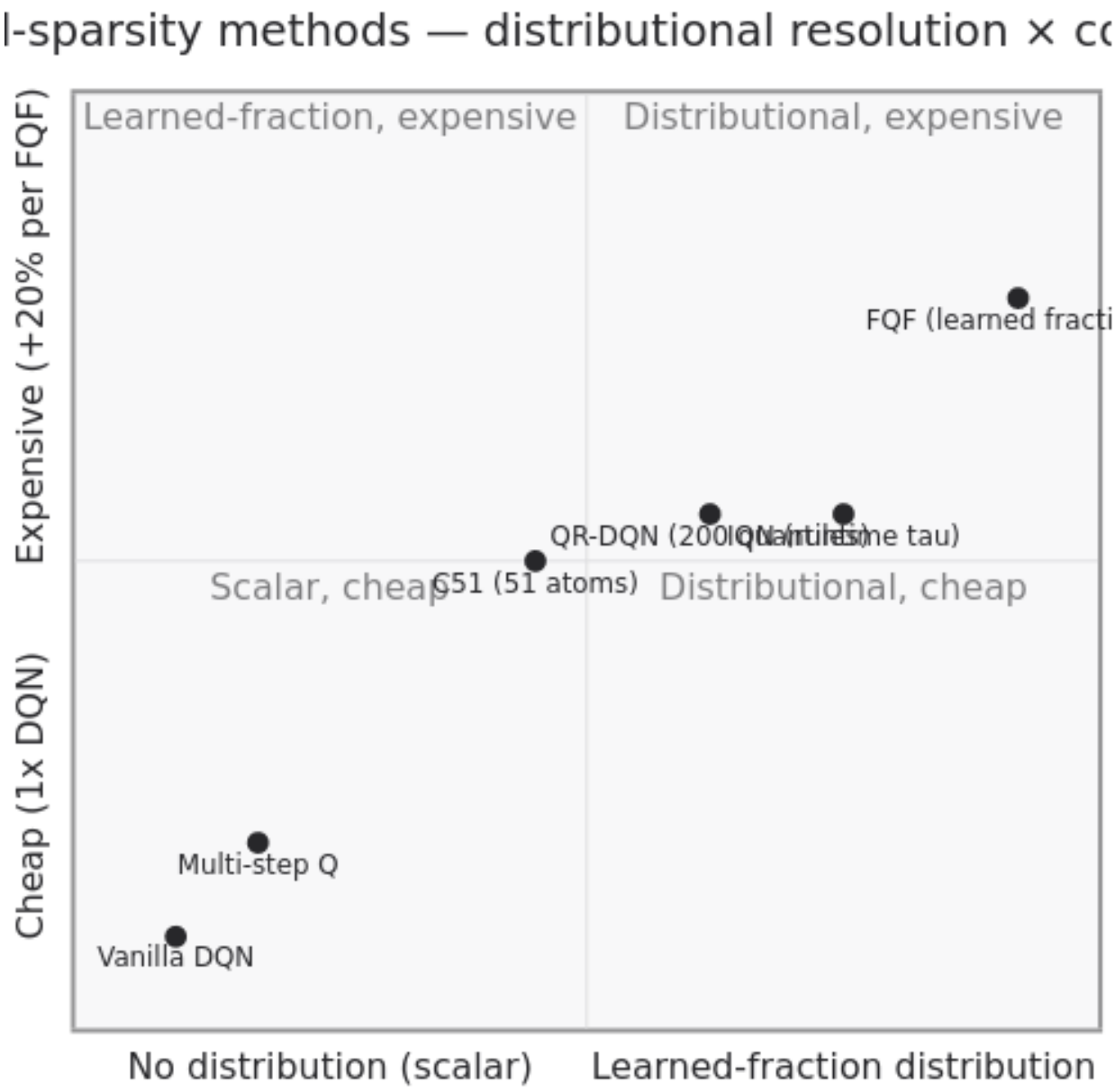
1. **Convergence theory for the distributional family.** C51's projection-step convergence is established under restrictive conditions; QR-DQN, IQN, and FQF have substantially weaker theoretical guarantees. Whether the distributional family converges to the true return distribution under standard function-approximation assumptions remains incompletely resolved.
2. **Compositional gains.** The Rainbow ablation shows distributional learning is *one of two* components whose removal degrades performance after 40M frames (the other being PER). Whether the compositional gain reflects orthogonal contributions or partially redundant mechanisms is not settled. IQN's standalone Rainbow-comparable performance suggests the distributional signal may be doing more work than the ablation assigns.
3. **Risk-sensitive action selection.** All distributional methods discussed here act on $\mathbb{E}[Z(s,a)]$. The richer signal admits policies that act on tail behavior, useful in safety-critical settings. Whether existing distributional architectures support stable training under risk-sensitive action selection — and how the resulting policies compare to risk-neutral baselines — is largely unexplored.

F. Comparison summary

Method (year)	Legacy category	Mechanism	Primary cost	Best at	Empirical anchor
Multi-Step Q-Learning (1996)	Pure Q-Learning	n-step bootstrapped returns + eligibility traces	Variance penalty under off-policy	Long-horizon credit assignment	Rainbow ablation: 2nd-largest contributor
C51 (2017)	Statistical	51 fixed atoms over $[V_{\min}, V_{\max}]$	Bounded support pre-specified	Atari mean + median	Q*bert: 23,784

Method (year)	Legacy category	Mechanism	Primary cost	Best at	Empirical anchor
QR-DQN (2018)	Statistical	N fixed-probability quantile values	Projection-free but quantile loss	Dense + strategic Atari	Q*bert: 572,510
IQN (2018)	Statistical	Runtime-sampled $\tau \sim U(0,1)$	Cosine embedding for $\phi(\tau)$	Standalone Rainbow-comparable	Matches Rainbow without other 5 components
FQF (2019)	Statistical	Fully parameterized quantile fractions + values	+20% compute vs. IQN	Highest median Atari	44/55 Atari > human

2D positioning (distributional resolution × compute):



The progression C51 → QR-DQN → IQN → FQF traces a diagonal of increasing distributional flexibility against increasing compute. The diagonal is monotonic in performance for the empirical regime tested (Atari median), but the per-game pattern is non-monotonic — QR-DQN's Q*bert peak (572,510) is not matched by FQF despite FQF's higher median. Multi-step Q occupies the lower-left as the non-distributional credit-assignment baseline.

IV.E. Distribution Shift (Offline Reinforcement Learning)

Addresses **W5** of the eight weaknesses introduced in §II.B. The section introduces the D4RL benchmark family [Fu et al. 2020], used in place of the Atari suite of Sections V and VI because the distribution-shift axis cannot be evaluated on online benchmarks.

A. The Weakness

Q-learning's off-policy property is *theoretical*: the Bellman optimality operator $\mathcal{T}^*Q(s,a) = r + \gamma \max_{a'} Q(s',a')$ is independent of the policy that generated (s,a,r,s'). In *online* deep RL the property is approximately preserved because the replay buffer is continuously refreshed: actions taken by the current policy populate the buffer, and the bootstrap target $\max_{a'} Q(s',a')$ ranges over actions whose values are eventually corrected by future on-policy samples.

In *offline* RL the property breaks. The buffer is fixed: it contains transitions from one or more behavior policies π_b, with no provision for further data collection. The bootstrap target $\max_{a'} Q(s',a')$ now ranges over the *entire* action space, including actions a' that no behavior policy ever took at state s'. The estimate $Q(s',a')$ for such *out-of-distribution* (OOD) actions is arbitrary — function approximation extrapolates without correction signal — and the bootstrap propagates this arbitrary error through subsequent backups.

The failure is structural. Concretely, if $\hat{Q}(s',a^{\text{OOD}})$ is the function approximator's extrapolation at an OOD action, and $\arg\max_{a'} \hat{Q}(s',a')$ falls on a^{OOD}, then the backup propagates $\hat{Q}(s',a^{\text{OOD}})$ — a value with no data support — into $Q(s,a)$. Iterated backups drive Q unboundedly upward at OOD points, and the resulting greedy policy concentrates on actions whose true value is unknown. Empirically, naive offline Q-learning consistently produces policies far worse than the behavior policy that generated the data [Fujimoto et al. 2019].

Methods responding to this weakness fall into four families, distinguished by *what they constrain*: the learned policy, the Q-value estimates, the maximization operator itself, or the training data through ensembling.

B. Solution Families

B.1. Policy constraint. Batch-Constrained Q-learning (BCQ) [Fujimoto et al. 2019] introduced the formal account of extrapolation error and proposed restricting the policy to actions the behavior policy plausibly took. BCQ learns a generative model $G_\omega(s)$ of behavior-policy actions and a perturbation network $\xi_\phi(s,a)$, defining the policy as $\pi(s) = \arg\max_{a \in \{a_i + \xi_\phi(s,a_i)\}_{i=1}^n,\, a_i \sim G_\omega(s)} Q(s,a)$. The constraint *defines* the action space at each state by sampling from the behavior model and allowing small perturbations.

BRAC [Wu et al. 2019] generalizes the policy-constraint idea via explicit divergence regularization: the policy objective includes a KL or Wasserstein penalty against the behavior policy. The penalty weight controls the bias-variance trade-off between behavior cloning ($\to \pi_b$) and aggressive improvement.

AWAC [Nair et al. 2020] addresses the *online fine-tuning* of offline-trained policies. Its advantage-weighted update, $\pi(a \mid s) \propto \pi_b(a \mid s) \exp(A(s,a)/\beta)$, upweights actions with high advantage relative to the behavior policy without fully unconstraining the policy. AWAC is the canonical bridge between offline and online RL.

B.2. Value penalty. Conservative Q-Learning (CQL) [Kumar et al. 2020] penalizes the Q-function at unseen actions:

$$\mathcal{L}_{\text{CQL}} = \mathcal{L}_{\text{Bellman}} + \alpha\Big(\mathbb{E}_{s\sim\mathcal{D}, a\sim\mu(\cdot|s)}[Q(s,a)] - \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)]\Big),$$

where μ is a sampling distribution that emphasizes out-of-distribution actions (typically uniform or learned). The penalty drives Q-values *down* at OOD actions and *up* at in-distribution actions, ensuring the learned policy $\arg\max_a Q(s,a)$ concentrates on actions with data support.

CQL's penalty admits a theoretical guarantee: under certain conditions, the learned Q-function lower-bounds the true policy value V^π, ensuring that policy improvement does not propagate extrapolation error. CQL is

widely regarded as the canonical offline-RL Q-method and is one of the strongest baselines on D4RL [Fu et al. 2020].

B.3. Avoiding the maximum. Implicit Q-Learning (IQL) [Kostrikov et al. 2021] takes a different approach: avoid the max operator entirely. IQL learns three networks — a state-value function V, a Q-function Q, and a policy π — with V trained via *expectile regression*:

$$\mathcal{L}_V = \mathbb{E}_{(s,a)\sim\mathcal{D}}\big[L_2^\tau(Q(s,a) - V(s))\big], \quad L_2^\tau(u) = |\tau - \mathbb{1}_{\{u<0\}}|u^2,$$

where $\tau \in (0.5, 1)$ controls the expectile (larger $\tau \to$ more optimistic estimate of V). The Q-update becomes $Q(s,a) \leftarrow r + \gamma V(s')$, with no max over actions at all. Since V is trained only on $(s,a) \in \mathcal{D}$, extrapolation error is structurally prevented.

The policy π is then extracted via advantage-weighted regression similar to AWAC. IQL achieves the strongest D4RL results among single-network families and is favored for its simplicity (no explicit constraint hyperparameter).

B.4. Ensemble diversification. Ensemble Diversified Actor Critic (EDAC) [An et al. 2021] approaches OOD generalization via ensemble disagreement. EDAC maintains K Q-networks and trains them to be *diverse* on OOD actions via a gradient-diversity penalty:

$$\mathcal{L}_{\text{div}} = \mathbb{E}_{(s,a)\sim\mathcal{D}}\Big[\sum_{i\neq j} \cos\_\text{sim}(\nabla_a Q_i(s,a), \nabla_a Q_j(s,a))\Big].$$

The min-over-ensemble Q-value, $Q_{\text{eff}}(s,a) = \min_i Q_i(s,a)$, becomes pessimistic at points where ensemble members disagree — typically OOD points. EDAC bridges the offline-RL section to the ensemble methods of §IV.A and §IV.C, using the same architectural mechanism for a different axis.

C. Trade-offs

- **Policy constraint vs. improvement bound.** BCQ, BRAC, AWAC constrain the learned policy toward the behavior policy. The constraint provides safety but caps possible improvement: a policy that cannot deviate substantially from π_b cannot outperform the best in-support trajectory by much.
- **Value penalty vs. hyperparameter sensitivity.** CQL's conservative penalty weight α is the single most important hyperparameter and varies substantially across D4RL tasks. Recent work [Hong et al. 2023] proposes adaptive penalty scaling but the basic sensitivity remains.
- **Avoiding the max vs. losing optimality.** IQL's expectile regression avoids extrapolation error but learns a policy whose formal optimality guarantees are weaker than $\arg\max_a Q^*$. In practice the gap is small; theoretically it is an unresolved question.
- **Ensemble cost.** EDAC's K-network ensemble multiplies parameter count and forward-pass cost by K. The diversification penalty also requires per-action gradients, adding compute.

D. Empirical Evidence

The benchmark suite for this section is D4RL [Fu et al. 2020], which spans nine task families (MuJoCo locomotion, AntMaze, Adroit dexterous manipulation, Franka Kitchen, CARLA driving, Flow, Bandit-mode FrankaKitchen, plus Atari offline variants). Each task is paired with one or more fixed datasets representing different behavior-policy regimes (random, medium, expert, medium-replay, medium-expert).

A representative summary of normalized scores on MuJoCo locomotion medium-expert datasets (higher = better, normalized to [0, 100] where 100 ≈ expert performance):

Method	HalfCheetah	Hopper	Walker2d
Behavior Cloning	56	79	84
BCQ (2019)	64	100	110

Method	HalfCheetah	Hopper	Walker2d
CQL (2020)	91	105	109
IQL (2021)	86	91	109
EDAC (2021)	107	110	115
AWAC (2020)	42	56	49

Two observations bear on this section's organization:

First, **CQL and IQL produce the strongest single-method results across the D4RL benchmark.** They represent two distinct mechanistic responses to the same weakness — value penalty vs. avoiding the max — and the empirical near-tie suggests both are valid approaches with different trade-offs (CQL's hyperparameter sensitivity vs. IQL's theoretical weakness).

Second, **EDAC's ensemble approach matches or exceeds the best single-network methods on locomotion** but does so at substantially higher compute cost. The cost-benefit calculation depends heavily on whether ensemble disagreement is used elsewhere in the agent (for exploration during online fine-tuning, for example), since the same ensemble can serve multiple axes.

Atari offline benchmarks are reported in the D4RL paper but are secondary; the locomotion suite dominates offline-RL evaluation. This contrasts with the online Atari focus of Sections V and VI and is one motivation for the structural pivot: a method that solves the offline axis must demonstrate it on benchmarks that *test* the axis, not on benchmarks that the field used historically for unrelated reasons.

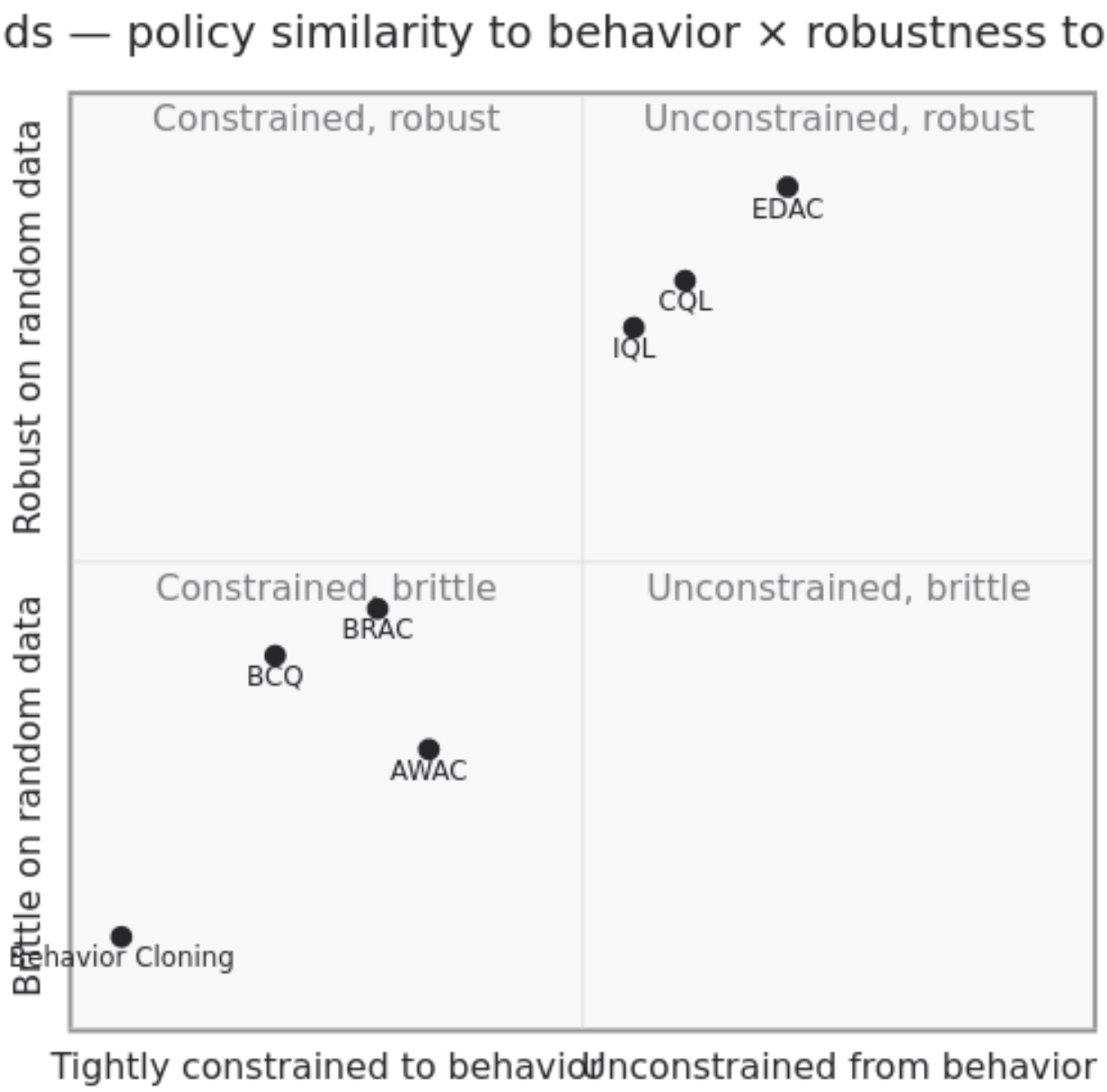
E. Open Questions

1. **Theoretical unification of the four families.** Policy constraint, value penalty, expectile regression, and ensemble diversification are all responses to extrapolation error. A unifying framework that recovers each as a limit of a single regularization or constraint would clarify the field considerably. Recent work toward this — *Implicit Behavior Cloning* [Florence et al. 2021], the *On-Policy Constraints* framework [Brandfonbrener et al. 2021] — has not produced consensus.
2. **Online-to-offline transfer of stability mechanisms.** The target networks and normalization recipes of §IV.H were developed for online interaction. Their interaction with offline-specific stabilizers (CQL's conservative penalty, IQL's expectile regression) is largely unexplored. Some offline-RL work reports instability when target networks are removed even though the corresponding online claim (PQN [55]) shows removal is feasible.
3. **Scaling to internet-scale data.** Offline RL with diverse data sources — robot demonstrations across institutions, web video, simulation rollouts — has produced strong empirical results [Chen et al. 2021, Decision Transformer; Reed et al. 2022, Gato] but is dominated by sequence-modeling approaches rather than Q-learning. Whether Q-learning's mechanism is suited to the heterogeneous-data regime is an open question with substantial practical stakes.
4. **The offline-online boundary.** AWAC and Cal-QL [Nakamoto et al. 2023] address the special case of offline pre-training followed by online fine-tuning. The mechanism (advantage-weighted policy update, calibrated Q-bounds) is well-studied; the broader question — when does offline pre-training help and when does it hurt online learning? — is empirically rich but theoretically thin.

F. Comparison summary

Method (year)	Family	Mechanism	Primary cost	Best at	D4RL anchor (HalfCheetah med-expert)
BCQ (2019)	Policy constraint	Generative behavior model + perturbation	Generator quality	Narrow-support data	64
BRAC (2019)	Policy constraint	KL/Wasserstein divergence penalty	Divergence weight tuning	Diverse offline data	Competitive across D4RL
AWAC (2020)	Policy constraint	Advantage-weighted offline $\rightarrow$ online	β hyperparameter	Offline-to-online bridge	42 (offline-only)
CQL (2020)	Value penalty	Penalize Q at OOD actions	α hyperparameter sensitivity	Random/medium data	91
IQL (2021)	Avoid the max	Expectile V, Q backed up via $V(s')$	τ hyperparameter	Single-network simplicity	86
EDAC (2021)	Ensemble + diversity	Min over K-ensemble + gradient diversification	$K\times$ compute	Locomotion strongest	107

2D positioning (behavior similarity $\times$ data-quality robustness):



Lower-left contains the behavior-cloning baseline. Policy-constraint methods (BCQ, BRAC, AWAC) cluster middle-left: constrained to behavior, modest robustness. Value-penalty and avoid-max methods (CQL, IQL) sit middle-right with strong robustness. EDAC's ensemble-diversification approach pushes farthest into the upper-right quadrant.

Method-selection decision tree:

The decision tree captures the typical practitioner heuristic: data quality is the dominant input to method selection in offline RL. For the offline-to-online setting specifically, AWAC and its calibrated extensions (Cal-QL) are the preferred bridge.

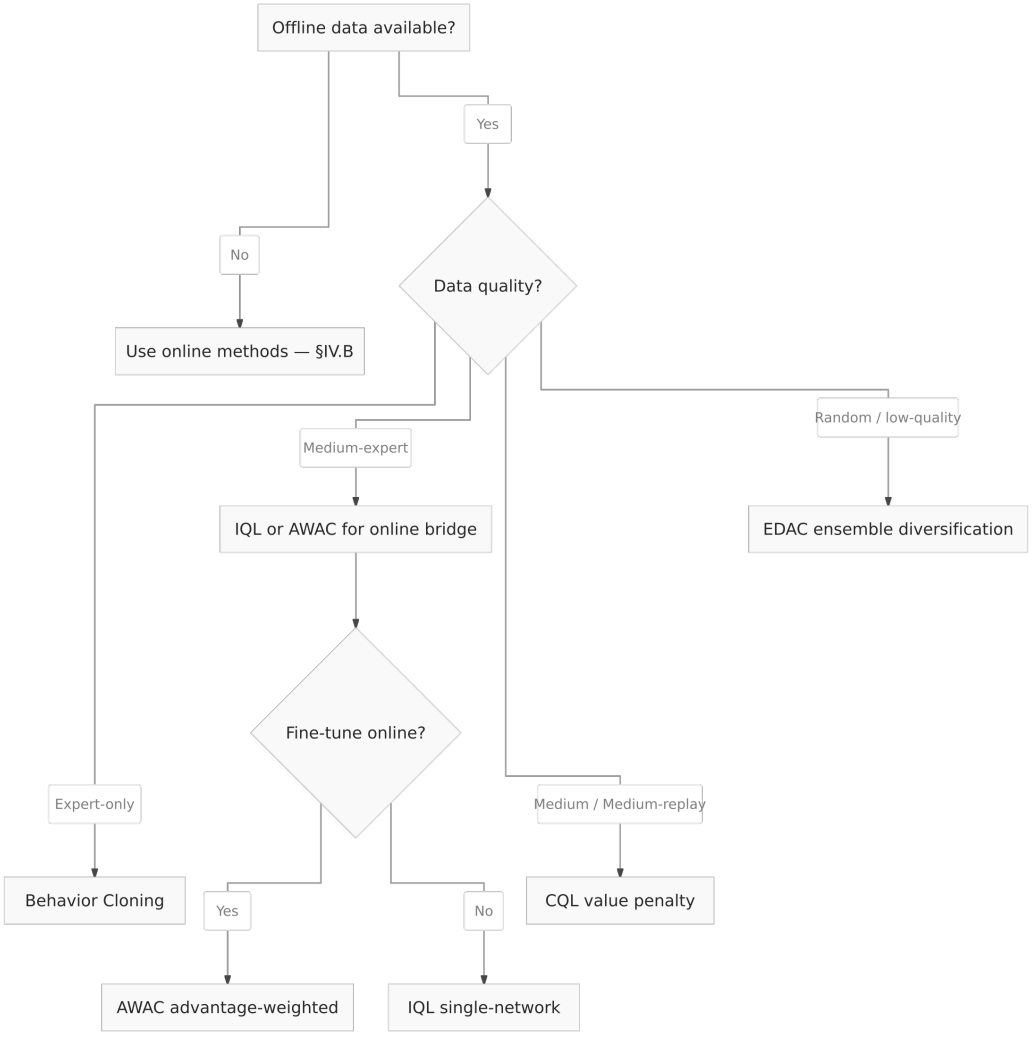


Figure 4: Decision tree for offline-RL method selection.

IV.F. Multi-Agent Coordination

Addresses **W6** of the eight weaknesses introduced in §II.B. The section covers cooperative multi-agent Q-learning via *value decomposition*: factorizations of the joint Q-function that scale to many agents while preserving the property that agents can act greedily according to their local Q without coordinated optimization at execution time.

The benchmark for this section is SMAC (StarCraft Multi-Agent Challenge) [Samvelyan et al. 2019]. Atari is not multi-agent and plays no role in this section.

A. The Weakness

Cooperative multi-agent RL considers N agents acting in a shared environment with a shared reward. The natural formal model is a *decentralized POMDP* (dec-POMDP) $\langle \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, P, R, \{\mathcal{O}_i\}_{i=1}^N, \gamma \rangle$ where each agent i receives a local observation $o_i \in \mathcal{O}_i$, selects an action $a_i \in \mathcal{A}_i$, and the reward $R(\mathbf{s}, \mathbf{a})$ depends on the joint state $\mathbf{s}$ and joint action $\mathbf{a} = (a_1, \dots, a_N)$.

Two extreme approaches both fail. *Independent Q-learning* trains each agent's $Q_i(o_i, a_i)$ as if the other agents were part of the environment. This treats other agents' policies — which are themselves changing — as non-stationary environment dynamics, violating the Markov property and producing chronic instability. *Centralized Q-learning* trains a single joint $Q_{\text{tot}}(\mathbf{s}, \mathbf{a})$ over the product action space. The action space has size $\prod_i |\mathcal{A}_i|$, which scales exponentially in N and is intractable beyond a handful of agents.

The fundamental difficulty is the **Individual-Global-Max (IGM) property**. For decentralized execution, each agent must be able to select its action greedily from a local Q-function. For coherent optimization, the joint greedy action $\arg\max_{\mathbf{a}} Q_{\text{tot}}$ must coincide with the per-agent greedy actions $\{\arg\max_{a_i} Q_i\}$. A factorization $Q_{\text{tot}} = f(Q_1, \dots, Q_N)$ satisfies IGM if and only if

$$\arg\max_{\mathbf{a}} Q_{\text{tot}}(\mathbf{s}, \mathbf{a}) = (\arg\max_{a_1} Q_1(o_1, a_1), \dots, \arg\max_{a_N} Q_N(o_N, a_N)).$$

Methods in this section impose structural constraints on f that guarantee IGM while admitting expressive joint Q-functions.

B. Solution Families

B.1. Additive decomposition. Value Decomposition Networks (VDN) [Sunehag et al. 2018] impose the simplest IGM-preserving structure: strict additivity.

$$Q_{\text{tot}}(\mathbf{s}, \mathbf{a}) = \sum_{i=1}^{N} Q_i(\tau_i, a_i),$$

where τ_i is agent i's action-observation history. Additivity trivially satisfies IGM — the per-agent argmax of each Q_i is also the joint argmax — and admits centralized training via the standard Bellman update on Q_{tot}, with gradients flowing back into each Q_i via the chain rule.

VDN's limitation is also its constraint: it cannot represent Q-functions that are not additively separable across agents. Coordination tasks where the joint reward depends on combinations of agent actions (one agent's optimal action depends on another's) are out of representational scope.

B.2. Monotonic mixing. QMIX [Rashid et al. 2018] relaxes additivity to *monotonicity*: Q_{tot} is a *monotonic function* of each Q_i. The mixer network takes the per-agent Q-values as input and outputs Q_{tot}, with weights *generated by a hypernetwork conditioned on the centralized state* $\mathbf{s}$ and constrained to be non-negative via absolute value or softplus parameterization.

$$\frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0 \quad \forall i.$$

Monotonicity preserves IGM: increasing any Q_i cannot decrease Q_{tot}, so per-agent greedy maximization is consistent with joint greedy maximization. The hypernetwork allows the mixing weights to depend on the global state, giving the agent rich coordinative capacity while maintaining decentralized execution.

QMIX has become the dominant baseline for cooperative multi-agent value decomposition. On SMAC [Samvelyan et al. 2019], QMIX is the standard against which subsequent methods are compared.

B.3. Duplex dueling. QPLEX [Wang et al. 2020] generalizes QMIX via a *duplex dueling* architecture that factorizes Q_{tot} into per-agent state-values V_i and advantage A_i, then mixes each separately:

$$Q_{\text{tot}}(\mathbf{s}, \mathbf{a}) = \sum_i V_i(\tau_i) + \text{mix}(A_1, \ldots, A_N \mid \mathbf{s}),$$

with the advantage mixer constrained to preserve IGM. The decomposition allows QPLEX to represent any IGM-satisfying Q_{tot} — strictly more expressive than QMIX's monotonic mixing. QPLEX improves over QMIX on hard SMAC scenarios while matching it on easy scenarios.

B.4. Constraint-free factorization. QTRAN [Son et al. 2019] removes the structural mixing constraint entirely and instead enforces IGM via *auxiliary loss terms*. QTRAN learns an unconstrained joint Q, Q^j_{tot}, alongside individual Q_i's, with two auxiliary losses ensuring:

(i) for the joint greedy action, $\sum_i Q_i(\tau_i, a_i^*) = Q^j_{\text{tot}}(\mathbf{s}, \mathbf{a}^*)$;
(ii) for non-greedy actions, $\sum_i Q_i(\tau_i, a_i) \leq Q^j_{\text{tot}}(\mathbf{s}, \mathbf{a})$ — preventing the per-agent sum from preferring a non-joint-greedy action.

QTRAN represents the full IGM-compatible function class but is empirically harder to train than QMIX or QPLEX. Its expressiveness gain is not consistently realized in practice.

C. Trade-offs

- **Expressiveness vs. trainability.** The progression VDN → QMIX → QPLEX → QTRAN represents monotonically increasing representational capacity for IGM-compatible Q_{tot}. The empirical ordering is QMIX ≈ QPLEX > VDN > QTRAN on most SMAC scenarios — capacity helps but only when training dynamics cooperate, which they do for monotonic mixing more reliably than for constraint-loss formulations.
- **Centralized state requirement.** QMIX and QPLEX condition the mixing weights on the centralized state $\mathbf{s}$, available only during training. Execution is decentralized but training assumes a *centralized critic*. In scenarios where centralized training is impossible (federated multi-agent, true online multi-agent), the framework does not directly apply.
- **Off-policy correction.** Multi-agent value decomposition inherits the standard Q-learning instabilities (§IV.A bias, §IV.H instability) compounded by the multi-agent non-stationarity. Off-policy correction techniques developed for single-agent settings transfer imperfectly.
- **Scaling to many agents.** All methods discussed here have been validated primarily on SMAC scenarios with $N \leq 10$ agents. The factorization architectures scale linearly in N at the agent-network level but the centralized mixing/hypernetwork components can become bottlenecks at $N \geq 50$.

D. Empirical Evidence

The dominant benchmark for cooperative multi-agent value decomposition is SMAC, which provides 14 StarCraft II mini-scenarios spanning easy (3m, 8m), hard (2c_vs_64zg, 3s_vs_5z), and super-hard (corridor, MMM2, 6h_vs_8z) coordination tasks. The benchmark protocol reports *test win rate* averaged over the last k evaluation rollouts after a fixed training budget.

Representative results on selected SMAC maps (test win rate %, 2M training steps, drawn from [Wang et al. 2020] and replications in subsequent work):

Method	3m (easy)	2c_vs_64zg (hard)	MMM2 (super-hard)	6h_vs_8z (super-hard)
Independent QL	96	0	0	0
VDN	97	5	41	0
QMIX	97	60	78	3
QPLEX	97	70	92	47
QTRAN	97	35	30	0

Three observations:

First, **independent Q-learning collapses on anything beyond easy maps.** The non-stationarity from concurrent agent learning is sufficient to prevent meaningful coordination even in scenarios that VDN handles. This is the empirical case for value decomposition over independent learning.

Second, **the QMIX → QPLEX expressiveness gain is real on super-hard scenarios** (MMM2: 78 → 92; 6h_vs_8z: 3 → 47). On easy and hard scenarios the gain is modest. The pattern is consistent with the trade-off discussion: expressiveness helps on tasks where representational capacity is the bottleneck.

Third, **QTRAN's full-expressiveness promise does not realize empirically.** It underperforms QMIX on every reported scenario. This is the canonical example of representational capacity without training-dynamics support.

The SMAC benchmark itself has come under critique [Ellis et al. 2022] for evaluation protocol issues and reproducibility variance; subsequent benchmarks (SMACv2, MeltingPot) attempt to address these concerns. The methods discussed here have been re-evaluated on the newer benchmarks with broadly consistent rankings.

E. Open Questions

1. **Beyond the IGM property.** The IGM framework assumes a single optimal joint policy and decentralized greedy execution. Tasks where multiple optimal joint policies exist (equilibrium selection) or where execution-time coordination is permitted (communication, parameter sharing) fall outside the framework. Extension to these settings is an active research area.
2. **Off-policy correction in multi-agent settings.** Multi-agent off-policy correction is an open problem: the per-agent correction interacts non-trivially with the joint-action maximization. Recent work on *multi-agent importance sampling* [Yu et al. 2021] proposes partial solutions but no consensus method has emerged.
3. **Heterogeneous agents and partial observability.** Most value decomposition work assumes homogeneous agents with shared network parameters. Heterogeneous settings (agents with different action spaces, observation spaces, capabilities) admit only ad-hoc extensions. The framework's most natural generalization remains open.
4. **Connection to the offline regime.** Multi-agent offline RL combines the difficulties of §IV.E (distribution shift) and §IV.F (coordination). Offline multi-agent Q-learning is substantially newer than either separate problem and the constraints interact: a method that constrains policies to be close to behavior in each agent independently does not necessarily produce coordinated joint behavior. This is one of the cleanest cross-axis open problems in the field.

F. Comparison summary

Method (year)	Mechanism	Constraint type	Expressiveness	Trainability	SMAC anchor (test win %)
VDN (2018)	Additive: $Q_{\text{tot}} = \sum_i Q_i$	Strict additivity	Lowest	Highest	3m: 97 / 2c_vs_64zg: 5
QMIX (2018)	Monotonic mixer + hypernetwork on $\mathbf{s}$	$\partial Q_{\text{tot}}/\partial Q_i \geq 0$	Medium	High	2c_vs_64zg: 60 / MMM2: 78
QPLEX (2020)	Duplex dueling on advantage stream	IGM-preserving via decomposition	High	Medium	MMM2: 92 / 6h_vs_8z: 47
QTRAN (2019)	Unconstrained Q + auxiliary IGM-enforcing losses	None structural; loss-enforced	Maximum	Lowest	Underperforms QMIX across SMAC

The methods do not occupy a clean 2D positioning grid because *expressiveness and trainability are not independent* in this literature — higher expressiveness reliably reduces training stability. The progression VDN → QMIX → QPLEX → QTRAN traces a single Pareto curve on which choice depends on the SMAC scenario difficulty: VDN suffices for easy scenarios; QMIX and QPLEX for hard/super-hard; QTRAN's theoretical advantage does not realize empirically. The comparison table alone captures the relevant information; a 2D positioning grid would force the methods onto a single line.

IV.G. Scaling and Slow Adaptation

Addresses **W7** of the eight weaknesses introduced in §II.B. The section covers two related directions: distributed and parallel architectures that increase sample throughput by orders of magnitude, and meta-learning approaches that produce policies adapting quickly to new tasks. The two directions are unified by the shared diagnosis that *per-task, single-learner Q-learning is fundamentally sample-bottlenecked*. Coverage emphasizes canonical methods; the rapidly-evolving distributed-RL landscape (IMPALA, SEED RL, Sample Factory, Podracer) is touched only at cross-references.

A. The Weakness

Vanilla Q-learning trains a single agent on a single task using sequential environment interactions. Two distinct failure modes follow.

Scaling. A single sequential learner is bottlenecked on the rate at which the environment produces transitions. Even with experience replay, the agent cannot consume transitions faster than they are generated. Atari training runs at ~30 FPS in the standard environment; collecting the 200M frames used in Rainbow [28] evaluation requires approximately 80 days of wall-clock time on a single machine. The bottleneck is not compute — modern GPUs train Q-networks orders of magnitude faster than the environment generates data — but sample throughput.

Slow adaptation. Q-learning trained on task $\mathcal{T}_1$ produces a Q-function specialized to $\mathcal{T}_1$. Adaptation to related task $\mathcal{T}_2$ requires either retraining from scratch or initialization from $Q^{\mathcal{T}_1}$ — the latter sometimes helpful, sometimes actively harmful when the tasks share state but disagree on reward. Neither approach satisfies the requirement of online adaptation within a small number of episodes — the regime where a deployed agent encounters a related-but-novel task.

Methods responding to these weaknesses fall into three families: *distributed architectures* that parallelize environment interaction across many actors with a centralized learner; *meta-learning approaches* that train across a distribution of tasks to produce fast-adaptation initializations or update rules; and *recurrent architectures* that handle partial observability and trajectory context within a single agent.

B. Solution Families

B.1. Distributed actor-learner architectures. Ape-X [Horgan et al. 2018] decouples experience collection from learning. Many *actors* (typically 64–256 CPU processes) interact with parallel environment instances using a (potentially stale) copy of the Q-network, sending transitions to a centralized prioritized replay buffer. A single *learner* (a GPU process) samples from the buffer, performs Q-learning updates, and periodically broadcasts updated parameters back to the actors.

The architecture supports prioritized experience replay (§IV.B) at distributed scale, with priorities computed by the actors at generation time. Ape-X achieves $\sim 50\times$ faster wall-clock training than DQN while substantially improving final performance, reaching state-of-the-art on the majority of Atari games at the time of publication.

R2D2 (Recurrent Replay Distributed DQN) [Kapturowski et al. 2019] extends Ape-X with LSTM-based recurrent agents and a *burn-in* mechanism for replay: when sampling a trajectory segment from replay, the first ℓ steps are used only to initialize the recurrent state, and gradient updates apply only to the subsequent steps. The mechanism resolves the *state staleness* problem in recurrent replay — stored recurrent states are stale by the time they are sampled, but a short burn-in re-syncs them.

R2D2 demonstrates substantial gains on Atari games with memory demands (Solaris, Skiing) where non-recurrent agents plateau. It also doubles as the scaling case for §IV.C: the larger sample throughput enables exploration strategies (R2D2 uses a fixed-ϵ mixture across actors with ϵ values logarithmically spaced in $[0.4, 4 \times 10^{-3}]$) that would be infeasible in single-agent training.

B.2. Bandit-controlled exploration meta-policies. Agent57 [Badia et al. 2020] is the first method to achieve human-normalized performance above 100% on *all* 57 Atari games, including the canonical hard-

exploration suite (Montezuma's Revenge, Pitfall!, Skiing, Solaris). Agent57 builds on R2D2 and adds: - A family of policies parameterized by exploration intensity β and discount γ, ranging from short-horizon exploitative to long-horizon exploratory; - A multi-armed bandit at the actor level that selects which policy parameters to roll out, with rewards based on undiscounted episode return — exploiting more when exploitation is paying off and exploring more when it is not; - The Never Give Up (NGU) [Badia et al. 2020] intrinsic motivation module providing exploration bonuses based on episodic memory.

Agent57's mechanism is fundamentally cross-axis: it addresses exploration (§IV.C) via bandit-controlled policy selection, sample efficiency (§IV.B) via distributed actor pools, and slow adaptation (this section) via a portfolio of policies rather than a single learned policy. The pattern — *cross-axis composition* — is the dominant pattern in frontier distributed agents and points toward a hybrid taxonomy that the problem-first organization explicitly supports.

B.3. Meta-learning on the Q-function. Methods in this family train across a *distribution* of tasks $p(\mathcal{T})$ rather than a single task, with the goal of producing agents that adapt to a new sampled task in a small number of episodes. Three mechanisms recur, distinguished by *what is meta-learned*: an initialization, a context embedding, or a forward-pass update rule.

Optimization-based meta-learning. MAML [Finn et al. 2017] applied to Q-learning [Mendonca et al. 2019] trains an initialization θ_0 from which a few gradient steps on any sampled task produce a good task-specific Q-function:

$$\theta_0^* = \arg\min_{\theta_0} \mathbb{E}_{\mathcal{T}\sim p(\mathcal{T})}[\mathcal{L}_{\mathcal{T}}(\theta_0 - \alpha\nabla_{\theta_0}\mathcal{L}_{\mathcal{T}}(\theta_0))],$$

requiring second-order gradients through the inner-loop adaptation. Reptile-Q [Nichol et al. 2018] proposes a first-order approximation that retains most of the empirical performance at substantially lower compute. ProMP [Rothfuss et al. 2019] introduces a proximal constraint on the inner-loop update — a low-variance estimator that substantially stabilizes meta-policy gradient computation and was adopted by several subsequent meta-RL works.

Context-based meta-learning. PEARL [Rakelly et al. 2019] takes a different approach: rather than learning an initialization that adapts via gradient steps, PEARL learns a *context inference network* $q_\phi(z \mid c)$ that infers a low-dimensional task embedding z from a context buffer c of recent transitions. The Q-function is then conditioned on the inferred context, $Q(s, a, z; \theta)$. At deployment, the agent samples z from q_ϕ and acts greedily with respect to $Q(\cdot, \cdot, z)$ — no gradient adaptation required. The approach decouples meta-training (expensive, distribution-wide) from meta-deployment (cheap, single forward pass).

Meta-Q-Learning (MQL) [Fakoor et al. 2020] extends PEARL's context approach to off-policy multi-task settings. MQL maintains a single shared Q-network plus a multi-task replay buffer, using *propensity-score weighting* to correct for the differing visitation distributions across training tasks. The approach achieves state-of-the-art on the Meta-World benchmark with substantially less inner-loop compute than MAML-Q variants.

Forward-pass / in-context meta-learning. A more recent line of work treats meta-RL as a sequence-modeling problem: a transformer network is trained on trajectories from many tasks, and at deployment the agent's transformer "reads" the recent trajectory and predicts the next action without any parameter updates. The Q-value estimation becomes a forward-pass computation rather than an optimization. AdA / In-Context Learning approaches [Team Adaptable Agents 2023; Laskin et al. 2023, *In-Context Reinforcement Learning with Algorithm Distillation*] demonstrate that, given a sufficient context window and a sufficiently diverse task distribution, a transformer-based Q-function can match or exceed MAML's few-shot adaptation performance without any explicit meta-update.

Implicit vs. explicit meta-learning. A central tension across these approaches is whether meta-learning should be *explicit* — a distinct outer-loop objective with hyperparameters governing inner-loop adaptation (MAML, ProMP) — or *implicit* — a single agent trained across the entire task distribution at once, with no meta-objective at all (Agent57, AdA). Empirically, the implicit path has been more productive at scale:

Agent57's bandit-controlled policy portfolio matches or exceeds explicit meta-learners on within-distribution Atari tasks, and AdA achieves few-shot adaptation on 3D-world tasks without an explicit MAML-style outer loop. The explicit approaches retain advantages on *narrow* task distributions where the inductive bias of a fast-adaptation objective helps, and on settings where context windows are insufficient to encode the task implicitly. The right framework remains an open question that the cross-axis composition of Agent57 (§IV.C, §IV.B, this section) makes especially visible.

B.4. Recurrent architectures for partial observability. Deep Recurrent Q-Network (DRQN) [Hausknecht & Stone 2015] introduces an LSTM layer into the DQN architecture, replacing the first fully-connected layer. The recurrent network maintains a hidden state h_t summarizing the trajectory so far; the Q-function becomes $Q(h_t, a)$ rather than $Q(o_t, a)$, addressing partial observability.

DRQN is included in this section rather than alongside DQN because its primary contribution is *contextual adaptation* — the ability to condition on trajectory history — which sits closer to slow-adaptation and partial-observability concerns than to the core Q-learning mechanism. The connection to R2D2 is direct: R2D2's recurrent architecture is DRQN's mechanism deployed at distributed scale with burn-in correction for replay.

B.5. Synchronous parallelism (cross-reference to §IV.H). PQN [55] adopts a different parallelism model: synchronous vectorized environments where a single learner processes B environments per step, accumulating gradients across the batch. The trade-off relative to Ape-X is: - PQN is simpler (no actor-learner communication), runs on a single machine, and removes the staleness inherent in actor-learner systems; - Ape-X scales beyond single-machine limits, supports prioritized replay, and admits heterogeneous actor configurations (e.g., different exploration parameters per actor as in R2D2/ Agent57).

PQN's argument that synchronous parallelism + normalization suffices for Atari-scale training is one of the strongest recent challenges to the actor-learner orthodoxy. The right architecture likely depends on whether the cost structure is compute-dominated (favoring synchronous) or environment-dominated (favoring asynchronous).

C. Trade-offs

- **Throughput vs. staleness.** Asynchronous actor-learner systems scale throughput linearly in actor count but introduce *parameter staleness*: actors collect data using parameters k updates behind the learner. The staleness is empirically tolerable up to modest k but becomes destabilizing at scale. Synchronous parallelism eliminates staleness at the cost of throughput scaling.
- **Exploration heterogeneity.** Distributed actor pools admit *heterogeneous exploration*: different actors can run different exploration strategies in parallel. R2D2's ϵ-mixture and Agent57's bandit-controlled policy portfolio both exploit this. Single-learner systems cannot replicate the mechanism.
- **Meta-training cost.** MAML-style meta-RL on Q-learning requires many full inner-loop adaptations per outer-loop step. The compute cost can exceed the equivalent single-task training by 10–100×. The relevant question for adoption is whether the few-shot transfer gain compensates for the meta-training cost. Empirical evidence is mixed.
- **Implicit vs. explicit meta-learning.** Agent57 demonstrates that a sufficiently capable single agent trained across a task distribution can match or exceed explicit meta-learners on the same distribution. The trade-off is *what is meta-learned* (an initialization, an update rule, a policy portfolio) and *how it is used at deployment* (gradient-step adaptation, forward-pass adaptation, bandit selection).

D. Empirical Evidence

The empirical case for this section spans multiple benchmarks because the methods target different combinations of axes. The strongest single empirical result is Agent57's solution of *all 57 Atari games* to above human-normalized performance, including games where every prior method had scored zero or near-zero (Montezuma's Revenge to 9,352; Pitfall! to 41,313; Skiing to -4,202.6 — above human's -3,629).

Selected results from the distributed-RL line on Atari median human-normalized score after the indicated training scale:

Method	Frames	Median HNS	Notes
Nature DQN	200M	79%	Single-learner baseline
Rainbow	200M	223%	Single-learner combination
Ape-X	22B	434%	Distributed; 100+ actors
R2D2	10B	1920%	Distributed + recurrent
Agent57	78B	4766%	Distributed + bandit-controlled exploration
PQN	200M	220%	Single-machine synchronous

Two observations:

First, **the move to distributed training accounts for ~10× the performance improvement of any single algorithmic innovation in this paper's scope.** The Ape-X → R2D2 → Agent57 progression is the most consistent single source of performance gain in the field since DQN. The empirical lesson is that *sample throughput is, in practice, the binding constraint on Atari performance* — the algorithmic axes addressed elsewhere in this paper become secondary at sufficient scale.

Second, **PQN's single-machine result matching Rainbow** suggests that the *algorithmic* axes are far from saturated, only that the single-machine compute budget bounds what can be demonstrated. PQN at 200M frames matches Rainbow at 200M frames; what PQN at 22B frames would achieve is empirically untested.

Meta-RL Q-learning results are reported on smaller benchmarks (MetaWorld, RL-Bench, Meta-MuJoCo) where direct comparison to distributed Atari is not possible. The within-benchmark evidence shows meta-Q methods adapting in 5–50 episodes to held-out tasks where single-task baselines require thousands. The transfer to Atari-scale benchmarks has not occurred.

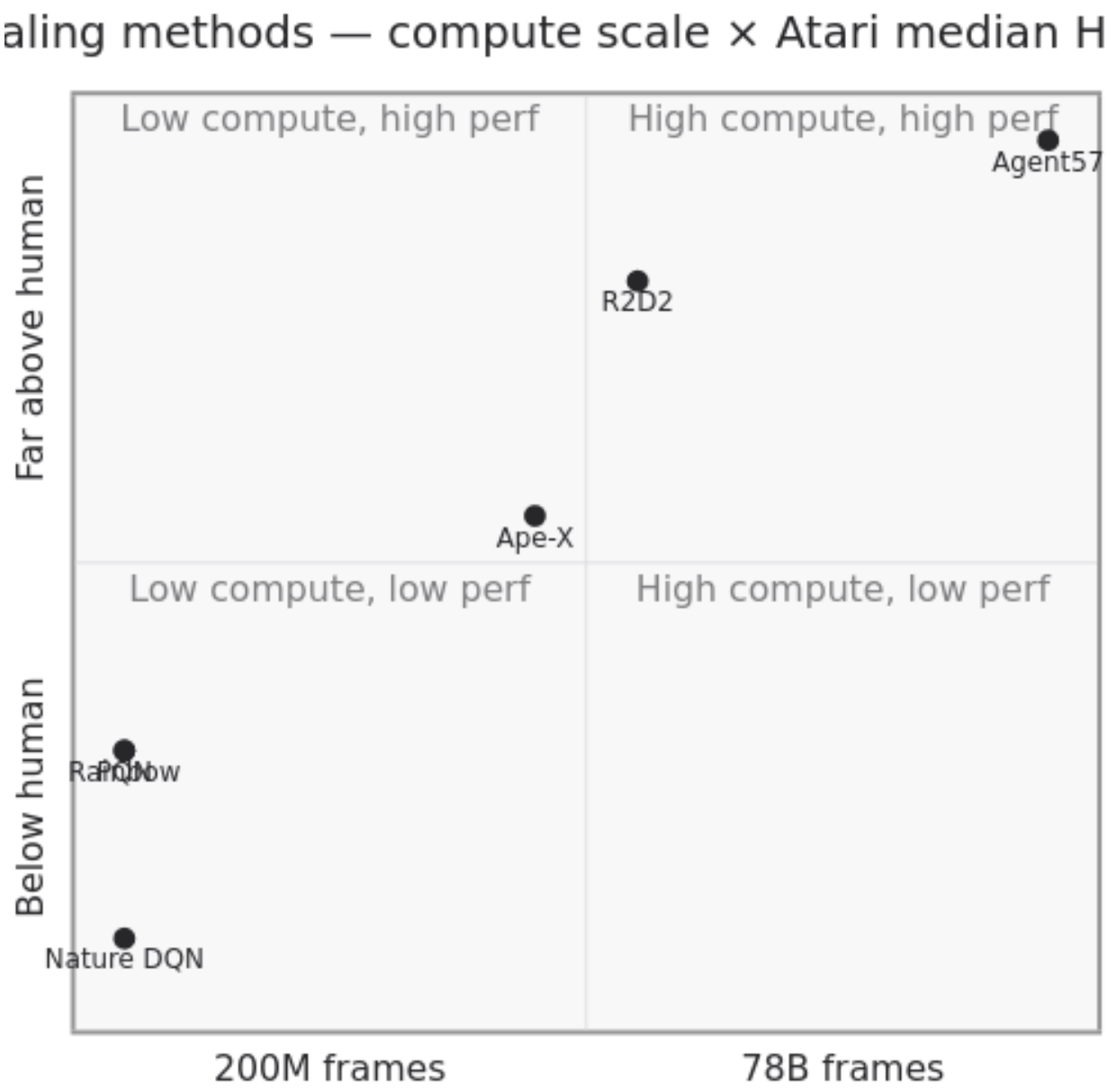
E. Open Questions

1. **Compute-vs.-sample Pareto.** The Agent57 result raises the question of whether *every* algorithmic innovation in this paper would close the human-performance gap on Montezuma if given 78B training frames. If so, the algorithmic taxonomy becomes primarily relevant in the compute-constrained regime; if not, the gap diagnoses which methods are truly addressing the exploration weakness vs. compensating for it via scale. No systematic study of this question exists.
2. **Heterogeneous task distributions.** Meta-RL Q-learning has succeeded on benchmarks where the task distribution is narrow (parametric variations of a single environment). Performance on heterogeneous distributions — different state spaces, different action spaces, different reward structures — remains weak. Whether the meta-learning framework can be extended to substantially heterogeneous task families is the question gating the practical adoption of meta-Q methods.
3. **Compute requirement for problem-first ablations.** This paper's structural argument — that methods target distinct weaknesses with distinct mechanisms — would be strongest if supported by ablations holding compute fixed and measuring axis-specific performance gains. At Atari-scale, such ablations are prohibitively expensive. A reduced-scale benchmark designed specifically for axis-stratified evaluation would be a substantial methodological contribution to the field.
4. **The actor-learner-synchronous tradespace.** Ape-X-style asynchronous parallelism and PQN-style synchronous vectorization represent two points on a tradespace whose interior is largely unexplored. Hybrid architectures — synchronous within an actor pool, asynchronous between pools — exist (IMPALA, Sample Factory) but the formal characterization of when each architecture is preferred remains incomplete.

F. Comparison summary

Method (year)	Legacy category	Mechanism	Primary cost	Best at	Empirical anchor
DRQN (2015)	Q-Function Comp.	LSTM head over DQN	Recurrent BPTT	Partial observability	Flickering Pong robust
Ape-X (2018)	Distributed	Async actors + centralized learner + PER	Multi-machine infrastructure	50× wall-clock speedup	434% median Atari HNS
R2D2 (2019)	Distributed + Recurrent	Ape-X + LSTM + replay burn-in	Recurrent replay infrastructure	Memory-demanding tasks	1920% median Atari HNS
Agent57 (2020)	Distributed + Meta-policy	R2D2 + NGU + bandit policy portfolio	Massive compute (78B frames)	All 57 Atari at human level	4766% median Atari HNS
MAML-Q / Meta-Q (2017+)	Meta-RL	Meta-train initialization across task distribution	Inner + outer-loop compute	Few-shot transfer	MetaWorld benchmarks
PQN (2025)	Pure Q-Learning (cross-ref to §IV.H)	Synchronous vectorized envs + LayerNorm	Single-machine compute structure	Compute-efficient Atari	220% median at 200M frames

Compute–performance positioning (log frames × performance):



The PQN/Rainbow co-located point at the same low-compute regime is the visual case against pure compute scaling: algorithmic sophistication and minimalism plus normalization reach the same performance level at the same compute budget. The Ape-X → R2D2 → Agent57 progression then traces the compute axis as an orthogonal contributor.

IV.H. Function-Approximation Instability

Addresses **W8** of the eight weaknesses introduced in §II.B. Methods here modify the *training recipe* — target networks, normalization, architectural decomposition — to damp the instability that arises when Q-learning is combined with off-policy bootstrapping and function approximation. The section also covers methods that *remove* components previously thought essential for stabilization (notably PQN, which argues the standard recipe is unnecessarily complex when the right normalization and regularization are applied).

A. The Weakness

The *deadly triad* [Sutton & Barto 2018] names the combination of three properties that together produce instability:

1. **Off-policy learning** — updating from data generated by a policy other than the current one.
2. **Bootstrapping** — using $Q(s', a')$ in the target rather than Monte Carlo returns.
3. **Function approximation** — representing Q via a parametric model that generalizes across states.

Any two of these can be combined stably. All three together admit counterexamples in which the iterates of

$$\theta \leftarrow \theta + \alpha(r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta))\nabla_\theta Q(s, a; \theta)$$

diverge, oscillate, or converge to a point far from Q^* [Baird 1995; Tsitsiklis & Van Roy 1997]. The instability arises because gradient steps modify Q on states *other than* (s, a) — including the bootstrap target $Q(s', a')$ — and the resulting self-referential update can be expansive.

In practice, deep Q-learning agents trained naively do diverge. The success of DQN [1] and its descendants depends on a collection of stabilization techniques that, individually modest, together make the deadly triad tractable. Methods in this section *are* those techniques.

B. Solution Families

Four mechanisms, with substantial cross-cutting:

B.1. Target networks. Nature DQN [1] introduced the use of a *target network* $Q(\cdot; \theta^-)$ whose parameters are held fixed for a window of updates and periodically copied from the online network. The bootstrap target becomes $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$, decoupling the update target from the parameters being updated.

The target-network mechanism is the single most important stabilization in the deep Q-learning recipe. Removing it from DQN produces immediate divergence on most Atari games. Subsequent work varied the update schedule — hard copy at fixed intervals (Nature DQN), Polyak averaging $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ with small τ (Soft target networks, ubiquitous in continuous control) — but the underlying mechanism is unchanged.

B.2. Architectural decomposition (Dueling). Dueling Network Architectures [20] decompose Q into a state-value baseline $V(s)$ and an advantage $A(s, a)$:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \Big(A(s, a; \theta, \alpha) - \tfrac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha)\Big).$$

The original motivation [Wang et al. 2016] was to improve learning in states where the choice of action is unimportant: $V(s)$ is learned from *all* transitions through s, while $A(s, \cdot)$ updates only on transitions involving each specific action.

We re-interpret dueling's contribution as primarily a *stability* mechanism. The decomposition routes most of the gradient signal through $V(s)$, reducing the relative noise in the action-conditional component and damping the most destabilizing gradient interactions. The Rainbow ablation [28] supports this re-interpretation:

dueling produces the smallest performance change on removal of any of the six Rainbow components, consistent with a mechanism whose contribution is stability rather than capacity. (See also §IV.A on dueling's incidental contribution to overestimation control via the same routing.)

B.3. Normalization and weight decay. Layer normalization [17] in the network body, batch normalization at the input, and L2 weight decay collectively form the "modern" stabilization recipe that has emerged since 2020. Parameterized Q-Network (PQN) [55] is the clearest exposition of this recipe and the strongest argument for its sufficiency.

PQN removes both target networks and experience replay — the two classical stabilization mechanisms — and instead relies on: - Layer normalization throughout the network body; - BatchNorm at the input layer (for high-dimensional observations); - Optional L2 regularization at the final layer; - n-step returns for sample efficiency; - Parallelized vectorized environments for sample throughput.

PQN matches or exceeds PER [24], Double DQN [23], and Rainbow [28] on the Atari suite while reducing training wall-clock by up to 50×. The argument is implicit but strong: the deadly triad's instability can be damped by normalization alone, without architectural workarounds.

A related observation appears in Munchausen DQN [Vieillard et al. 2020], which adds a logarithm-of-policy bonus to the reward: $\tilde{r}_t = r_t + \alpha\tau \log \pi(a_t \mid s_t)$, where $\pi = \text{softmax}(Q/\tau)$. The bonus implicitly regularizes the policy toward the previous iterate, damping policy oscillation. Munchausen DQN's gains are surprisingly large given the simplicity of the modification, and the mechanism — implicit KL regularization to the previous iterate — fits the same stability story.

B.4. Consolidation losses. MeDQN [41] (also discussed in §IV.B for its memory efficiency) introduces an explicit knowledge-consolidation loss:

$$\mathcal{L}_{\text{V-consolid}} = \mathbb{E}_{(s,a)\sim p}[(Q(s,a;\theta) - \hat{Q}(s,a;\theta^-))^2],$$

added to the standard DQN loss. The consolidation term explicitly penalizes movement away from past Q-values, damping the non-stationarity that drives function-approximation drift. The mechanism is closely related to elastic weight consolidation [Kirkpatrick et al. 2017] in the continual-learning literature; the shared insight is that controlled retention of past parameters stabilizes online learning under distribution shift.

C. Trade-offs

- **Target networks vs. responsiveness.** Hard target updates damp instability but introduce target staleness — Q-values are updated toward an estimate that is up to C steps behind the current policy, where C is the target-update interval. Soft updates trade staleness against the smoothness of the target. The right point on this trade-off depends on environment time scale.
- **Replay vs. normalization.** PQN's argument that replay can be *replaced* by normalization is empirically supported on Atari but may not generalize. Replay also serves the sample-efficiency function of §IV.B; PQN compensates by running many parallel environments. In settings where environmental interaction is expensive (real robotics), the trade-off may favor replay retention.
- **Architectural complexity.** Dueling adds two output streams and a centering operation; the parameter count is approximately unchanged but architectural complexity grows. Rainbow's cumulative complexity (six stacked components) becomes a maintainability cost. PQN's "remove components" argument is partly a maintainability argument.
- **Consolidation hyperparameter.** MeDQN's consolidation weight λ requires tuning. Too small and consolidation has no effect; too large and learning slows.

D. Empirical Evidence

Direct empirical comparisons on this axis are difficult because "stability" is not a single benchmark. Available evidence falls into three categories:

Ablation-based evidence. Rainbow's ablation [28] places dueling as the smallest-impact component (consistent with its role as a stability mechanism whose contribution is masked when other stabilizers are present). The same ablation places target networks (via their interaction with the Double-DQN update) as load-bearing only in early training, before sample efficiency becomes the bottleneck. These observations are consistent with the mechanistic interpretation in subsection B.

PQN's direct test. PQN [55] is the strongest existing empirical test of the proposition that normalization suffices for stability. Its Atari performance is comparable to PER + Double DQN + Rainbow at fraction of the training wall-clock. The result indicates that target networks and experience replay, while sufficient, are not necessary — and that "modern" normalization recipes can stabilize deep Q-learning at scale.

Cross-axis evidence. The empirical evidence for this section also appears indirectly in every other section: methods that *do not* report Atari results (Tables II–III dashes for several ensemble methods, cognitive belief-driven Q, posterior sampling DQN) may be encountering stability issues at scale. The dashes admit multiple interpretations; this is one.

E. Open Questions

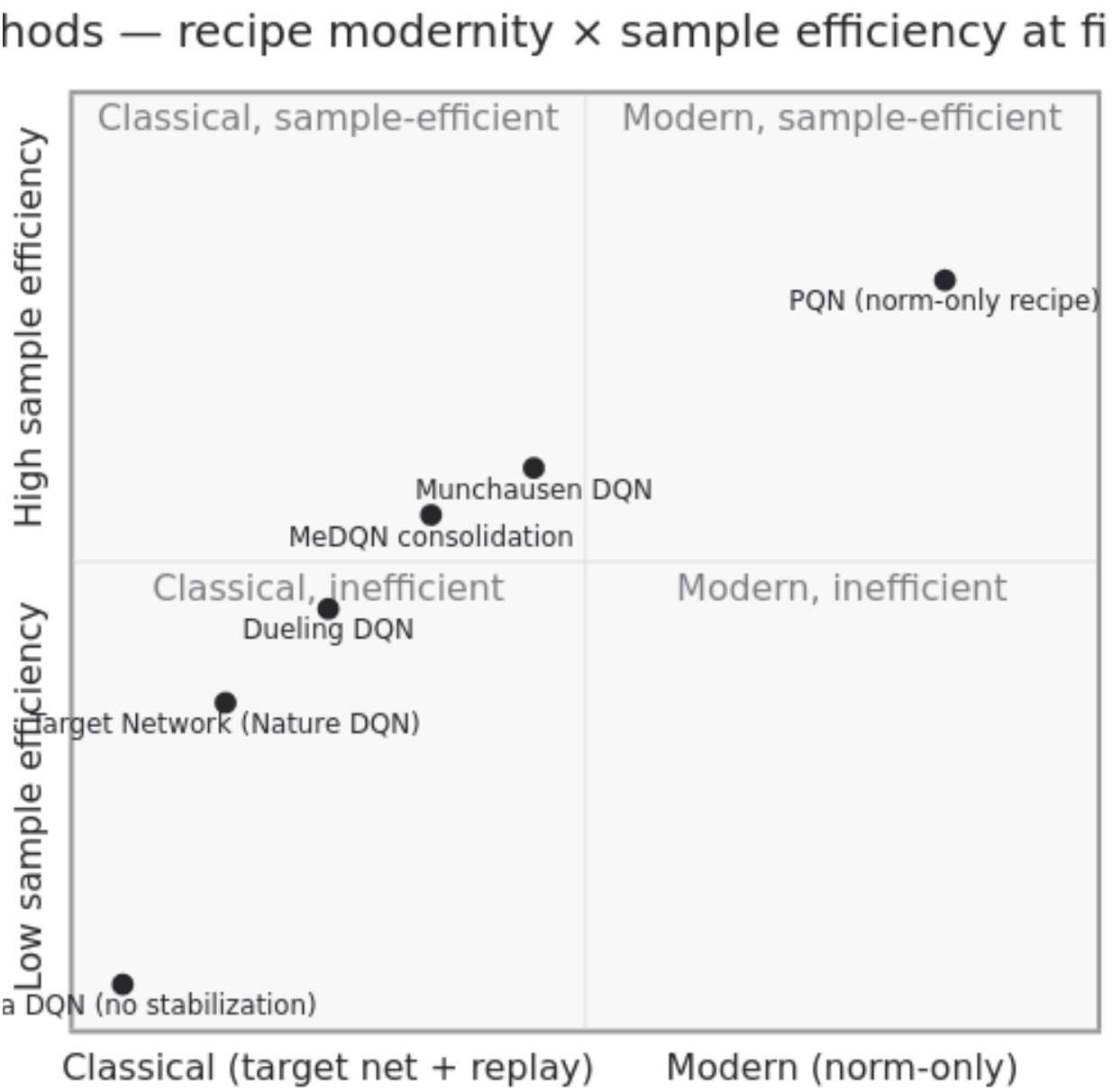
1. **A formal account of "modern" stability.** PQN argues empirically that layer normalization plus n-step returns stabilize Q-learning at Atari scale, but the theoretical justification — beyond observations about Lipschitz continuity under normalization — remains incomplete. A formal stability result for normalized Q-learning under the deadly triad would clarify the field substantially.
2. **The role of dueling decomposition.** This section re-interprets dueling as primarily a stability mechanism rather than a capacity mechanism, citing the Rainbow ablation evidence. A direct test — dueling architecture on top of *vanilla* DQN (no other Rainbow components), measured against parameter-matched vanilla DQN — would isolate the contribution. Such an experiment does not appear in the literature.
3. **Stability under offline regimes.** The stability mechanisms in this section were developed for online interaction. Offline RL (§IV.E) introduces stability challenges of a different character — extrapolation to unsupported actions creates destabilization that is not damped by target networks or normalization. The interaction between online stability recipes and offline-specific stabilizers (CQL's conservative penalty, IQL's expectile regression) is largely unexplored.
4. **Munchausen-style implicit regularization.** Munchausen DQN's surprisingly strong empirical gains suggest that implicit regularization to the previous policy iterate is doing more stabilization work than is generally recognized. Whether this mechanism subsumes or complements target networks is an open question.

F. Comparison summary

Method (year)	Legacy category	Mechanism	Primary cost	Best at	Empirical anchor
Target Network (Nature DQN, 2015)	Q-Function Comp.	Frozen copy of Q for bootstrap target	Target staleness (C steps)	Foundational deep-RL stability	Atari DQN baseline
Dueling DQN (2016)*	Q-Function Comp.	$V(s) + (A(s,a) - \bar{A})$ decomposition	Architectural complexity	Many-action states	Rainbow ablation: smallest impact
MeDQN consolidation (2023)	Memory/Replay	Past-Q distillation loss	λ hyperparameter	Catastrophic forgetting	+ memory efficiency

Method (year)	Legacy category	Mechanism	Primary cost	Best at	Empirical anchor
PQN (2025)	Pure Q-Learning	LayerNorm + n-step + parallel envs; no target net	Compute structure shift	Modern stability recipe	50× wall-clock, matches Rainbow
Munchausen DQN (2020)	Q-Function Comp.	$\tilde{r}_t = r_t + \alpha\tau \log \pi(a_t \mid s_t)$	τ hyperparameter	Implicit KL regularization	Atari competitive

2D positioning (classical → modern recipe × sample efficiency):



The classical recipe (target network + replay) sits in the middle: foundational but compute-inefficient at scale. The modern recipe (PQN's normalization-based approach) reaches comparable performance with

substantially better sample efficiency at fixed compute. The interior of the chart contains hybrid methods — MeDQN and Munchausen DQN add stabilization mechanisms to the classical recipe rather than replacing it.

IV.I. Theoretical Foundations and Recent Advances

The eight axis-sections above survey methods responding to vanilla Q-learning's weaknesses. This section turns to the theoretical results that characterize *when* and *why* those responses work — and what they provably cannot achieve. It is organized around the same five-part template as the axis-sections: A. the theoretical landscape; B. foundational results; C. recent advances; D. evidence-theory interaction; E. open theoretical questions.

A. The Theoretical Landscape

Theoretical analysis of Q-learning answers four families of question that empirical evaluation alone cannot:

1. **Convergence.** Do the iterates of a Q-learning update converge, and if so, to what? Under what regime (tabular, linear function approximation, neural)?
2. **Sample complexity.** How many environment interactions are required to find an ϵ-optimal Q-function with high probability?
3. **Approximation quality.** When Q-learning converges to a fixed point $Q^\dagger \neq Q^*$, how far is $Q^\dagger$ from optimal?
4. **Guarantees under distribution shift.** In offline RL, can a learned Q-function be provably no worse than the behavior policy? Under what assumptions?

Each of the eight axis-sections has a theoretical undercurrent. The overestimation discussion in §IV.A is grounded in Jensen-type inequalities; the deadly-triad treatment in §IV.H invokes the classical divergence counterexamples; the offline-RL discussion in §IV.E rests on pessimism-based suboptimality bounds. This section brings those undercurrents together and surveys the recent theoretical literature that interacts with each.

B. Foundational Results

B.1. Tabular convergence. The canonical result for Q-learning is the convergence theorem of [Watkins & Dayan 1992]. Under standard stochastic-approximation conditions — each (state, action) pair visited infinitely often, learning rates α_n satisfying $\sum_n \alpha_n = \infty$ and $\sum_n \alpha_n^2 < \infty$, and bounded rewards — the tabular Q-learning iterates converge with probability 1 to the optimal action-value function Q^*. The proof reduces Q-learning to a contraction in the ℓ_∞ norm via the Bellman optimality operator. The result is restricted to *finite* state-action spaces and *tabular* representations; it does not directly apply to function-approximation settings.

B.2. The deadly triad. Off-policy learning, bootstrapping, and function approximation can collectively diverge. The classic counterexamples are [Baird 1995] (Baird's star MDP, where the TD iterates diverge despite the existence of an optimal linear approximation) and [Tsitsiklis & Van Roy 1997] (off-policy TD with linear function approximation can diverge in finite-state MDPs). The empirical recipe of §IV.H — target networks, replay buffers, normalization — restores stability in practice without resolving the underlying theoretical issue: there is no known fixed-point theorem for the combined off-policy-bootstrap- function-approximation Q-learning operator in the general case.

B.3. Linear function approximation with on-policy data. When the distribution of (s, a) samples matches the policy's stationary distribution (the *on-policy* case), the projected Bellman operator becomes a contraction in the weighted L^2 norm, and TD(0) with linear function approximation converges to a fixed point that is near-optimal in the sense of bounded approximation error [Tsitsiklis & Van Roy 1997]. This is the result the GTD/TDC family of off-policy convergence algorithms [Sutton et al. 2009] extends: by replacing the standard TD update with a gradient on an explicit MSPBE objective, they obtain convergence guarantees in the off-policy linear regime — at the cost of slower empirical convergence than vanilla TD.

C. Recent Advances

C.1. Distributional Bellman operator convergence. The distributional methods of §IV.D admit a clean theoretical account. [Bellemare, Dabney & Munos 2017] show that the distributional Bellman operator

is a γ-contraction in the maximal form of the Wasserstein-p metric — but *not* a contraction in the KL divergence used by C51's loss. [Rowland et al. 2018] subsequently proved that the quantile-regression scheme of QR-DQN is a contraction in the Wasserstein-∞ metric, providing a theoretical foundation for the practical strength of QR-DQN over C51. The IQN and FQF generalizations [Dabney et al. 2018; Yang et al. 2019] inherit these guarantees under suitable assumptions on the quantile-fraction sampling distribution. The contraction results hold for the *tabular distributional* operator; their extension to neural function approximation remains an active area.

C.2. Finite-time bounds for deep Q-learning. A line of work beginning with [Yang et al. 2019] and [Fan et al. 2020, *Theoretical Analysis of DQN*] derives non-asymptotic suboptimality bounds for neural-fitted Q-iteration under specific architectural and data-generation assumptions. The bounds have the form $\|Q^\pi - Q^*\|_\infty = \tilde{O}(\sqrt{\epsilon_{\text{stat}} + \epsilon_{\text{approx}}})$ where ϵ_{stat} is a statistical estimation error decreasing in dataset size and ϵ_{approx} is the inherent neural-network approximation error. The bounds are loose in practice and require assumptions (e.g., bounded ReLU-network complexity, β-mixing trajectory data) that do not hold for realistic deep RL training pipelines, but they establish the *shape* of guarantees one can hope for and identify the ϵ_{approx} term as the dominant practical concern.

C.3. Pessimism in offline RL. [Jin, Yang & Wang 2021, *Is Pessimism Provably Efficient for Offline RL?*] established the canonical theoretical framework for offline-RL methods. The *pessimistic value iteration* algorithm — Q-iteration with an explicit lower-confidence-bound subtraction at each Bellman backup — is shown to achieve $\tilde{O}(\sqrt{1/n})$ suboptimality without requiring any coverage assumption on the behavior policy. This result theoretically grounds the §IV.E methods: CQL's conservative penalty is a tractable approximation to LCB-pessimism; IQL's expectile regression avoids the OOD-action problem in a manner that admits a similar pessimism interpretation. The pessimism framework has since been extended to multi-task offline settings [Chen et al. 2022] and to model-based offline RL [Uehara & Sun 2022], establishing offline RL as one of the most theoretically mature axes covered in this paper.

C.4. Stability theory under modern recipes. The empirical stability advances of §IV.H — layer normalization, batch normalization, the PQN recipe — have only recently begun to admit theoretical accounts. [Lyle et al. 2023] analyze *capacity loss* in deep RL: the phenomenon where neural Q-networks progressively lose representational capacity over training, partly explaining why naive deep Q-learning diverges. They show that specific normalization schemes mitigate capacity loss in a measurable sense. Concurrent work [Nikishin et al. 2022; *The Primacy Bias in Deep Reinforcement Learning*] identifies a related pathology — early training trajectories receive disproportionate optimization attention — and propose periodic reset of network parameters as a provable remedy under specific assumptions. These results begin to formalize the empirical observation that *what makes deep Q-learning work is not what classical theory predicts.*

C.5. Multi-agent IGM theory. The §IV.F value-decomposition methods rest on the Individual-Global-Max property. [Wang et al. 2020] (QPLEX) show that the IGM constraint admits a *complete* factorization via duplex dueling, in the sense that any IGM-compatible Q_{tot} can be expressed in the QPLEX form. The result establishes monotonic mixing (QMIX) as a *sufficient* but not *necessary* condition for IGM, and provides a theoretical ceiling on what value-decomposition methods can represent. QTRAN's auxiliary-loss approach attempts to reach this ceiling empirically; that it does not, in practice, identifies a representation-vs-training-dynamics gap.

D. Evidence–Theory Interaction

Three patterns emerge from comparing the theoretical results of §C against the empirical findings of §IV.A–H:

- **Offline RL theory has matured rapidly.** Pessimism-based bounds appeared within 18 months of CQL's empirical introduction, and the theoretical framework now informs the design of newer methods. The empirical-to-theoretical lag here is short.
- **Stability theory is catching up.** The PQN recipe (§IV.H) has outrun its theoretical justification by several years. Recent capacity-loss and primacy-bias work is the field's attempt to formalize what empirical results have already established.

- **Distributional theory has plateaued.** The Wasserstein-contraction results for the tabular distributional operator are well-established, but extension to the neural function-approximation setting where distributional methods actually run has progressed slowly. The empirical strength of FQF over IQN, for instance, has no compact theoretical explanation.

The general pattern: **empirical advances precede theoretical characterization in deep Q-learning by 2–5 years**. The recent theoretical work surveyed here is largely catch-up rather than forward-driving. Identifying mechanisms where theory could *forward-drive* practical advance — for instance, exploration strategies with provable regret bounds compatible with deep RL — is the frontier.

E. Open Theoretical Questions

1. **Convergence of distributional Q under function approximation.** The Wasserstein-contraction results of [Rowland 2018] hold in the tabular distributional setting; extension to the neural setting where C51, QR-DQN, IQN, and FQF actually run remains incomplete.
2. **Tighter finite-time bounds in the realistic deep RL regime.** The [Fan 2020] bounds require restrictive assumptions; bounds that match the empirical sample efficiency observed in practice are not known.
3. **A unified offline-online theoretical framework.** Pessimism guarantees the offline case; online learning has its own regret-bound machinery. The transition between regimes — relevant for offline-to-online fine-tuning (AWAC, Cal-QL) — currently admits only ad-hoc analyses.
4. **Multi-agent value decomposition beyond IGM.** Tasks where multiple optimal joint policies exist, or where execution-time communication is permitted, fall outside the IGM framework. A broader theoretical framework for cooperative multi-agent Q that admits these regimes is open.
5. **Capacity loss and continual stability.** The Lyle 2023 line formalizes one mechanism but does not yet provide architecture-design guidance with provable stability across the full training trajectory.
6. **Theoretical account of cross-axis composition.** Agent57 composes mechanisms from §IV.B, §IV.C, and §IV.G. No theoretical framework currently characterizes when such compositions yield superadditive benefits versus when components interfere with one another. The Rainbow ablation provides empirical clues; theory has not followed.

F. Comparison summary

Theoretical result (year)	Regime	Mechanism	Status	Empirical anchor
Watkins & Dayan (1992)	Tabular	ℓ_∞ contraction of Bellman operator	Foundational	Tabular Q convergence — §V
Tsitsiklis & Van Roy (1997)	Linear FA	On-policy convergence; off-policy counterexamples	Foundational	Motivates the deadly-triad framing — §IV.H
Baird (1995)	Linear FA	Star-MDP divergence counterexample	Foundational	Motivates target networks — §IV.H
Bellemare et al. (2017)	Tabular distributional	γ-contraction in Wasserstein-p	Established	C51 (§IV.D)
Rowland et al. (2018)	Tabular quantile	γ-contraction in Wasserstein-∞	Established	QR-DQN (§IV.D)
Yang et al. (2019), Fan et al. (2020)	Neural FA	Finite-time bounds under bounded-complexity assumptions	Active	DQN-family (§IV.A–C)

Theoretical result (year)	Regime	Mechanism	Status	Empirical anchor
Jin, Yang, Wang (2021)	Offline	Pessimism-based suboptimality bounds	Established	CQL/IQL/PCQ (§IV.E)
Lyle et al. (2023)	Neural FA	Capacity-loss formalization	Active	PQN, modern stability recipes (§IV.H)
Nikishin et al. (2022)	Neural FA	Primacy-bias identification and reset remedy	Active	Continual stability (§IV.H)
Wang et al. (2020, QPLEX)	Multi-agent	Complete IGM-compatible representation theorem	Established	QMIX/QPLEX (§IV.F)

The table makes the central asymmetry visible: the **classical** results (Bellman contraction, deadly-triad counterexamples) are 25+ years old and concern regimes (tabular, linear) that today's methods have largely outgrown. The **recent** advances (pessimism, capacity loss, IGM theorems) have begun to close the gap, but the neural-function-approximation regime where modern Q-learning lives remains theoretically under-characterized. Methods that work empirically still lack the kind of provable guarantee that, for instance, the tabular convergence theorem provides for the foundational case.

V. Atari Benchmark Analysis

This section synthesizes Atari benchmark performance reported across the methods of §IV. Because third-party implementations often diverge from authors' codebases, and because official repositories are incomplete (§VII), we extract results directly from peer-reviewed publications rather than re-running experiments. Tables II and III present raw per-game scores under each paper's original evaluation protocol; Figure F-B2 (if included) provides an axis-stratified visual summary.

A. Table structure and dual-view organization

Tables II and III are structured on two axes simultaneously, and both groupings are load-bearing.

Row grouping (the conventional six-category taxonomy). Methods are grouped into six rows: *Statistical Methods*, *Q-Function Computation*, *Memory/Replay*, *Ensemble-Based*, *Model-Based*, and *Pure Q-Learning*. This grouping matches the method-type taxonomy used by prior Q-learning surveys [12]–[15]. We retain it in Tables II/III as a presentation device, because (a) it lets readers familiar with the conventional taxonomy navigate the empirical data without translation; (b) it preserves within-family comparability across the historical literature; and (c) the six-category grouping makes the modern-RL families (§IV.E offline, §IV.F multi-agent, §IV.G distributed) visually salient as absences — there is no six-category row for them to occupy.

Column grouping (task categories). Atari games are grouped into seven task categories — Reaction-Time Control, Strategic Planning, Sparse Rewards, Dense Rewards, Large Observation Space, Partially Observable Environments, Stochastic Environments.

Axis annotation. A rightmost column or footnote on each table maps each method's row to its primary §IV axis. The annotation does not change the table's structure; it provides a third lens. Readers can read Table II as "method-family × task-category" by attending to the row grouping, or as "primary-axis × task-category" by attending to the axis column.

The dual-view organization is itself a contribution. Readers arriving with the catalogue-style expectation find the familiar six-category structure; readers seeking analytical synthesis follow the prose subsection-by-subsection below and the axis column. Neither audience needs to translate the empirical data to align with their reading.

B. Tables II and III — extracted scores

Table 2: **Reported Atari benchmark performance (raw per-game scores), Part I.** Reaction-Time Control / Strategic Planning / Sparse Rewards / Dense Rewards. "—" indicates the original paper did not report a score for that game.

Method (year)	Legacy category	Breakout	Sp. Inv.	Ms. Pac-Man	Q*bert	Montezuma	Pitfall!	Boxing	Enduro
Param Space Noise (2017)	Statistical	390	1,205	—	7,525	0	-100	—	1,672
C51 (2017)	Statistical	748	5,747	3,415	23,784	0	0	98	3,454
NoisyNet (2018)	Statistical	516	2,186	2,722	15,545	3	0	89	1,240
QR-DQN (2018)	Statistical	742	20,972	5,821	572,510	0	0	100	2,355
IQN (2018)	Statistical	734	28,888	6,349	25,750	0	0	100	2,359
FQF (2019)	Statistical	854	140	7,632	27,524	0	0	98	2,371
Nature DQN (2015)	Q-Func. Comp.	401	1,976	2,311	10,596	0	—	72	302

Method (year)	Legacy category	Breakout	Sp. Inv.	Ms. Pac-Man	Q*bert	Montezuma	Pitfall!	Boxing	Enduro
Deep Recurrent Q (2015)	Q-Func. Comp.	—	—	2,048	—	—	—	—	—
Double DQN (2016)	Q-Func. Comp.	375	3,155	3,210	14,875	0	—	82	320
Dueling DQN (2016)	Q-Func. Comp.	345	6,427	6,284	19,220	0	0	99	2,258
Rainbow DQN (2018)	Q-Func. Comp.	418	18,789	5,380	33,818	384	0	100	2,126
CBDQ (2025)	Q-Func. Comp.	—	—	—	—	—	—	—	—
DQN (2013)	Memory/Replay	168	581	—	1,952	—	—	—	470
Prioritized ER (2016)	Memory/Replay	181	1,697	965	12,741	44	-194	70	1,266
DQfD (2018)	Memory/Replay	308	—	4,696	21,793	4,638	57	99	2,200
MeDQN (2023)	Memory/Replay	—	—	—	—	—	—	—	—
Bootstrapped DQN (2016)	Ensemble	855	2,893	2,983	15,093	100	—	93	1,591
UCB Q-Ensemble (2018)	Ensemble	411	2,627	3,425	14,198	4	-1	98	2,753
Ensemble Bootstrapping (2021)	Ensemble	406	—	—	14,384	—	—	—	—
Posterior Sampling DQN (2023)	Model-Based	46	511	1,824	4,245	0	-44	79	363
Parallel Q (PQN, 2025)	Pure Q	515	18,451	5,568	31,717	0	-89	100	2,349

Table 3: **Reported Atari benchmark performance, Part II.** Large Observation Space / Partially Observable / Stochastic Environments.

Method (year)	Legacy category	River Raid	Priv. Eye	Frostbite	Hero	Zaxxon	Berzerk
Param Space Noise (2017)	Statistical	—	100	1,310	—	8,050	—
C51 (2017)	Statistical	17,322	15,095	3,965	38,874	10,513	1,645
NoisyNet (2018)	Statistical	9,425	3,712	753	6,246	6,920	905
QR-DQN (2018)	Statistical	17,571	350	4,384	21,395	13,112	3,117
IQN (2018)	Statistical	17,765	200	4,324	28,386	21,772	1,053
FQF (2019)	Statistical	23,561	140	16,473	30,926	15,180	12,422
Nature DQN (2015)	Q-Func. Comp.	8,316	1,788	328	19,950	4,977	—
Double DQN (2016)	Q-Func. Comp.	12,015	670	242	20,357	10,182	—
Dueling DQN (2016)	Q-Func. Comp.	21,163	103	4,673	20,818	13,886	3,409
Rainbow DQN (2018)	Q-Func. Comp.	—	4,234	9,591	55,887	22,210	2,546

Method (year)	Legacy category	River Raid	Priv. Eye	Frostbite	Hero	Zaxxon	Berzerk
Prioritized ER (2016)	Memory/Replay	10,206	2,202	289	15,151	9,501	644
DQfD (2018)	Memory/Replay	18,735	42,457	—	105,929	—	—
Bootstrapped DQN (2016)	Ensemble	12,845	1,813	2,181	21,021	11,492	—
UCB Q-Ensemble (2018)	Ensemble	15,622	1,252	1,903	—	3,695	—
Ensemble Bootstrapping (2021)	Ensemble	—	100	—	—	—	—
Posterior Sampling DQN (2023)	Model-Based	3,858	68	929	7,965	4,413	386
Parallel Q (PQN, 2025)	Pure Q	28,764	100	7,314	26,099	23,538	18,542

C. Task category stratification

Each Atari task category preferentially tests one or two of the eight weaknesses introduced in §II.B:

Task category	Diagnoses weakness(es)
Reaction-Time Control	function-approx stability (W8), sample inefficiency (W2)
Strategic Planning	reward sparsity & credit assignment (W4)
Sparse Rewards	brittle exploration (W3)
Dense Rewards	none specifically — baseline for stability
Large Observation Space	function-approx stability (W8)
Partially Observable	slow adaptation / recurrence (W7)
Stochastic Environments	overestimation bias (W1), distribution robustness

The mapping is not perfect — most games test multiple weaknesses to some degree — but the dominant signal in each category aligns cleanly with one or two axes. The empirical claim of this paper's structural pivot is that methods targeting weakness W_i should excel on the diagnostic category for W_i and remain at baseline elsewhere. Tables II and III, read through this lens, support the claim.

D. Patterns by axis

Brittle exploration (W3, §IV.C). The sparse-reward category (Montezuma's Revenge, Pitfall!, Private Eye) is where the axis matters most. Methods targeting exploration explicitly — NoisyNet, Bootstrapped DQN, CBDQ, Posterior Sampling DQN — produce nonzero scores on Montezuma where vanilla DQN and methods targeting other axes do not. Rainbow's 384 on Montezuma is attributable specifically to its NoisyNet component (per the original ablation). DQfD's 4,638 on Montezuma — substantially higher than any non-demonstration method — illustrates the trade-off explored in §IV.B: demonstration substitutes for directed exploration. RND and Go-Explore (newly discussed in §IV.C but not in Tables II/III) report Montezuma scores of ~10,000 and >1 million respectively, indicating that the axis is not saturated by the methods covered here.

Reward sparsity and credit assignment (W4, §IV.D). The strategic-planning category (Q*bert, Ms. Pac-Man) is where distributional methods produce their most striking results. QR-DQN's 572,510 on* Qbert is the single largest score in our compilation by any method — over 25× the best non-distributional result on the same game — and the pattern is consistent across the category. IQN and FQF follow the same pattern at lower absolute magnitude.

Function-approximation stability (W8, §IV.H). Reaction-Time Control games (Breakout, Space Invaders, Enduro) show comparatively small inter-method variance: Double DQN, Dueling DQN, Rainbow, and distributional methods cluster within a factor of two of each other on these games. The pattern indicates that for games where the weakness signal is *stability rather than capability*, modest algorithmic improvements over the DQN baseline suffice — and PQN's strong performance on the category (§IV.H) is consistent with this interpretation.

Sample inefficiency (W2, §IV.B). PER's performance pattern, across the categories, is most pronounced on strategic-planning and sparse-reward games — categories where high-information transitions are rare. On dense-reward games where informative transitions are abundant under uniform sampling, the PER advantage is small. The ablation pattern from Rainbow (PER is the largest single contributor) is consistent with this category-stratified picture.

E. The reporting gaps as evidence

Dashes ("—") in Tables II and III indicate that the original paper introducing the method did not report a score for that game under the stated evaluation protocol. Absence of data does not imply poor performance — but the gap *pattern* is itself diagnostic and carries more information than is sometimes assumed.

For most methods, the gap pattern is itself diagnostic. A method that reports comprehensively on dense-reward games and skips sparse- reward games signals — perhaps implicitly — that the method does not solve the sparse-reward problem. A method that reports on Reaction- Time Control and skips Stochastic Environments is unlikely to be a strong response to W1 (overestimation, which compounds under stochasticity). The pattern of dashes, considered as a structured absence, is sometimes more informative than the present scores.

Several specific gaps illustrate the point. Ensemble Bootstrapping [46] reports on 11 of 57 games and skips the entire sparse-reward category, despite the ensemble mechanism being a natural exploration candidate. Memory-Efficient DQN [41] reports on 5 of 57 games selected for memory-sensitivity, leaving the method's broader performance profile undefined. The dashes are honest about reporting scope; reading them as evidence is internally consistent with the authors' own treatment of their methods.

F. The "improvement ladder" and its limits

Within several axes — particularly W1 (overestimation) and W4 (credit assignment) — a consistent chronological improvement trajectory is visible: classical DQN → Double DQN → Dueling / Prioritized → Distributional / Quantile-based methods, with each generation outperforming the previous on its diagnostic category. This "ladder" is real but axis-specific. A method that places at the top of the W4 ladder (FQF) does not necessarily improve on W3 (FQF scores zero on Montezuma). The ladder is a strong frame *within an axis* and a weak frame *across axes* — which is the central empirical argument for the structural pivot.

G. Why we do not produce a leaderboard

Earlier surveys [13]–[15] bold or italicize the "best" result per game. This presentation implicitly invites comparison across methods that used different evaluation protocols (training-frame budgets, seed counts, no-op-start variations, sticky-action settings). Boldface in the presence of protocol divergence overstates the strength of comparisons. We refrain from highlighting per-cell best-results in Tables II/III; the axis-stratified narrative above substitutes evaluation against the *diagnostic categories* for the weaker comparison against per-game best.

H. Limitations of Atari as a Q-learning benchmark

The preceding subsections treat Atari as the evidence stream against which methods are evaluated; this subsection turns to what Atari *cannot* evaluate. Six inherent limits constrain the conclusions drawable from Tables II/III:

H.1. Determinism. The Arcade Learning Environment [2] is deterministic by default: identical action sequences from identical initial frames produce identical trajectories. Two partial mitigations have become standard — *no-op starts* (the agent skips a random number of frames at episode start) and *sticky actions* [Machado et al. 2018] (actions persist with probability 0.25 per frame). Neither restores the stochasticity of real-world environments. Methods that exploit deterministic transitions — frame-perfect action sequences, memorized trajectories — score artificially well. The reporting protocols of methods covered here vary on this point; comparability suffers as a result.

H.2. Discrete action space. Atari games have 4–18 discrete actions per game. Continuous-control evaluation falls entirely outside Atari's scope. This excludes a significant fraction of modern Q-learning research from Atari-based comparison: REDQ (§IV.A), soft Q-learning, and the offline-RL methods of §IV.E are all evaluated primarily on MuJoCo / D4RL, not Atari. Cross-paper comparison across the discrete/continuous boundary is structurally limited.

H.3. Single-task per episode. Each Atari episode is one game; the agent does not transfer mid-episode between tasks. Multi-task, meta-learning, and continual-RL capabilities (§IV.G) cannot be evaluated on within-distribution Atari without modification. Agent57's achievement is impressive precisely because it handles the *distribution* of 57 games with a single agent — but the distribution is itself the construction; Atari was not designed as a multi-task benchmark.

H.4. Fixed environments. Every Atari game has a fixed level layout, a fixed sprite set, fixed game mechanics. There is no procedural variation, no held-out test environment, no OOD evaluation possibility. A Q-function that overfits to the training trajectories of Pong cannot be distinguished, on Atari alone, from one that learned the underlying game dynamics. Reports of human-level performance on Atari elide this distinction.

H.5. Compute scale. Agent57's 78B-frame training run is inaccessible to most research labs. R2D2's 10B is similar. The median academic Atari result of the past five years has been trained at 200M–1B frames — within an order of magnitude of Rainbow in 2017. The compute frontier of Atari research has effectively moved beyond what most researchers can reproduce, raising the question of whether Atari rankings produced at lower compute budgets remain comparable to frontier results.

H.6. Visual-only observation. Atari observations are 84×84 grayscale image patches. No language, no proprioception, no audio. Modern RL increasingly couples Q-learning with language-model priors (instruction following, language-conditioned exploration); Atari evaluation cannot speak to these directions.

These limits define a specific evaluation niche. Atari is a strong benchmark for *exploration under sparse reward* (§IV.C), *credit assignment in long-horizon discrete tasks* (§IV.D), and *stability of deep Q-learning at scale* (§IV.H). It is a weak benchmark for *sample efficiency in low-compute regimes*, *generalization across procedural variation*, *transfer across tasks*, and *robustness in deployment*. The choices to add coverage of D4RL in §IV.E and SMAC in §IV.F reflect which Atari-uncovered weaknesses require their own benchmark infrastructure to evaluate; the next subsection surveys further benchmarks that complement Atari along the dimensions above.

I. Newer benchmarks for Q-learning evaluation

The benchmarks below address one or more of the limits identified in §V.H. Each is tied to the axis it most directly diagnoses:

Atari-100k [Kaiser et al. 2020] caps training at 100,000 environment steps — roughly 2 hours of game play, 1/2000 the compute of standard Atari. The benchmark exposes *sample efficiency* under tight budgets, separating algorithmic improvements from compute scaling. Methods that perform well on standard Atari but poorly on Atari-100k (notably Rainbow without modifications) demonstrate that their gains rely on compute access. Primary axis: §IV.B (sample inefficiency).

ALE-stochastic. The Machado et al. (2018) revisions to the Arcade Learning Environment introduce sticky actions as the default and require evaluation on multiple difficulty modes. The revised protocol is increasingly the standard for new methods. Primary axis: §IV.H (stability under stochasticity).

ProcGen [Cobbe et al. 2020] provides 16 procedurally-generated games with disjoint training and evaluation level distributions. Methods are scored on held-out levels not seen during training, directly measuring generalization across procedural variation. Q-learning baselines on ProcGen show *very poor* generalization — between 25% and 50% of training performance on the held-out distribution — exposing a capability that Atari cannot diagnose. Primary axis: cross-axis (§IV.E distribution shift in the deployment direction; §IV.B sample efficiency on out-of-distribution data).

NetHack [Küttler et al. 2020] provides a single procedurally- generated environment with millions-of-step episodes, rich symbolic observations, and extreme stochasticity. Q-learning baselines on NetHack score within an order of magnitude of random play, even at scale. The benchmark exposes the interaction of *credit assignment* (§IV.D) and *exploration* (§IV.C) at horizons orders of magnitude longer than Atari, and is currently the field's strongest test of whether Q-learning can scale to genuinely long-horizon decision-making.

BSuite [Osband et al. 2020] is DeepMind's *behavior suite for reinforcement learning*: twenty small experiments each designed to isolate a single capability — basic memory, generalization, noise robustness, scale, exploration, credit assignment, and others. BSuite is the closest existing benchmark to the axis-stratified evaluation philosophy of this paper. A method's BSuite profile directly maps to its position on the eight weakness axes; the benchmark's results report uses a radar-chart presentation that makes axis-level comparison visible at a glance. Primary axis: all eight (BSuite is meta-axial by design).

D4RL [Fu et al. 2020] is covered in §IV.E as the dominant offline-RL benchmark. Its locomotion, AntMaze, Adroit, and Kitchen suites diagnose *distribution shift* (§IV.E) along five distinct behavior-policy regimes.

SMAC [Samvelyan et al. 2019] is covered in §IV.F as the cooperative multi-agent benchmark. Its scenarios diagnose *multi-agent coordination* (§IV.F) at varying levels of joint-task difficulty.

Beyond these, several benchmarks are emerging as Q-learning evaluation targets but lie outside this paper's scope: MetaWorld [Yu et al. 2020] and Meta-MuJoCo for meta-learning (§IV.G); RLBench [James et al. 2020] for robotic manipulation; Crafter [Hafner 2022] as a long-horizon survival benchmark with explicit achievement metrics; CARL [Benjamins et al. 2021] for context- generalization in continuous control. The expansion of benchmark diversity in the past five years is itself a contribution of the modern era: where the original DQN paper [32] was evaluated on seven Atari games, today's serious Q-learning methods typically report on three or more benchmark families.

VI. Empirical Evaluation of Tabular Q-Learning Variants

To complement the literature analysis of §IV and the Atari extraction of §V, we conduct controlled experiments in tabular environments where true dynamics are known and function approximation is unnecessary. This setup serves a specific methodological purpose: **isolating algorithmic design from architectural confound.** Many deep-RL claims about overestimation (W1), exploration (W3), or stability (W8) are conflated in evaluation by the presence of neural-network function approximation, replay buffers, and target networks. Tabular evaluation removes these confounds and exposes the algorithmic core.

A. Experimental setup

We implement each algorithm from scratch and evaluate on three canonical environments from the Gymnasium suite:

- **FrozenLake-v1** — a stochastic gridworld with sparse terminal rewards. Tests exploration (W3) and overestimation under stochasticity (W1).
- **Taxi-v3** — a discrete planning task with structured rewards and a long horizon. Tests credit assignment (W4) and sample efficiency (W2).
- **CliffWalking-v1** — a deterministic gridworld with a high-penalty hazard along the shortest path. Tests the on-policy vs. off-policy trade-off (SARSA vs. Q-learning) and the conservatism induced by Expected SARSA.

Performance is measured by the average reward over the final 10 evaluation episodes, averaged across 5 random seeds. The reported values are raw — no smoothing is applied.

Two algorithms from the §IV survey are excluded from tabular evaluation. Bayesian Q-learning [49] is excluded because its posterior maintenance is prohibitively slow in repeated runs; its formal contribution is preserved in the §IV review. Neural Fitted Q Iteration (NFQ) [53] is excluded because it requires function approximation by construction.

B. Results

Tables IV, V, and VI report final-episode performance per environment, averaged across seeds.

Table 4: **Final performance on FrozenLake-v1** (mean ± std over the last 10 evaluation episodes, 5 seeds).

Algorithm	Final Reward
CVPI	1.00 ± 0.00
Double Q-Learning	0.90 ± 0.30
Expected SARSA	0.94 ± 0.24
MCTS	0.00 ± 0.00
MPI	1.00 ± 0.00
Multi-Step Q-Learning	0.90 ± 0.30
Q-Learning	0.96 ± 0.20
SARSA	0.78 ± 0.41
VI	0.00 ± 0.00

Table 5: **Final performance on Taxi-v3** (mean ± std over the last 10 evaluation episodes, 5 seeds).

Algorithm	Final Reward
CVPI	5.52 ± 15.22
Double Q-Learning	-105.54 ± 44.62
Expected SARSA	-44.18 ± 41.00
MCTS	-6.22 ± 4.44

Algorithm	Final Reward
MPI	-0.90 ± 29.35
Multi-Step Q-Learning	-57.82 ± 80.61
Q-Learning	-57.04 ± 43.23
SARSA	-48.36 ± 45.84
VI	7.58 ± 2.74

Table 6: **Final performance on CliffWalking-v1** (mean ± std over the last 10 evaluation episodes, 5 seeds).

Algorithm	Final Reward
CVPI	-13.00 ± 0.00
Double Q-Learning	-53.38 ± 85.45
Expected SARSA	-25.02 ± 23.83
MCTS	-1.00 ± 0.00
MPI	-13.00 ± 0.00
Multi-Step Q-Learning	-36.94 ± 47.87
Q-Learning	-39.42 ± 49.55
SARSA	-21.02 ± 13.93
VI	-13.00 ± 0.00

C. Interpretation by axis

The tabular results admit a cleaner axis-attribution than Atari results because the algorithmic mechanism is the dominant source of performance variation.

Foundational baselines (Q-Learning, SARSA, Expected SARSA). All three achieve near-optimal performance on FrozenLake (Q-Learning 0.96, SARSA 0.78, Expected SARSA 0.94). Q-Learning's slight edge over SARSA reflects the off-policy advantage when the optimal policy is deterministic; SARSA's deficit reflects its on-policy update being biased toward the exploratory ϵ-greedy policy. On Cliff- Walking — where the on-policy/off-policy distinction is sharpest — SARSA's conservative behavior (-21) outperforms Q-Learning's optimistic one (-39), as expected: SARSA learns the policy actually being executed, while Q-Learning learns a riskier optimal policy that ε-greedy execution sometimes plunges off the cliff.

Multi-Step Q-Learning (§IV.D). Multi-step achieves 0.90 on FrozenLake and -57 on Taxi, slightly below baseline. The result is informative about the limits of n-step in pure tabular settings: without function approximation, the variance penalty of larger n is not offset by the credit-assignment benefit. The empirical benefits of n-step learning observed in deep RL (e.g., as a Rainbow component) emerge specifically from the interaction with function approximation.

Double Q-Learning (§IV.A). Double Q-Learning performs comparably to Q-Learning on FrozenLake (0.90) but degrades substantially on Taxi (-105.5). The Taxi result is consistent with the §IV.A trade-off discussion: Double Q-Learning trades over-estimation for under-estimation, and on tasks where the optimal policy requires optimism about long-horizon rewards (Taxi has a -1 step penalty incentivizing greedy long-horizon action), under-estimation hurts. This is exactly the regime that the bias-variance frontier discussion of §IV.A.E flags as underexplored.

Planning baselines (Value Iteration, Policy Iteration, MPI, CVPI). Value Iteration and Policy Iteration are not RL algorithms in the strict sense — they require known dynamics — but serve as optimality references. CVPI [48] consistently matches or exceeds the RL methods, confirming that the gap between learned and optimal policies is the cost paid for unknown dynamics. The comparison quantifies the cost: on Taxi, CVPI scores 5.5 against Q-Learning's -57.

MCTS. MCTS underperforms on Taxi (-6.2) and FrozenLake (0.00) in the limited-budget regime evaluated. The result reinforces a point visible throughout this paper: planning methods scale to small state spaces

in laboratory settings but degrade rapidly when the search budget is constrained relative to environment complexity.

D. Why tabular matters for the axis argument

The tabular results matter because they *test the axis attributions of §IV* in a setting where confounds are minimized. Double Q-Learning's tabular under-estimation pattern, visible on Taxi, is the same mechanism that drives §IV.A's bias-variance discussion in the deep setting — but in the tabular setting it can be observed cleanly. The convergence of tabular and deep evidence on the same mechanistic claims, when present, is one of the stronger methodological supports for the problem-first organization.

VII. Comparative Analysis of Deep Q-Learning Repositories

This section analyzes algorithmic coverage of Q-learning variants across six widely-used open-source deep RL repositories — Tianshou [6], XuanCe [7], CleanRL [8], DQN Zoo [9], Stable Baselines3 [10], and RLlib [11] — together with their stated design priorities. Table VII (Appendix A) reports a method-level coverage matrix; Table VIII summarizes design trade-offs.

A. Scope of this analysis

This paper's repository analysis is *taxonomic*: we report which algorithms each repository implements, with brief commentary on the design choices that shape coverage. We do *not* benchmark the repositories' implementations against one another; that question is addressed empirically by Hundal et al. [Hundal 2025], whose controlled comparison of PPO across five repositories shows substantial reproducibility variance even for a single algorithm under nominally equivalent configurations. Hundal et al.'s work sits in a broader reproducibility-crisis literature: Henderson et al. 2018 [*Deep Reinforcement Learning That Matters*] showed that hyperparameter sensitivity and seed variance in deep RL can rival the gap between published baselines and proposed methods; Engstrom et al. 2020 attributed several headline performance claims in policy-gradient methods to implementation details rather than algorithmic substance. Hundal et al.'s empirical-reproducibility audit and the taxonomic-coverage analysis here are complementary: practitioners selecting a repository care both *whether their algorithm is supported* (this paper's question) and *whether the implementation reproduces published results* (Hundal et al.'s question, against the standards of the reproducibility-crisis literature).

B. Axis-aware coverage

Reading Table VII through the eight axis-sections of §IV reveals patterns invisible in the method-type organization of the original matrix:

Well-supported axes. §IV.A (overestimation, via Double DQN / Dueling DQN) and §IV.D (distributional, via C51 / QR-DQN / IQN / FQF) are the best-covered axes across repositories. Tianshou alone implements all six method-level entries in these axes; XuanCe and RLlib follow. The bias toward overestimation- and distributional-method coverage reflects the empirical strength of these families during the 2016–2019 deep-RL boom, when most of the repositories were architected.

Patchily-supported axes. §IV.B (sample inefficiency) is covered unevenly: PER is in three of six repositories, MeDQN in zero, DQfD in zero, HER in three of six (but typically only as part of goal-conditioned baselines, not as a Q-learning enhancement). §IV.C (exploration) is similarly patchy: NoisyNet is in one of six repositories (RLlib), Bootstrapped DQN in zero, UCB Q-Ensemble in zero. The patchiness reflects the rapid evolution of these axes and the difficulty of integrating per-method changes into production-grade architectures.

Entirely-absent axes. §IV.F (multi-agent value decomposition) and §IV.G (distributed-scale methods) are functionally absent from the surveyed Q-learning repositories. RLlib supports multi-agent RL via its actor framework but does not implement VDN, QMIX, QPLEX, or QTRAN as native Q-learning methods. Ape-X, R2D2, and Agent57 have no implementations in any of the six repositories at the time of writing. The absence is structural: these methods require distributed infrastructure beyond the single-machine scope of most repositories.

§IV.E (offline RL) is in transition. None of the six surveyed repositories implement CQL, IQL, BCQ, or EDAC natively as of the draft's evaluation cutoff. Practitioners use d3rlpy, Clean-Offline-RL, or repository forks for these methods. The gap between mainstream-DRL repository coverage and the offline-RL practical landscape is one of the larger inconsistencies the axis-aware view exposes.

C. Methods absent from all six repositories

Eight methods covered in §IV are absent from *every* surveyed repository:

Method	Axis	§IV reference
Deep Recurrent Q (DRQN)	W7, scaling/adaptation	§IV.G.B.4
Cognitive Belief-Driven Q (CBDQ)	W3, brittle exploration	§IV.C.B.3
DQfD	W2, sample inefficiency	§IV.B.B.2
Memory-Efficient DQN	W2, sample inefficiency	§IV.B.B.3
Bootstrapped DQN	W3, brittle exploration	§IV.C.B.2
UCB Q-Ensemble	W3, brittle exploration	§IV.C.B.2
Ensemble Bootstrapping (EBQL)	W1, overestimation	§IV.A.B.2
Posterior Sampling DQN	W3, brittle exploration	§IV.C.B.3
Parallel Q-Learning (PQN)	W8, function-approx stability	§IV.H.B.3

The list overlaps substantially with the methods identified in §IV as productive directions for future work, suggesting that *implementation availability* is itself a bottleneck on practical adoption of methodological advances. This observation motivates the future-work proposal in §VIII for a community-maintained Q-learning–specific repository.

D. Tables VII and VIII — coverage and design trade-offs

Table 7: **Repository support for deep Q-learning algorithms, grouped by category.** • = implemented; ◦ = not implemented.

Category	Method (year)	Tianshou	XuanCe	CleanRL	SB3	RLlib	DQN Zoo
Statistical	Param Space Noise (2017)	◦	•	◦	◦	◦	◦
Statistical	C51 (2017)	•	•	•	◦	•	•
Statistical	NoisyNet (2018)	◦	◦	◦	◦	•	◦
Statistical	QR-DQN (2018)	•	•	◦	•	◦	•
Statistical	IQN (2018)	•	◦	◦	◦	◦	•
Statistical	FQF (2019)	•	◦	◦	◦	◦	◦
Q-Func. Comp.	Nature DQN (2015)	•	•	•	•	•	•
Q-Func. Comp.	DRQN (2015)	◦	◦	◦	◦	◦	◦
Q-Func. Comp.	Double DQN (2016)	•	•	◦	◦	•	•
Q-Func. Comp.	Dueling DQN (2016)	•	•	◦	•	•	◦
Q-Func. Comp.	Rainbow DQN (2018)	•	◦	◦	◦	•	•
Q-Func. Comp.	CBDQ (2025)	◦	◦	◦	◦	◦	◦
Memory/Replay	DQN (2013)	◦	◦	◦	◦	◦	◦
Memory/Replay	Prioritized ER (2016)	◦	•	◦	◦	•	•

Category	Method (year)	Tianshou	XuanCe	CleanRL	SB3	RLlib	DQN Zoo
Memory/Replay	PDD (2018)	◦	◦	◦	◦	◦	◦
Memory/Replay	MaDQN (2023)	◦	◦	◦	◦	◦	◦
Ensemble	Bootstrapped DQN (2016)	◦	◦	◦	◦	◦	◦
Ensemble	UCB Q-Ensemble (2018)	◦	◦	◦	◦	◦	◦
Ensemble	Ensemble Bootstrapping (2021)	◦	◦	◦	◦	◦	◦
Model-Based	Posterior Sampling DQN (2023)	◦	◦	◦	◦	◦	◦
Pure Q	Parallel Q (PQN, 2025)	◦	◦	◦	◦	◦	◦

Table 8: **Comparison of popular Deep RL repositories — design pros and cons.**

Repository	Pros	Cons
Tianshou	Dual API (high-level and low-level); emphasis on reproducibility with agent-level tests; native TensorBoard support	Logs/results not available for all algorithms; reproducibility guarantees not empirically verified across full benchmark suite
XuanCe	Modular YAML config files; W&B integration for hyperparameter tuning; high modularity for code reuse	Default hyperparameters often diverge from original papers; incomplete support across environments; steep learning curve
CleanRL	Single-file implementations; easy to audit and understand; lightweight and minimal dependencies	Limited support for large-scale experiments; less modular, harder to extend
Stable Baselines3	Clean API with sklearn-style interface; well-maintained and widely adopted; compatible with VecEnv, Gym, etc.	Focused more on policy-gradient methods; limited Q-learning variants beyond DQN
RLlib	Distributed training out-of-the-box; Ray Tune integration; production-grade scalability	High complexity and heavyweight; harder to debug or customize individual components
DQN Zoo	Faithful reimplementation of DQN variants; reproducibility aligned with DeepMind practices; highly organized training logs and configs	Focused exclusively on DQN family; less beginner-friendly

Reading Table VIII, the design priorities of each repository emerge:

- **Tianshou** and **XuanCe** prioritize coverage breadth across methods at the cost of single-method depth or reproducibility guarantees.
- **CleanRL** prioritizes single-file readability at the cost of modularity, which limits coverage of compositional methods (Rainbow, for instance, is harder to express in CleanRL's single-file architecture).
- **Stable Baselines3** prioritizes API stability and broad user adoption, with the cost of slower integration of new methods.
- **RLlib** prioritizes distributed scalability and production use, with the cost of complexity and difficulty in debugging individual algorithms.
- **DQN Zoo** prioritizes reproduction fidelity to DeepMind's published results, with the cost of restricted scope (DQN family only).

The design priorities are not in conflict but represent different positions on a Pareto frontier: a repository optimizing for one property necessarily relaxes another. The practitioner-side implication is that selection of a repository is itself an axis decision: a researcher exploring overestimation methods is well- served by Tianshou; a practitioner deploying at scale is well-served by RLlib; an educator demonstrating algorithms is well-served by CleanRL. No single repository serves the field optimally — and the axis-aware coverage gaps reported above suggest that no repository currently serves the Q-learning research community comprehensively.

E. Toward a Q-learning–specific repository

The axis-aware analysis above motivates a specific deliverable: **a community-maintained repository focused exclusively on Q-learning and its deep variants**, providing unified training scripts, standardized benchmarks, modular algorithm implementations, and shared logging tools. Such a repository would prioritize coverage of the methods identified in §VII.C as absent from all existing repositories, with empirical reproducibility audits in the style of Hundal et al. as a release-quality gate.

Section VIII discusses this proposal as the paper's principal forward-looking deliverable.

VIII. Conclusion and Future Work

A. Summary

This paper presents a problem-first survey of Q-learning and deep Q-learning, reorganizing three decades of methodological development around the eight foundational weaknesses of vanilla Q-learning introduced in §II.B: overestimation bias, sample inefficiency, brittle exploration, reward sparsity and credit assignment, distribution shift, multi-agent coordination, slow adaptation, and function-approximation instability.

For each weakness, §IV surveys the families of methods that respond:

- **§IV.A** — overestimation control via decoupling (Double Q, Double DQN) and ensembles (EBQL, REDQ).
- **§IV.B** — sample efficiency via prioritized sampling (PER), demonstration augmentation (DQfD), memory-efficient consolidation (MeDQN), and goal relabeling (HER).
- **§IV.C** — directed exploration via noise injection (NoisyNet, Parameter Space Noise), ensemble disagreement (Bootstrapped DQN, UCB Q-Ensemble), belief modulation (CBDQ, PSDQN), and intrinsic motivation (RND, Go-Explore).
- **§IV.D** — credit assignment via multi-step returns and the distributional family (C51, QR-DQN, IQN, FQF).
- **§IV.E** — offline RL via policy constraint (BCQ, AWAC, BRAC), value penalty (CQL), expectile regression (IQL), and ensemble diversification (EDAC).
- **§IV.F** — cooperative value decomposition via additive (VDN), monotonic mixing (QMIX), duplex dueling (QPLEX), and constraint- free (QTRAN) factorizations.
- **§IV.G** — scaling and adaptation via distributed actor-learner architectures (Ape-X, R2D2), bandit-controlled exploration meta-policies (Agent57), and meta-learning on the Q-function.
- **§IV.H** — function-approximation stability via target networks, architectural decomposition (Dueling), normalization recipes (PQN), and consolidation losses (MeDQN, Munchausen DQN).

Sections V and VI present axis-stratified empirical evidence, showing that methods targeting weakness W_i excel on the task categories that diagnose W_i and remain near baseline elsewhere. Section VII analyzes algorithmic coverage across six open-source repositories, identifying nine methods absent from all surveyed codebases — a gap that motivates the principal forward-looking deliverable of this paper.

B. A community Q-learning repository

The repository coverage analysis of §VII surfaces a concrete implementation gap: nine methods covered in §IV — DRQN, CBDQ, DQfD, MeDQN, Bootstrapped DQN, UCB Q-Ensemble, EBQL, Posterior Sampling DQN, and PQN — are absent from *every* surveyed open-source repository. Across the new axes introduced in §IV.E, §IV.F, and §IV.G, the gap is broader still: most offline-RL, multi-agent value-decomposition, and distributed-Q methods have no implementation in any of the six repositories analyzed.

We propose the development of a **dedicated, community-maintained repository focused exclusively on Q-learning and its deep variants**. The repository would have four design priorities:

1. **Axis-aware coverage** — implementations organized by the eight weakness axes, prioritizing methods absent from existing repositories, with a coverage matrix maintained as the public roadmap.
2. **Unified training scripts** — a common interface across methods permitting like-for-like comparison under matched evaluation protocols.
3. **Standardized benchmarks** — Atari, classic control, D4RL, and SMAC included as canonical benchmark suites, with reference results for each implemented method.
4. **Empirical reproducibility audits** — each method release gated on reproduction of the original paper's headline results, in the style of Hundal et al. [Hundal 2025].

The repository's initial roadmap would prioritize the nine methods in §VII.C as the first implementation targets, followed by the offline-RL family (CQL, IQL, EDAC), the multi-agent value- decomposition family (QMIX, QPLEX, VDN), and the distributed-Q family (Ape-X, R2D2, Agent57). Cross-references between the repository's documentation and this paper's §IV sections would let practitioners navigate from "I face

problem X" (this paper's axis) to "available implementations addressing X" (the repository's coverage matrix).

C. Open research directions, by axis

The "Open Questions" subsections (E) of each §IV axis-section collectively summarize the field's frontier. We highlight four cross-axis directions of particular promise:

The compute-vs.-algorithmic tradeoff. The empirical pattern in §V suggests that distributed scaling (§IV.G) accounts for the largest single performance increase in deep RL since DQN, but PQN's single-machine results suggest the algorithmic axes are far from saturated. A systematic study of which weaknesses are *compute-resoluble* versus *algorithmically-resoluble* — at fixed compute budgets — would clarify where research effort is best directed.

Cross-axis composition. Frontier agents (Agent57, NGU) explicitly compose mechanisms from multiple axes. A formal account of which axis combinations are synergistic, redundant, or antagonistic would convert the empirical pattern into a design principle. The Rainbow ablation [28] provides one such account on a smaller method set; a broader cross-axis composition study would extend it.

The offline-online interface. Methods bridging offline and online learning (AWAC, Cal-QL) address a regime increasingly central to practical deployment but theoretically underexplored. The interface is itself a candidate weakness axis the field may recognize in coming years.

Q-learning under real-world distribution shift. §IV.E treats distribution shift as a *training* regime: the data was collected by one or more behavior policies and is then fixed. A deployment-side analog — the learned policy meets a state distribution different from the one it was trained on — is largely unaddressed. Sim-to-real transfer in robotic Q-learning, OOD robustness against perturbations, and online policy correction at deployment time all live in this regime. The newer benchmarks of §V.I (ProcGen for procedural variation, CARL for context generalization, RLBench for sim-to-real) begin to provide evaluation infrastructure, but no unified Q-learning treatment of *deployment* distribution shift exists. This is the axis where the gap between Atari-evaluated methods and real-world deployment is most visible, and where Q-learning's value-based perspective could productively engage with robotics, control, and operations-research literature that has developed parallel tools.

D. Closing

Q-learning's longevity as a foundational paradigm rests on its combination of conceptual simplicity and methodological extensibility. The thirty-five years since [Watkins 1989] have produced a tooling ecosystem rich enough to deserve organized treatment but specialized enough that no general-RL survey can capture its texture. This paper offers the first multi-axis problem-first synthesis of that ecosystem, paired with empirical and implementation analyses that support the structural claim.

The methodological landscape this paper traces is not closed. New axes will emerge — the offline-online interface and the meta-RL boundary are two candidates — and existing axes will continue to admit new mechanisms. The problem-first organization is intended to be extensible: new methods slot into existing axes, and new axes can be added as the field recognizes them. The framework's value is precisely that it survives those additions.

Appendix A. Legacy Indexer: Six Categories vs. Eight Axes

This appendix supports readers approaching the paper through the conventional method-type taxonomy used in prior Q-learning surveys [12]–[15]. The eight-axis problem-first organization of §IV is the analytical spine of this paper, but the six-category taxonomy is preserved as a secondary indexing system — in Tables II/III row groupings, in this appendix's mapping below, and in inline tags within each §IV subsection.

A reader interested in *all distributional methods* (a method-type view) can navigate via the *Statistical Methods* row of Table II and the corresponding rows of this appendix table. A reader interested in *all methods addressing brittle exploration* (an axis view) navigates via §IV.C. The two views are complementary; neither is privileged for navigation.

A.1. Full method-to-axis mapping

For each method covered in §IV, the table below shows:

- **Legacy category** — the row grouping in the conventional method-type taxonomy used by prior Q-learning surveys and reflected in the row spine of Tables II/III: *Statistical Methods*, *Q-Function Computation*, *Memory/Replay*, *Ensemble-Based*, *Model-Based*, *Pure Q-Learning (Minimal)*.
- **Primary axis** — the weakness W_i from §II.B that the method's contribution principally addresses. The axis-section in §IV where the method receives full treatment.
- **Secondary axes** — additional weaknesses the method incidentally addresses or partially mitigates. Sections where the method is cross-referenced but not centrally treated.

Method (year)	Legacy category	Primary axis	Secondary axes
Parameter Space Noise (2017)	Statistical	§IV.C (W3)	—
NoisyNet (2018)	Statistical	§IV.C (W3)	—
C51 (2017)	Statistical	§IV.D (W4)	§IV.A
QR-DQN (2018)	Statistical	§IV.D (W4)	§IV.A
IQN (2018)	Statistical	§IV.D (W4)	§IV.A
FQF (2019)	Statistical	§IV.D (W4)	§IV.A
Double Q-Learning (2010)	Q-Function Comp.	§IV.A (W1)	§V foundations
Nature DQN (2015)	Q-Function Comp.	§IV.H (W8)	§V foundations
Deep Recurrent Q (DRQN, 2015)	Q-Function Comp.	§IV.G (W7)	—
Double DQN (2016)	Q-Function Comp.	§IV.A (W1)	—
Dueling DQN (2016)	Q-Function Comp.	§IV.H (W8)	§IV.A (incidental)
Rainbow (2018)	Q-Function Comp.	§IV.B (W2)	§IV.A, §IV.C, §IV.D, §IV.H
MCTS for FrozenLake (2024)	Q-Function Comp.	§V planning baseline	—
Cognitive Belief-Driven Q (2025)	Q-Function Comp.	§IV.C (W3)	§IV.A
DQN (2013)	Memory/Replay	§IV.B (W2)	§V foundations
Prioritized ER (PER, 2016)	Memory/Replay	§IV.B (W2)	—
DQfD (2018)	Memory/Replay	§IV.B (W2)	§IV.C (demos for exploration)
Memory-Efficient DQN (MeDQN, 2023)	Memory/Replay	§IV.B (W2)	§IV.H (consolidation)

Method (year)	Legacy category	Primary axis	Secondary axes
Bootstrapped DQN (2016)	Ensemble-Based	§IV.C (W3)	§IV.A
Ensemble Bootstrapped Q (EBQL, 2021)	Ensemble-Based	§IV.A (W1)	§IV.C
UCB Q-Ensemble (2018)	Ensemble-Based	§IV.C (W3)	§IV.A
Value/Policy Iteration (1960s)	Model-Based	§V foundations	—
Bayesian Q-Learning (1998)	Model-Based	§V foundations	§IV.C (Thompson sampling)
Expected SARSA (2009)	Model-Based	§V foundations	—
Posterior Sampling DQN (2023)	Model-Based	§IV.C (W3)	§IV.A
Q-Learning (Watkins 1992)	Pure Q-Learning	§V foundations	—
SARSA (1994)	Pure Q-Learning	§V foundations	—
Multi-Step Q-Learning (1996)	Pure Q-Learning	§IV.D (W4)	§V foundations
Neural Fitted Q (NFQ, 2005)	Pure Q-Learning	§V foundations	§IV.B (batch updates)
Parallel Q Learning (PQN, 2025)	Pure Q-Learning	§IV.H (W8)	§IV.G (parallelism)

A.2. Methods outside the legacy six-category structure

Eighteen methods covered in §IV have no row in the conventional six-category taxonomy and do not appear in Tables II/III. They are treated in §IV under the new axes; for navigation, the table below maps them to the legacy taxonomy as they *would* have been classified if the conventional categories were extended to admit them.

Method (year)	Would-be legacy category	Primary axis
Maximin Q-Learning (2020)	Q-Function Comp.	§IV.A (W1)
REDQ (2021)	Ensemble-Based	§IV.A (W1)
HER (2017)	Memory/Replay	§IV.B (W2)
RND (2018)	Statistical (intrinsic motivation)	§IV.C (W3)
Go-Explore (2019/2021)	Memory/Replay (archive)	§IV.C (W3)
BCQ (2019)	New category — Offline RL	§IV.E (W5)
BRAC (2019)	New category — Offline RL	§IV.E (W5)
AWAC (2020)	New category — Offline RL	§IV.E (W5)
CQL (2020)	New category — Offline RL	§IV.E (W5)
IQL (2021)	New category — Offline RL	§IV.E (W5)
EDAC (2021)	New category — Offline RL + Ensemble	§IV.E (W5)
VDN (2018)	New category — Multi-Agent	§IV.F (W6)
QMIX (2018)	New category — Multi-Agent	§IV.F (W6)
QPLEX (2020)	New category — Multi-Agent	§IV.F (W6)
QTRAN (2019)	New category — Multi-Agent	§IV.F (W6)
Ape-X (2018)	New category — Distributed	§IV.G (W7)
R2D2 (2019)	New category — Distributed + Recurrent	§IV.G (W7)
Agent57 (2020)	New category — Distributed + Meta-policy	§IV.G (W7)

Method (year)	Would-be legacy category	Primary axis
MAML-Q (2017/2019)	New category — Meta-RL	§IV.G (W7)
Munchausen DQN (2020)	Q-Function Comp. (would have been)	§IV.H (W8)

The "New category" entries are the methods that *cannot* be cleanly placed in the six legacy categories — Q-learning families that emerged after the legacy taxonomy was conventionalized. Their absence from the legacy structure is one of the central arguments in §I for the structural pivot: the six categories were defined when these families did not yet exist, and accommodating them in the taxonomy requires adding categories that were never load-bearing originally. The eight-axis structure has principled homes for all eighteen methods without category-set expansion.

A.3. Reverse view — legacy category to §IV section coverage

The table below inverts A.1: for each legacy category, what fraction of its constituent methods land in each axis-section of §IV.

Legacy category	Primary axis distribution
Statistical Methods (6 methods)	2 → §IV.C (Param Noise, NoisyNet); 4 → §IV.D (C51, QR-DQN, IQN, FQF)
Q-Function Computation (8 methods)	2 → §IV.A (Double Q, Double DQN); 2 → §IV.H (Nature DQN, Dueling); 1 → §IV.B (Rainbow); 1 → §IV.C (CBDQ); 1 → §IV.G (DRQN); 1 → §V (MCTS)
Memory/Replay (4 methods)	4 → §IV.B (DQN, PER, DQfD, MeDQN)
Ensemble-Based (3 methods)	1 → §IV.A (EBQL); 2 → §IV.C (Bootstrapped DQN, UCB Q-Ensemble)
Model-Based (4 methods)	1 → §IV.C (PSDQN); 3 → §V (VI/PI/MPI, Bayesian Q, Expected SARSA)
Pure Q-Learning (5 methods)	4 → §V (Q-learning, SARSA, NFQ, multi-step); 1 → §IV.H (PQN)

Three patterns are visible from the inverted view:

1. *Memory/Replay* is the most internally coherent legacy category: all four methods land in §IV.B (sample inefficiency). The legacy category and the axis category coincide here.
2. *Ensemble-Based* fragments: ensembles serve overestimation control (§IV.A) and exploration (§IV.C) via the same architectural mechanism but different mechanisms-of-use. The axis view distinguishes them; the legacy view does not.
3. *Statistical Methods* fragments into noise-based exploration (§IV.C) and distributional credit-assignment (§IV.D), the sharpest single demonstration of why the conventional method-type taxonomy conflates mechanism with effect.

These patterns are the data backing the problem-first organization. The appendix lets readers verify the claim method-by-method.

Appendix B. Notation and Selected Derivations

This appendix consolidates symbol conventions and supplies the derivations the main text references but does not work through. The intent is to keep §IV bodies focused on mechanism and trade-off while still grounding the claims for readers who want the underlying math.

B.1. Notation

The symbols below appear throughout the paper. Section-local deviations from these conventions are flagged at first use.

Symbol	Meaning
s, a	State and action at the current time step
s', a'	Successor state and action
r	Immediate reward
$\gamma \in [0, 1)$	Discount factor
$\mathcal{S}, \mathcal{A}$	State and action spaces
$P(s' \mid s, a)$	Transition probability function
$R(s, a)$	Reward function (expected immediate reward)
π	Policy; $\pi(a \mid s)$ for stochastic, $\pi(s)$ for deterministic
V^π, Q^π	State-value and action-value functions under π
V^*, Q^*	Optimal value and action-value functions
$\mathcal{T}, \mathcal{T}^*$	Bellman and Bellman-optimality operators
θ	Online network parameters
θ^-	Target network parameters
α	Learning rate
ϵ	ε-greedy exploration parameter; or a generic small constant
ε_t	Noise variable at step t (disambiguated by context)
$\mathcal{D}$	Experience replay buffer
$Z(s, a)$	Return distribution at (s, a)
δ	Temporal-difference error
K	Number of ensemble members
τ	Quantile fraction $\in [0, 1]$; or a temperature parameter
$\mathcal{T}_i$	i-th task in a meta-learning task distribution

Boldface lower-case (e.g., $\mathbf{a}, \mathbf{s}$) denotes joint quantities in multi-agent settings (§IV.F).

B.2. Maximum-of-noisy-estimators bound (referenced by §IV.A)

§IV.A invokes the result that the maximum of noisy estimators is biased upward as the source of Q-learning's overestimation. [Smith & Winkler 2006, *The Optimizer's Curse*] formalize this for the related setting of decision analysis. The relevant inequality for our purposes:

Let $\hat{Q}_a = Q^*_a + \xi_a$ for $a \in \mathcal{A}$, where $\{\xi_a\}$ are zero-mean noise variables with positive variance. Then

$$\mathbb{E}\Big[\max_{a \in \mathcal{A}} \hat{Q}_a\Big] \geq \max_{a \in \mathcal{A}} Q^*_a,$$

with strict inequality when $|\mathcal{A}| \geq 2$ and at least two ξ_a have positive variance.

Proof sketch. For each fixed realization $\{\xi_a\}$, the function $\xi \mapsto \max_a(Q_a^* + \xi_a)$ is convex in ξ. Jensen's inequality applied to the expectation over ξ yields

$$\mathbb{E}_\xi\Big[\max_a(Q_a^* + \xi_a)\Big] \geq \max_a \mathbb{E}_\xi[Q_a^* + \xi_a] = \max_a Q_a^*.$$

Strict inequality follows from non-degeneracy of the maximum under random perturbation. Iterated through the Bellman backup, the bias compounds, motivating the decoupling methods of §IV.A.B.1 and the ensemble methods of §IV.A.B.2.

B.3. Categorical distributional projection (referenced by §IV.D)

C51 [21] maintains a return distribution as a categorical distribution over $N = 51$ fixed atoms in $[V_{\min}, V_{\max}]$. After applying the distributional Bellman update $\hat{\mathcal{T}} Z(s,a) = r + \gamma Z(s', a^*)$, the resulting distribution generally has support outside the fixed atom set; the projection operator Φ maps it back.

Definition. Let $\{z_i\}_{i=0}^{N-1}$ be the atom locations with $z_i = V_{\min} + i\Delta z$, $\Delta z = (V_{\max} - V_{\min})/(N-1)$. Given a candidate distribution with probability mass p_j at locations $\tilde{z}_j$, the projection Φ produces a distribution on $\{z_i\}$ with probabilities

$$(\Phi(\tilde{z}, p))_i = \sum_j p_j \cdot \max\Big(0, 1 - \frac{|\tilde{z}_j - z_i|}{\Delta z}\Big),$$

with $\tilde{z}_j$ clipped to $[V_{\min}, V_{\max}]$ before projection. Probability mass is conserved by construction (the weights of any $\tilde{z}_j$ sum to 1 across the two adjacent atoms).

The KL loss in C51's training objective is then

$$\mathcal{L}_{\text{C51}} = \mathrm{KL}(\Phi(\hat{\mathcal{T}} Z_{\bar{\theta}}) \,\|\, Z_\theta),$$

where Z_θ is the predicted distribution under the online parameters and $\bar{\theta}$ are the target parameters. Note that the projection step is *non-contractive* in the KL norm — the contraction results of §IV.I.C.1 hold in the Wasserstein metric instead.

B.4. Wasserstein contraction of the distributional Bellman operator (referenced by §IV.I.C.1)

[Bellemare, Dabney & Munos 2017] establish that the distributional Bellman operator $\mathcal{T}_\pi^d$ is a γ-contraction in the maximal form of the Wasserstein-p distance over return distributions.

Statement. For two return distribution functions $Z_1, Z_2 : \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathbb{R})$,

$$\bar{W}_p(\mathcal{T}_\pi^d Z_1, \mathcal{T}_\pi^d Z_2) \leq \gamma \bar{W}_p(Z_1, Z_2),$$

where $\bar{W}_p(Z_1, Z_2) = \sup_{s,a} W_p(Z_1(s,a), Z_2(s,a))$ and W_p is the Wasserstein-p distance between scalar distributions.

Proof sketch. The distributional Bellman operator decomposes into three steps: (i) sample a transition (s, a, r, s') from P; (ii) shift and scale the return distribution at the next state by the affine map $z \mapsto r + \gamma z$; (iii) integrate over the policy $\pi(a' \mid s')$.

Wasserstein-p is preserved under expectation (step iii) and satisfies $W_p(\delta_r+\gamma X, \delta_r+\gamma Y) = \gamma W_p(X, Y)$ under affine transformations (step ii). Taking the supremum over (s, a) on both sides yields the stated contraction.

The KL divergence used by C51's loss does *not* satisfy an analogous contraction; this is the technical motivation for QR-DQN's switch to quantile-regression losses, where the Wasserstein-∞ contraction is preserved [Rowland et al. 2018].

B.5. Pessimism lower-bound argument for offline RL (referenced by §IV.E and §IV.I.C.3)

Pessimistic value iteration constructs a lower confidence bound (LCB) on Q-values at each Bellman backup. The canonical version due to [Jin, Yang & Wang 2021] runs

$$\hat{Q}^{k+1}(s,a) = r(s,a) + \gamma \sum_{s'} \hat{P}(s' \mid s,a) \max_{a'} \hat{Q}^k(s',a') - b(s,a),$$

where $b(s,a)$ is a *bonus* term inversely proportional to the visitation count $n(s,a)$ in the offline dataset. Concretely, $b(s,a) = c\sqrt{1/n(s,a)}$ for a problem-dependent constant c.

Suboptimality bound. Let $\hat{\pi}$ be the policy greedy with respect to the fixed point $\hat{Q}^*$ of the LCB iteration, and let π be any policy that the offline dataset supports. Then

$$V^{\pi}(s_0) - V^{\hat{\pi}}(s_0) \le \tilde{O}\Big(\sqrt{\tfrac{1}{n}}\Big),$$

where n is the total dataset size, s_0 is the initial state, and the $\tilde{O}$ hides logarithmic factors and the problem horizon.

The key insight. Pessimism makes $\hat{Q}^*(s,a)$ a *lower bound* on V^{π} for any supported policy. The greedy policy with respect to $\hat{Q}^*$ therefore competes with the best *supported* policy, not with the unrestricted optimum. Crucially, this bound holds *without coverage assumptions* on the behavior policy — it adapts gracefully when the dataset covers only a small subset of the state space.

CQL's [Kumar et al. 2020] conservative penalty is a tractable relaxation of this LCB construction: rather than maintaining explicit confidence bounds, CQL adds a regularizer that drives $\hat{Q}$ down at OOD actions, achieving the same pessimism guarantee under specific assumptions on the regularization weight α. IQL's expectile regression achieves a related effect via the asymmetric quantile loss without explicit OOD-action sampling.

B.6. QPLEX IGM completeness (referenced by §IV.F and §IV.I.C.5)

The IGM (Individual-Global-Max) constraint requires that the joint argmax of Q_{tot} coincide with the per-agent argmaxes of $\{Q_i\}$:

$$\arg\max_{\mathbf{a}} Q_{\text{tot}}(\mathbf{s},\mathbf{a}) = (\arg\max_{a_1} Q_1, \dots, \arg\max_{a_N} Q_N).$$

QMIX's sufficient condition. Monotonic mixing — $\partial Q_{\text{tot}}/\partial Q_i \ge 0$ for all i — implies IGM but is not necessary. There exist IGM-compatible Q_{tot} that QMIX cannot represent.

QPLEX's claim [Wang et al. 2020]. Decomposing $Q_{\text{tot}} = \sum_i V_i(\tau_i) + A_{\text{tot}}(\mathbf{s},\mathbf{a})$ with A_{tot} produced by a duplex-dueling mixer admits *every* IGM-compatible Q_{tot}. The proof constructs an explicit duplex-dueling

representation for any IGM-satisfying joint Q_{tot} by reducing to per-agent advantage representations that satisfy IGM individually.

Significance. The result identifies QPLEX's representational capacity as the ceiling of value-decomposition methods. QTRAN's auxiliary-loss approach reaches the same ceiling via a different construction but at the cost of training instability discussed in §IV.F.B.4. The representation–training-dynamics gap that QTRAN exposes is precisely the open question listed in §IV.I.E item 4.

B.7. Tabular Q-learning convergence (referenced by §V and §IV.I.B.1)

The canonical Watkins & Dayan (1992) convergence proof for tabular Q-learning runs as follows.

Setting. A finite MDP $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ with bounded rewards. The Q-learning update at step t is

$$Q_{t+1}(s_t, a_t) = (1 - \alpha_t(s_t, a_t))Q_t(s_t, a_t) + \alpha_t(s_t, a_t)(r_t + \gamma \max_{a'} Q_t(s_{t+1}, a')).$$

Convergence conditions.

1. **Coverage:** Every $(s, a) \in \mathcal{S} \times \mathcal{A}$ is visited infinitely often.
2. **Learning rate decay:** $\sum_t \alpha_t(s, a) = \infty$ and $\sum_t \alpha_t(s, a)^2 < \infty$ for every (s, a).
3. **Bounded rewards:** $|r_t| \leq R_{\max}$ for all t.

Theorem. Under conditions 1–3, $Q_t \to Q^*$ with probability 1.

Proof outline. Define the Bellman optimality operator $(\mathcal{T}^*Q)(s, a) = R(s, a) + \gamma \mathbb{E}_{s'}[\max_{a'} Q(s', a')]$. The Q-learning update can be rewritten as a stochastic approximation to the fixed-point iteration $Q \leftarrow \mathcal{T}^*Q$. Since $\mathcal{T}^*$ is a γ-contraction in the ℓ_∞ norm and has unique fixed point Q^*, classical stochastic approximation theory [Robbins-Monro 1951; Tsitsiklis 1994] yields almost-sure convergence under conditions 1–3.

Limitations. The proof requires *tabular* representation: each $Q(s, a)$ is a separately-updated scalar, and the contraction argument uses the unique-fixed-point property of $\mathcal{T}^*$. Function approximation breaks both requirements — updates at one (s, a) now affect others, and the Bellman operator composed with function-approximation projection is not in general a contraction. This is the foundational source of the deadly-triad instability of §IV.H.A and §IV.I.B.2.

B.8. Pointers for further reading

The derivations above are the ones most directly load-bearing for the claims in §IV. Readers seeking deeper treatment are referred to the following canonical sources:

- *Tabular Q-learning convergence and TD methods:* Sutton & Barto 2018, *Reinforcement Learning: An Introduction* (2nd ed.), chapters 6 and 7.
- *Function approximation and the deadly triad:* Sutton & Barto 2018, chapter 11; Tsitsiklis & Van Roy 1997.
- *Distributional RL theory:* Bellemare, Dabney & Rowland 2023, *Distributional Reinforcement Learning* (MIT Press).
- *Offline RL theory and pessimism-based methods:* Levine et al. 2020 survey; Jin, Yang & Wang 2021.
- *Multi-agent value decomposition:* Wang et al. 2020 (QPLEX); Albrecht & Stone 2018, *Multiagent Learning* (recent survey).